

TECHNISCHE UNIVERSITÄT ILMENAU
Institute of Applied Computer Science
Department of Computer Science and Automation
Databases and Information Systems Group

Master's thesis

in the degree programme
Research in Computer and Systems Engineering (M. Sc.)

Design and Evaluation of a Database-Centric Agentic Architecture for Nursing Home Evacuation Coordination in Storm Surge Scenarios

presented by

Anik Dewanje
Matriculation No. 64746

Responsible professor: Prof. Dr.-Ing. habil. Kai-Uwe Sattler

Supervisor: M. Sc. Saba Zamankhani

Ilmenau, 16 October 2026

Abstract

Nursing homes near the coast have to move their residents before a storm surge arrives. This needs several organizations to work together, and the plan often changes on the way. Simulations of this task are usually scripts that cover only the disruptions their authors expected. Agents built from large language models (LLMs) could, in principle, also handle disruptions that nobody scripted.

We design a database-centric agentic architecture and compare it with a scripted baseline that follows the German DV 100 doctrine. The case is the evacuation of the Augustinum nursing home in Hamburg with 100 residents. Ten agents represent people at eight organizations. Six of them reason with an LLM, and all coordinate through one PostgreSQL database. Both systems write to one shared event log, and one deterministic scorer reads it. We compared them on seven scenarios under a pre-registered protocol.

Both confirmatory hypotheses failed. The baseline sheltered every resident in all 50 runs with a disruption it was not written for, but it did not adapt. A controller that ignores every disruption scores the same, because no disruption changed what the scorer counts. The agentic system did react. On 16 rubric criteria its agents matched the baseline on 5 and fell short on 5. The best configuration sheltered only 89.4% of the residents. In every scenario with such a disruption it scored below a controller that ignores the disruption, partly because it lost residents even without one. For two of the three models, most lost residents were never delivered to a shelter, while failing runs dispatched as many missions as successful ones. The more often a run revised its plan, the fewer residents it delivered. Coordination worked, with 93.8% of the requests between agents answered. Our leading hypothesis is a failure of commitment, but we did not measure comprehension. The agentic bus drivers were live LLM agents where the baseline used a simulator, so a driver may have dropped a lost leg. Since no scenario required adaptation, these results show who won here, not whether adaptation can ever pay off.

From this we derive design considerations. An agent should not cancel a bus run once it has started, every delivery should be confirmed, and test scenarios should be ones where ignoring the disruption costs residents. As a next step, we propose to rerun the same scenarios with agents that may revise their plan only a few times.

Acknowledgements

I would like to thank Prof. Dr.-Ing. habil. Kai-Uwe Sattler for the chance to write this thesis in the Databases and Information Systems Group, and for the guidance along the way. I am also grateful to M. Sc. Saba Zamankhani for the supervision, the time given to this work and the helpful feedback on the design and the evaluation.

This thesis was written at Absolute Software GmbH in Hamburg, as part of the company's work in the RESCUE-MATE project. RESCUE-MATE is a research project funded under the German programme Research for Civil Security. It studies how rescue forces can be supported in complex crisis situations, and its main scenario is a severe storm surge in Hamburg that requires large evacuations near the Elbe. This scenario gave my thesis its case. I thank Absolute Software GmbH for the opportunity to work on this topic within the project, and for the support I received during the whole thesis. My special thanks go to my company supervisor, Mr. Rüdiger Höfert, for the trust, the encouragement and the many helpful discussions along the way.

Contents

Abstract i

Acknowledgements ii

1. Introduction 1
- 1.1. Motivation . . . 1
- 1.2. Problem Statement . . . 1
- 1.3. Research Questions . . . 2
- 1.4. Contributions . . . 3
- 1.5. Scope and Delimitations . . . 4
- 1.6. Thesis Outline . . . 4

2. Background and Related Work 5
- 2.1. Scripted Coordination and Unanticipated Events . . . 5
- 2.2. LLM-Driven Agents . . . 6
- 2.3. Intentions and Commitment . . . 6
- 2.4. Limits of LLM Planning . . . 7
- 2.5. Shared-State Coordination . . . 7
- 2.6. Related Work and the Gap . . . 7

3. Case Study: Storm Surge in Hamburg 9
- 3.1. Scenario Overview . . . 9
- 3.2. Organizational Agents . . . 9
- 3.3. Coordination Storyline . . . 10
- 3.4. Disruption Scenarios . . . 11
- 3.5. Assumptions and Simplifications . . . 11

4. Database-Centric Agentic System 14
- 4.1. Architecture Overview . . . 14
- 4.2. The Agent Specification Template . . . 15
- 4.3. The Wake Cycle . . . 16
- 4.4. Prompt Structure . . . 17
- 4.5. Runtime Ownership and Ground Truth . . . 19
- 4.6. Time and the Simulation Clock . . . 20
- 4.7. Agent Roster . . . 21
- 4.8. Replanning by Plan Rewrite . . . 22
- 4.9. Action Vocabulary . . . 23

5. Workflow-Based Baseline 24
- 5.1. Design . . . 24

5.2. The Command Cycle . . . 25
5.3. Handling Disruptions . . . 26
5.4. The Execution Model . . . 27
5.5. Freeze and Determinism . . . 28
5.6. What the Baseline Cannot Do . . . 28

6. Data Architecture 29
6.1. Design Principles . . . 29
6.2. Data Model . . . 29
6.3. Append-Only Event Log . . . 31
6.4. The Event Log as Evaluation Backbone . . . 33
6.5. Runtime State and Message Passing . . . 34
6.6. Live Notifications . . . 35
6.7. Design Alternatives . . . 36
6.8. External Data Sources . . . 36
6.9. Determinism and Replay . . . 36

7. Observation Interface 38
7.1. Purpose . . . 38
7.2. Backend . . . 38
7.3. Frontend . . . 40
7.4. The Acknowledgement Channel . . . 42
7.5. Runs Without a Browser . . . 42
7.6. Summary . . . 42

8. Method and Evaluation Design 43
8.1. Study Design . . . 43
8.2. Hypotheses . . . 45
8.3. Scenarios and the Disruption Firewall . . . 45
8.4. Measuring Survival . . . 46
8.5. The Three Gates . . . 48
8.6. Secondary Measures . . . 50
8.7. Statistical Analysis . . . 52
8.8. Pre-Registration and Amendments . . . 53
8.9. Availability of Code and Data . . . 54

9. Results 55
9.1. Execution Summary . . . 55
9.2. Anticipated Disruptions . . . 56
9.3. Novel Disruptions . . . 57
9.4. What the Losses Were Made Of . . . 58
9.5. Decision Quality . . . 61
9.6. Response Time and Cost . . . 63
9.7. Model Sweep and Capability Threshold . . . 64
9.8. Integrity of the Reported Numbers . . . 64
9.9. Summary of Findings . . . 65

10. Methods Critique **66**
10.1. The Defect . . . 66
10.2. Gate 0: Treatment Validity . . . 67
10.3. What Is Withdrawn and What Stands . . . 68
10.4. The Pre-Registered Primary Was Underpowered by Construction . . . 69
10.5. Limitations Declared, With the Position Taken on Each . . . 69

11. Discussion **71**
11.1. RQ1: Did the Agentic Architecture Win? . . . 71
11.2. RQ2: Why the Agentic System Lost . . . 71
11.3. Cost and Transferability . . . 73

12. Design Considerations for Agentic Emergency Simulation **74**
12.1. Agent Architecture: Commit, Then Confirm . . . 74
12.2. Scenario Design: Make the Problem Real . . . 74
12.3. Evaluation Design: Test the Test First . . . 75
12.4. Cost, and the Limits of These Considerations . . . 75

13. Conclusion and Future Work **76**
13.1. Summary . . . 76
13.2. Contributions Revisited . . . 77
13.3. Future Work . . . 77
13.4. Closing Remarks . . . 78

Bibliography **79**

A. Agent Prompts **82**
A.1. Persona . . . 82
A.2. Contract . . . 83
A.3. Playbook for a New Message . . . 86
A.4. The Other Two Playbooks . . . 88

B. Scenario Disruption Messages **89**
B.1. S02, Road Closure, 10:20 . . . 89
B.2. S04, Medical Emergency in Flight, 10:35 . . . 89
B.3. S05, Unverified Persons Report, 10:30 and 11:05 . . . 89
B.4. S06, Conflicting Evacuation Orders, 10:50 . . . 89
B.5. S07, Reduced Bus Capacity, 10:05 . . . 90
B.6. S08, Partial Shelter Capacity Loss, 10:15 . . . 90

C. Decision Rubric Counts **91**

D. Supplementary Tables **92**

Declaration of Authorship **95**

List of Figures

3.1. The coordination storyline 12

4.1. Architecture of the agentic system 15
4.2. The shared engine that runs every agent 18

5.1. The baseline's command cycle 26

6.1. The twelve tables of the database 30
6.2. The event log as the backbone of the evaluation 33

7.1. How data moves through the observation interface 39
7.2. The observation interface during a run 40
7.3. The tick drawer for one reasoning tick 41

8.1. The three gates of the evaluation protocol 48

9.1. Residents delivered per run 60
9.2. Plan revisions against delivery 61
9.3. The sixteen rubric criteria 62

List of Tables

3.1. The agent roster. Six agents reason with a language model. Four are scripted responders. . . . 10
3.2. The seven evaluated scenarios. . . . 13

5.1. The baseline's phase ladder. Each phase starts when its condition holds, and must close within its time budget. The last column maps the phases to the storyline steps of Section 3.3. . . . 25

6.1. The twelve tables, grouped by job. . . . 31

8.1. The seven parity deviations . . . 44
8.2. The registered hypotheses. . . . 45
8.3. The held-out decision rubric, condensed. Each clause from the supplier's scenario text becomes a mechanical rule on the log. . . . 50
8.4. The amendments that change how a result is read. The full set of sixteen is in the pre-registration's amendment log. . . . 54

9.1. What was run. . . . 55
9.2. Anticipated arm, mean survival S. . . . 56
9.3. Novel arm, mean survival S over all 50 runs per model against the 50 paired baseline runs. . . . 57
9.4. Novel-arm risk differences with bootstrap confidence intervals. Positive favours the agentic system. The baseline scored 1.000 in every cell, so a model's mean uncapped S on a disruption equals $1 + RD$. . . . 58
9.5. Residents never delivered to any shelter, per 100. Lower is better. . . . 59
9.6. Plan revisions against delivery, 150 agentic novel runs. Exploratory. . . . 60
9.7. Coordination surface on the novel arm. Exploratory. . . . 63
9.8. Verdict on each registered hypothesis. . . . 65

10.1. Gate 0 evidence: the no-replan control on the scored scenarios . . . 67
10.2. Null-policy regret on the novel arm . . . 68
10.3. Further declared limitations and the position taken on each. . . . 70

11.1. Candidate causes of the agentic loss, and the verdict on each . . . 72

A.1. The two playbooks not printed in full, Bus-Driver-1. . . . 88

C.1. Held-out decision rubric, `yes` judgements out of 10. . . . 91

D.1. Recovery levels and cut-point sensitivity per cell . . . 92
D.2. Gate 1 per cell with the Wilcoxon check . . . 93
D.3. Secondary measures per cell . . . 94

1. Introduction

1.1. Motivation

Hamburg lies on the Elbe, upstream of the North Sea. A strong storm surge can push water up the estuary and into the city. When a surge is forecast, people who live near the river have to leave in time. For most of them this means a car or a train. For the residents of a nursing home it means a planned operation. Many of them cannot walk far, some need medical care on the way, and all of them depend on vehicles and staff that the home does not own. Elderly people make up a large share of flood deaths [Yin+24]. One simulation suggests that models which ignore differences in mobility underestimate how long the operation takes [Alq+25].

Such an evacuation is a coordination problem across organizations. A crisis team decides. A transport company supplies buses and drivers. The police escort the convoy, a medical service travels with it, and schools open as shelters. Each organization sees only its own part of the situation, and all of them work against one fixed deadline, the arrival of the water. The plan also rarely stays as it was. A road closes, a shelter reports fewer beds, or a resident collapses on a bus that is already moving.

In Germany such operations follow a written doctrine, DV 100 [FwDV04]. A computer simulation in this tradition is a script. It encodes the steps of the doctrine and adds a branch for each disruption its author expected. Research on business processes calls these anticipated exceptions. It separates them from unanticipated exceptions, which a fixed script cannot handle by itself [RW12]. Woods [Woo15] draws a similar line between robustness against disruptions that were modelled in advance and graceful extensibility beyond them. A script can be robust, but it only covers the cases its author wrote down.

Agents built from large language models (LLMs) suggest a different design. An LLM agent observes, reasons and acts in a loop, and it can revise its plan when something unexpected happens [Yao+23]. If each organization is played by such an agent, and the agents coordinate through a shared database, the system could in principle respond to a disruption that nobody scripted. The critical literature on LLM agents gives reasons for doubt [Kam+24; Cem+25]. Whether the promise holds when the score is the number of residents who reach a shelter was an open question (Section 2.6).

1.2. Problem Statement

We study this question on one concrete case. A storm surge is forecast for Hamburg. The Augustinum, a senior residence on the Elbchaussee near the river, has to move its 100 residents to two schools that serve as shelters before the water arrives. Each school offers 60 beds. The local transport company provides two buses with 50 seats each, one as the primary bus and one

in reserve. No bus can carry everyone in one trip, and no shelter can take everyone, so the allocation is a real decision. The resources belong to eight different organizations. During the operation a disruption arrives that may force the plan to change. Chapter 3 describes the case in detail.

We treat the problem as a design-and-evaluation study, in the sense of design science research, which builds an artifact and then evaluates it [Hev+04]. First, we design a database-centric agentic architecture. Ten agents, each representing a person at one of the eight organizations, coordinate through one PostgreSQL database.[1] Second, we build a workflow baseline that follows the DV 100 doctrine and has no language model in the loop. Both systems run the same scenarios and write to the same append-only event log, and one deterministic scorer reads that log for both. Third, we compare the two on seven scenarios under a pre-registered protocol, and we report what we learn in whichever direction the result falls.

1.3. Research Questions

The thesis asks three research questions.

RQ1 Can a database-centric agentic architecture coordinate a nursing home evacuation better than a scripted workflow baseline?

RQ2 If it cannot, why not? Where does the deficit come from?

RQ3 What would have to change, and what should a designer consider, when building an agentic simulation for emergency coordination?

RQ1 has a boundary that we state here and keep throughout the thesis. None of our scenarios required adaptation. A controller that ignores every disruption reaches the same score as the baseline (Chapter 10). We therefore answer RQ1 for these scenarios only. The question is whether the agentic architecture won here, and we make no claim about whether it can ever win. RQ2 then looks for the cause of the result in the evidence, and RQ3 turns that cause into guidance for the next design.

Building the two systems also raises four engineering questions. They support the research questions.

1. How can the actors, states, events and dependencies of an evacuation be modelled in a database? (Chapter 6)
2. How can agents that represent different organizations be integrated into one system? (Chapter 4)
3. Which persistent structures can serve both runtime control and evaluation? (Sections 6.3 and 6.4)
4. How do the agentic system and the workflow baseline compare in performance? (Chapters 8 and 9)

[1] `https://www.postgresql.org`

1.4. Contributions

The thesis makes four contributions.

An agentic architecture and its baseline. We designed and built a database-centric agentic architecture for evacuation coordination across several organizations. Ten agents represent people at eight organizations. PostgreSQL is the single source of truth, and it also acts as the message broker that wakes the agents. Next to it we built a workflow baseline grounded in the DV 100 doctrine. The two share the scenarios, the action set and the scoring, and they differ mainly in how decisions are made. In the baseline, a simulator also stands in for the field agents (Chapters 4, 5 and 6).

A scored comparison with a negative result. To our knowledge, this is the first scored head-to-head comparison of the two approaches on a common evacuation case. It covers 310 runs over seven scenarios, with pre-registered hypotheses and deterministic scoring (Chapter 8). The agentic architecture does not outperform the scripted baseline, and both confirmatory hypotheses fail. Our main test, Gate 1, compares resident survival with the baseline in each cell, one model on one scenario, run up to ten times (Section 8.5.1). It fails in 26 of 27 cells. Three of them are on the road closure, where the two systems ran different versions of the scenario. Without them, 23 of 24 cells fail. On the five novel disruptions, written after the baseline was frozen, the baseline sheltered every resident in all 50 runs. The best agentic model reached a capped mean survival of 0.894 (Section 9.3). The agentic system also scores below a null policy, a control that ignores every disruption. In all 15 novel cells it did worse than this control, and 68 of 150 runs ended worse than doing nothing (Section 10.3.1). Part of this gap is there even without a disruption.

A diagnosis of the loss. A held-out decision rubric scores what each system decided with 16 checks on the event log, fixed before the novel runs (Section 8.6.1). The agents matched the baseline on 5 checks and fell short on 5. Their messages got through. Over 1,932 addressed requests, 93.8% of the handshakes were closed. Residents were lost on legs that were dispatched but not carried through to delivery. The losses come in discrete amounts. Of 150 novel runs, 102 delivered everyone, 19 delivered 50, one bus-load short, and 18 delivered 90. Runs that rewrote their plan more often delivered less. Full delivery falls from 90% at the fewest plan revisions to 32% at six or more. Our leading hypothesis is a failure of commitment. We did not measure comprehension, and we do not claim a causal direction. Live LLM field agents carried out the orders where the baseline used a simulator, so a lost leg may have been dropped in the field.

A method for a fair comparison. The shared append-only event log is the backbone of the comparison. Both systems write to it, one scorer reads it, and every run can be replayed and audited. We also propose Gate 0, a check on treatment validity. Before a campaign starts, it tests whether a control that ignores a disruption scores worse than a system that responds to it (our own novel arm fails it, Section 10.2).

1.5. Scope and Delimitations

Some limits are part of the design. The residents are a passive count and make no decisions of their own. The flood itself is not simulated. It enters the case as a forecast and a deadline. The baseline is plain Python and uses no workflow engine such as Camunda, Temporal or Airflow. Apps for citizens are outside the scope. The study covers one facility, one city and one hazard.

Other limits come from what was run. Every run uses one simulation time scale, a `SIM_TIME_SCALE` of 8.4, so hypothesis H4 about the effect of time scale was not tested. Each cell has $k = 10$ replicates, and the registered sign test runs over only five scenarios, which is too few for it to detect a moderate effect. Three models completed the full novel arm: glm-5.2, deepseek-v4-pro and qwen3.5. The third model slot changed after data were seen, so the model set of the novel arm is exploratory. Chapter 10 discusses each of these limits and the position we take on it.

1.6. Thesis Outline

Chapter 2 reviews scripted coordination, LLM agents, the limits of LLM planning and coordination through shared state, and places the thesis among related work. Chapter 3 presents the Hamburg case, the organizations involved and the disruption scenarios. Chapters 4 to 7 describe what we built: the agentic system, the workflow baseline, the data architecture that both share, and the interface we use to observe a run. Chapter 8 sets out the pre-registered method, and Chapter 9 reports the results. Chapter 10 reads the results back against the protocol and derives Gate 0 from what it finds. Chapter 11 interprets the results and answers RQ1 and RQ2. Chapter 12 answers RQ3 with design considerations for agentic emergency simulation. Chapter 13 summarizes the thesis and names the next experiment.

2. Background and Related Work

This thesis compares two ways of coordinating an evacuation. One is a script written in advance. The other is a group of language-model agents that reason over a shared database. Five lines of research meet here. The first is work on scripted processes and how they handle events nobody planned for. The second is work on agents built from large language models (LLMs). The third is classic agent theory on intentions and commitment. The fourth is a critical line of work that tests how well LLMs plan. The fifth is older work on programs that coordinate through a shared store. We close with the systems that come closest to ours and the gap this thesis fills.

2.1. Scripted Coordination and Unanticipated Events

Emergency services coordinate large incidents through fixed command structures. The best-studied example is the American Incident Command System (ICS). Research on ICS describes a tension between two kinds of organization. A mechanistic organization works by rules and a clear hierarchy. An organic one adapts its structure to the situation [Cha17]. ICS is usually seen as mechanistic. Still, Bigley and Roberts [BR01] show that such a bureaucratic system can respond in a flexible and reliable way when the situation keeps changing. The system also has known weak points. Buck, Trainor, and Aguirre [BTA06] find that ICS works less well when many independent organizations must cooperate, and that it has trouble absorbing help that was not part of the plan. German fire services and civil protection follow a comparable doctrine, DV 100 [FwDV04]. Chapter 5 describes how our baseline implements it.

Research on business processes asks the same question from a software angle. Reichert and Weber [RW12] separate two kinds of exceptions. An anticipated exception is one the designer foresaw, so the process can carry a handler for it from the start. An unanticipated exception is one nobody foresaw. It needs a change to the running process, and a fixed script cannot make that change by itself. Build-time handlers are the state of the art for the first kind. Worklets, for example, select a prepared sub-process that fits the current context [Ada+07]. Our baseline works the same way. It fixes the order of steps and the response to each anticipated exception in advance.

Resilience engineering draws a similar line. Woods [Woo15] separates robustness from graceful extensibility. Robustness means handling disruptions that were modelled in advance. Graceful extensibility means stretching to meet a surprise that falls outside the model. These terms give the comparison in this thesis its vocabulary. A scripted baseline can be robust against anticipated disruptions. Whether an agentic system extends further when the disruption is new is an empirical question, and Chapter 8 describes how we test it.

2.2. LLM-Driven Agents

An LLM agent is a language model placed in a loop. It observes its environment, reasons about what to do, acts, and observes the result. ReAct interleaves written reasoning steps with actions. The authors argue that the reasoning helps the model track and update its plan and deal with exceptions [Yao+23]. Reflexion adds a step in which the agent writes a short reflection on its own performance and keeps it in memory for the next attempt [Shi+23]. The authors also note that this works best when the agent receives a clear signal about what went wrong. Our agents run a similar loop of sensing, reasoning and acting, but they have no separate reflection step (Section 4.8).

A second line of work gives agents human roles. Generative Agents places 25 agents with memory, reflection and planning in a small simulated town, and human raters find their social behaviour believable [Par+23]. That study measures believability. It does not score how well the agents perform a task under pressure. Our ten agents also take human roles. Each one plays a person in a role at one of eight organizations.

Several frameworks let many such agents work together. MetaGPT, for example, writes human standard operating procedures into the prompts of role-specialized agents and passes work along an assembly line [Hon+24]. MetaGPT is interesting for this thesis because it mixes the two ideas we keep apart. Its agents reason, but their order of work comes from a script.

We use none of these frameworks. Our system runs its models through Ollama[1] inside an engine we wrote in plain asynchronous Python. Chapter 4 explains why.

2.3. Intentions and Commitment

Agent research had a precise word for keeping to a plan long before LLMs. In the belief-desire-intention (BDI) model, an agent holds beliefs about the world, desires it would like to reach, and intentions it has chosen to act on [RG95]. Rao and Georgeff explain why a practical agent needs intentions at all. The world changes while the agent acts, and the agent cannot afford to plan again from the start at every change. Cohen and Levesque [CL90] describe an intention as a choice with commitment. The agent keeps it until it believes the goal is reached, believes the goal can no longer be reached, or the reason for the goal is gone. Without this persistence, an intention would be only a passing wish.

It follows that an agent which reconsiders its plan too often never finishes what it started. One that reconsiders too rarely follows a plan that no longer fits the world. In an LLM agent, the plan usually lives in text that the model writes, and the model can rewrite it every time it is called. Our agents keep their plan as committed steps in the database, but the model may still revise those steps whenever it wakes (Section 4.8). How often it should do so is a design question for any agentic system. Section 11.2 returns to it.

[1] `https://ollama.com`

2.4. Limits of LLM Planning

A strong critical literature questions whether LLMs can plan at all. Kambhampati et al. [Kam+24] argue that an LLM on its own can neither plan reliably nor check its own plans. In their view, it should propose plans and leave the checking to a sound external verifier.

Self-correction is a related concern, because it is how a replanning agent is supposed to recover. Huang et al. [Hua+24] find that when a model corrects its own reasoning without outside feedback, the result often gets worse. Stechly, Valmeekam, and Kambhampati [SVK24] report the same pattern for planning tasks. Performance drops when the model criticizes its own answers and rises when a correct external checker gives the feedback.

Multi-agent systems add failure modes of their own. Cemri et al. [Cem+25] study execution traces from several popular frameworks and build a taxonomy of how they fail. The failures fall into three groups: poor system design, poor alignment between agents, and poor checking of whether the task is done. Other work doubts that several agents help much. A strong single agent with good prompts can match a multi-agent discussion [Wan+24].

For this thesis, the literature predicts that agents may understand a situation and still fail to act on it well. It also suggests that an LLM should not judge its own success. Our evaluation therefore scores every run with deterministic code over the event log, and not with a model. On the agentic side, the arrivals in that log are still the drivers' own reports (Section 8.4.2). Chapter 11 comes back to these failure types when it asks where the agentic deficit comes from.

2.5. Shared-State Coordination

Our agents do not call each other directly. They read and write one shared database, and every change is recorded in an append-only log. The idea of coordinating through shared state is old. In a blackboard system, independent knowledge sources watch a common data structure and add to it when they can contribute. The classic example is the Hearsay-II speech recognizer [Nii86]. In the Linda model, processes communicate by placing tuples in a shared space and taking them out again, without knowing each other [Gel85]. Later tuple spaces became reactive. They can run code when a tuple arrives, so a process reacts to new data without asking for it again and again [OZ99]. Our agents wake in a similar way when a notification tells them that a new row is waiting (Section 6.6). Event sourcing, a pattern from software engineering, keeps the full history of changes as the source of truth. Any past state can then be rebuilt by replaying the log. An industry study finds this useful for audits and replay but notes that it makes schema changes harder [Ove+21].

We therefore do not claim shared-state coordination as new. Our claim is narrower and concerns evaluation. Section 6.4 states it.

2.6. Related Work and the Gap

A small number of recent systems apply LLM agents to disasters. RESPOND couples a flood forecast with an agent-based model of the population. LLM-driven agents decide how residents

evacuate, look for resources and communicate [Sul+26]. It is a short demonstration paper and does not report a controlled comparison. Lee et al. [Lee+25] give LLM agents synthetic personas built from mobility and demographic data, and ask how well they predict real evacuation decisions. Both works model the people who evacuate. In our system the residents make no decisions. The agents are the organizations that move them.

Li, Das, and Shirado [LDS26] ran a long design study with emergency-preparedness practitioners. They conclude that LLM agent simulations are more useful for shaping procedures and training than for predicting outcomes. This matches the purpose of our simulation, which studies a coordination mechanism and does not forecast a real evacuation.

LLM agents are more and more often evaluated on realistic tasks, with scores computed by code. TravelPlanner asks agents to build travel plans with tools and checks each plan against hard constraints and common-sense rules [Xie+24]. TheAgentCompany scores agents on professional tasks in a simulated software company [Xu+25]. Scoring by code also avoids the known biases of using an LLM as the judge [Zhe+23]. These benchmarks compare models with each other. Neither runs a non-LLM controller through the same harness as a reference point.

Evacuation and relief logistics are usually studied as optimization problems. Huang, Smilowitz, and Balcik [HSB12] define performance measures for relief routing and show that efficiency, efficacy and equity can pull in different directions. Holguín-Veras et al. [Hol+13] argue that such models should minimize social cost. This adds the cost of human suffering from late or missing supplies to the cost of the logistics itself. For coastal cities, Yin et al. [Yin+24] plan storm-flood evacuations with a focus on moving elderly people. These models take the view of one planner who sees the whole problem and computes the best plan. In our case, the resources and the information belong to several organizations. Even the baseline's commander has to collect reports and give orders through messages. We therefore study how well the coordination works, and we do not search for an optimal route or allocation. Chapter 3 cites further work on evacuating people with reduced mobility.

To our knowledge, no prior work puts LLM agents in the role of the organizations that coordinate an evacuation and compares them with a doctrine-grounded scripted baseline on a scored outcome. This thesis makes that comparison. It holds the world model, the action set and the scoring fixed, and changes only the way decisions are made (Section 8.1).

3. Case Study: Storm Surge in Hamburg

3.1. Scenario Overview

We study evacuation coordination on one concrete case: a predicted storm surge in Hamburg that forces the evacuation of a nursing home on the river Elbe. The Elbe is tidal along its whole course through the city, and a storm over the North Sea can push water up the estuary into the city centre. Risk studies for the German Bight and the Elbe estuary model how such surges develop and which areas they reach [Oum+15]. We take this hazard as given and do not model the flood itself. For us, its key property is the warning time. A surge is forecast hours in advance, so responders have a window to act, but the window is fixed. The case is therefore a planning problem under a deadline.

The site is the Augustinum, a senior care facility at Elbchaussee 374, 22609 Hamburg, close to the river in the west of the city. Its 100 residents all have to leave before the water arrives. Many of them cannot walk far, cannot drive, and cannot leave on their own, so the evacuation is a transport and care operation. Evacuation models that ignore mobility differences can underestimate how long such an operation takes [Alq+25]. The German storm-surge work cited above studies the hazard, the risk and the damage, but not the evacuation of vulnerable groups on the Elbe. We found no ready-made model of this operation to reuse, which is one reason we built the case.

The case is small enough to run many times and large enough to be realistic. The roster is fixed at two buses with 50 seats each, two school shelters with 60 beds each, one police escort and one medical team. No bus can carry all 100 residents in one trip, and no shelter can take all of them. The 120 beds leave only 20 to spare. A plan therefore has to split the residents between the shelters and schedule more than one run, which makes the allocation a real decision. Bus-2 is declared a reserve, and the two systems use it differently (Section 5.2).

The case offers a large space of possible disruptions (the ones we ran never required a change of plan, Section 10.2).

3.2. Organizational Agents

Every agent stands for a person in an organizational role, such as a coordinator at a desk, a bus driver or a head teacher. Vehicles are passive resources that a person drives. Each agent has a fixed identity with an invented name, a role, a work address and an address book of the contacts it may message. It only knows what the person in that role could see. Table 3.1 lists the ten agents in eight organizations.

Table 3.1.: The agent roster. Six agents reason with a language model. Four are scripted responders.

Agent	Person and role	Organization	Reasoning
BIS	Dr. Andrea Schneider, duty head of the situation centre (*Lagedienstleiterin*)	Hamburg's interior authority (*Behörde für Inneres und Sport*)	LLM
Hochbahn	Markus Weber, dispatcher (*Disponent*) in the traffic control centre	Hamburger Hochbahn AG, the public transport operator	LLM
Bus-Driver-1	Hans Müller, drives bus-1 (50 seats, primary)	Hamburger Hochbahn AG	LLM
Bus-Driver-2	Petra Schmidt, drives bus-2 (50 seats, reserve)	Hamburger Hochbahn AG	LLM
Polizei	Susanne Krüger, patrol officer, drives patrol car PC-21-1	Hamburg Police (*Polizei Hamburg*), station PK 21	LLM
DRK	Jonas Brandt, paramedic, drives ambulance DRK-T1	German Red Cross (*Deutsches Rotes Kreuz*), Altona	LLM
ZKD	Dr. Thomas Richter, deputy head	the city's central disaster staff (*Zentraler Katastrophendienststab*)	scripted
Augustinum	Stefan Becker, head of the home (*Heimleiter*)	Augustinum, the origin facility	scripted
SC1	Maria Hoffmann, head teacher	Schule Dohrnweg, Altona (60 beds)	scripted
SC2	Klaus Werner, head teacher	Schule Marienthal, Wandsbek (60 beds)	scripted

We gave a language model only to the agents that make a decision in the case. ZKD relays the alarm, books the reports and countersigns a stand-down. The schools report their free beds and confirm arrivals, and the nursing home reports its residents and when the building is empty. None of the four has to choose between options, so all four are scripted.

The workflow baseline keeps this roster, but one scripted Incident Commander (IC) replaces the two coordinating agents, BIS and Hochbahn. The other agents are not started in a baseline run. An execution model carries out the IC's orders in their place, so no decision on the baseline side comes from a model. Chapter 4 describes how the agents reason, and Chapter 5 describes the baseline.

3.3. Coordination Storyline

The mission follows ten steps (Figure 3.1). They fix what has to happen and who is responsible, and leave the decisions to the coordinator.

1. **Identification.** ZKD relays the flood alert to BIS.
2. **Evacuation assessment.** The Augustinum confirms 100 residents and the deadline.

3. **Resource allocation.** Each school offers 60 beds, and BIS decides the split.
4. **Route planning.** Each driver routes its own legs with OpenRouteService.[1]
5. **Logistics.** Hochbahn sends BIS a run schedule, by default two runs of bus-1 with bus-2 in reserve.
6. **Police coordination.** The patrol car leads the convoy on every run.
7. **Medical support.** The ambulance follows at the tail on every run.
8. **Evacuation execution.** The driver reports each departure, and the home reports once whether the building is cleared.
9. **Arrival and handover.** The school confirms the intake, and Hochbahn tells BIS the run is complete.
10. **Mission report.** BIS reports to ZKD, which closes the record.

Steps 8 and 9 repeat for every bus run. How many residents go to each school, and on which run, is the coordinator's decision. The baseline's commander walks the same steps in four phases (Table 5.1).

One further path lies outside the ten steps. If BIS finds that no recovery is possible, ZKD countersigns a stand-down and BIS recalls the transport and the escorts. No evaluated scenario triggered it.

3.4. Disruption Scenarios

Every run begins with the alarm and ends at the forecast high water. In all scenarios the flood reaches the zone at 12:00, and a resident who reaches a shelter later does not count as rescued. We evaluated seven scenarios in two arms (Table 3.2). The two *anticipated* scenarios were known while the systems were built. S01 has no disruption, and the baseline has a written branch for the road closure in S02. The five *novel* scenarios were supplied by the domain expert, Rüdiger Höfert, from a brief that named only the kind of disruption wanted. They were revealed after the baseline was frozen on 2026-09-10 (Section 8.3). A shelter closure (S03) had no agentic counterpart and was not scored. Section 8.6.1 lists the response each scenario expects.

Every disruption arrives as a message and leaves the declared world unchanged, so S04, S06, S07 and S08 are judged mainly on the decisions (Section 10.2).

3.5. Assumptions and Simplifications

The case makes the following simplifications, for both systems.

- **Residents are a count,** with no behaviour and no subgroups by mobility or care level.
- **The flood is a deadline.** Each zone has one arrival time, and the surge is not simulated.

[1] `https://openrouteservice.org`

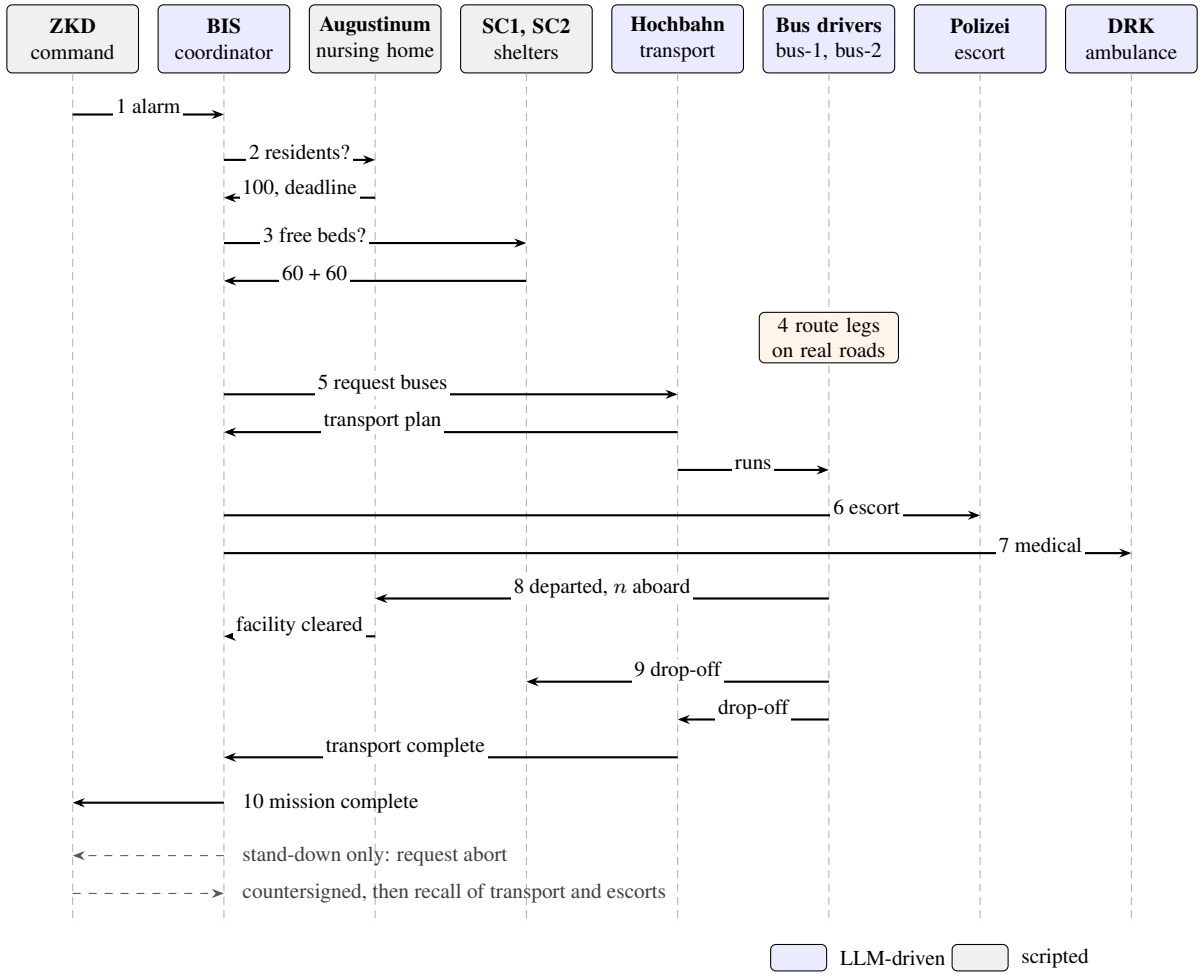


Figure 3.1.: The coordination storyline as message flow between the organisations of the case. Numbers give the storyline step. Steps 8 and 9 repeat for every bus run. Step 4 has no message, because each driver routes its own legs over real roads. The dashed branch is the conditional stand-down, which no evaluated scenario triggered. In the workflow baseline, one Incident Commander takes the place of the two coordinator lanes, BIS and Hochbahn. All other lanes are shared by both systems.

- **Roads have no traffic.** Distances and times come from OpenRouteService, with a straight-line fallback.
- **Bus seats are not enforced.** Shelter beds and the deadline are, and residents above a shelter's beds do not count as safe.
- **Messages always arrive,** after a fixed delay.
- **One clock speed.** All runs use 8.4 simulated seconds per real second.
- **One case.** There is one facility, with two buses and two shelters.

These choices keep the case small enough to run hundreds of times, and they limit what the results can show (Chapters 10 and 11).

Table 3.2.: The seven evaluated scenarios.

ID	Scenario	What happens
Anticipated arm		
S01	Closed loop	Nothing goes wrong. The undisrupted control.
S02	Road closure	A road on the transport corridor closes during the evacuation. The baseline has no routing command, so its version makes a school unreachable and declares only 60 residents. Its ceiling is 60, so this scenario is not a like-for-like comparison.
Novel arm		
S04	Medical emergency in flight	A resident collapses on a moving bus. The patient is not modelled.
S05	Unverified persons report	A 112 call reports extra people at the home and is later withdrawn.
S06	Conflicting evacuation orders	The district office (*Bezirksamt*) issues an order that contradicts the running plan.
S07	Reduced bus capacity	Bus-1 may carry only 25 residents. The simulator does not enforce this limit.
S08	Partial shelter capacity loss	A third party reports that SC1 can take only 40. SC1 still declares 60 beds.

4. Database-Centric Agentic System

In this chapter we describe the agentic system: how a single agent is built, how it wakes, what it sees, what it may answer, and what the runtime does with that answer. Chapter 3 introduced the agents (Table 3.1) and the task they share (Figure 3.1), so we do not repeat them here. We focus on how each agent works inside.

We describe the design as we built and evaluated it. Whether the design helped is the subject of Chapter 9.

4.1. Architecture Overview

The system has ten agents in eight organizations. Six of them reason with a language model: the coordinator BIS, the transport dispatcher at Hochbahn, the two bus drivers, the police officer and the paramedic. The other four are scripted: the disaster staff ZKD, the Augustinum and the two schools. Section 3.2 gives the rule behind this split. An agent that makes no decision in the case gets no language model.

Every agent is its own small program. It runs its own loop, keeps its own state and has its own mailbox. All ten programs are built on one shared engine, so they wake, reason, commit and log in the same way. What differs between them is the persona, the set of actions they may take, and a few role-specific rules. Section 4.2 lists these differences.

The agents never talk to each other directly. The only channel between them is a PostgreSQL database, which is also the single source of truth for the whole simulation. The database holds five kinds of record:

- the current *state* of every agent, one row per agent;
- the *mailboxes*, where every message between agents is stored;
- the *world events*, which report that a physical action has finished, for example that a bus has arrived;
- the *prompt records*, which keep the full prompt and the full answer of every model call;
- the *event log*, an append-only audit trail of everything each agent received and decided.

When one agent sends a message, it writes a row to the recipient's mailbox. The database then notifies the recipient, which wakes up and reads it. The same notification mechanism tells the observation interface that a state has changed. Figure 4.1 shows this structure. Every arrow in it passes through the database, and this is what we mean by *database-centric* in the title of this thesis. Chapter 6 describes the data layer in detail.

Vehicles are not agents. A bus, the patrol car and the ambulance are passive resources that

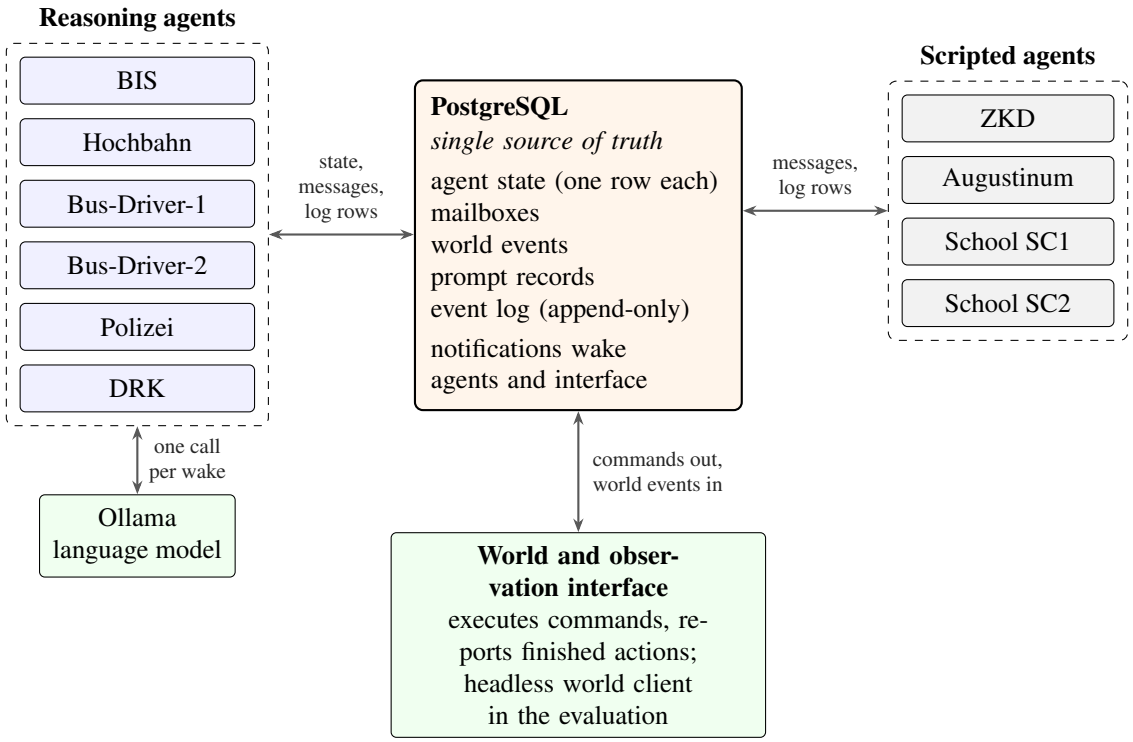


Figure 4.1.: Architecture of the agentic system. Each agent is its own program on one shared engine. Agents never message each other directly. Every message, state change, world report and decision passes through PostgreSQL, which also wakes the recipient. Only the six reasoning agents call the language model.

a person drives. The physical world makes no decisions either. It carries out the commands the agents send, for example “drive to this address”, and reports back when each one is done. During a live run the observation interface animates this on a map. In the evaluation, where no browser is open, a headless world client plays the vehicles instead and sends the same reports (Section 8.1).

Inference runs through Ollama. We wrote the engine around it ourselves, in plain asynchronous Python, and use no agent framework. Section 2.2 discusses such frameworks. We chose a custom engine because the runtime has to control three things that a general framework would keep for itself. The first is time, since the agents act in a simulated clock that keeps running while they think (Section 4.6). The second is validation, since an agent may not send a command or a message that its role does not allow (Section 4.5). The third is the audit trail. The evaluation reads every score from it, and the workflow baseline has to write the same rows. If a framework hid these three behind its own abstractions, the comparison in Chapter 8 would be much harder to defend.

4.2. The Agent Specification Template

We specify every agent, in both systems, along the same nine facets. Using one template means that no agent is described in more or less detail than another.

1. **Context and role.** Who the agent is, where it sits, and who gives it instructions.
2. **Sensing.** What the agent can perceive, and how.
3. **Messaging.** Which message types it receives and which it may send.
4. **Address book.** The concrete contacts it may reach, each with a reason.
5. **Tools.** The actions it can take.
6. **State.** What it knows and remembers, in a form that can be queried.
7. **Scheduling.** How its actions are placed in time.
8. **Instructions.** The instruction types it accepts, with example content.
9. **Reasoning loop.** How it turns an instruction into a plan and a plan into actions. There are no hard-coded subtasks. The agent derives its own plan from its goal.

In our engine the agent does not set its own wake-up times. For scheduling, it says only whether an action should start now or as late as possible before a deadline, and the runtime works out the clock time. Section 4.6 explains how.

The shared engine gives every agent the whole wake cycle for free. This covers the model call, the checking of the answer, the commit of a new step, the replanning rules, the timing checks, the delivery of messages, the handling of world events and every row in the event log. Each agent only has to supply six things:

- its identity, meaning the person, the address, the vehicle and the address book;
- its set of allowed actions;
- a way to estimate how long each action takes;
- its persona and its playbooks, which together form the system prompt;
- the way it renders its current situation into the user prompt;
- whether it holds a step that cannot be done in time, or lets it run late.

We drew the line here on purpose. Anything that could bias the comparison between agents, or between the two systems, sits in the shared part.

4.3. The Wake Cycle

An agent does nothing until something wakes it. Three kinds of event wake the language model:

- a **message** arrives in the agent's mailbox;
- the agent's **current step finishes**, because the world has confirmed its last action;
- a **timing conflict** is found, because the step the agent just committed cannot be finished by its deadline.

Each of these wakes leads to one model call, and the answer to that call is one decision.

One wake, in order. The engine handles every wake in the same sequence. Figure 4.2 shows the steps, and the path beside them that needs no model.

1. It writes the incoming event to the event log.
2. It updates the state before the model sees it. A new message is put into the inbox. A finished step is moved to the history, and the next leg of the plan is taken from the front of the remaining plan.
3. It takes a fingerprint of everything the model is about to see. This lets the evaluation tell whether two decisions were made from the same observation.
4. It builds the prompt, records it, and calls the model. The prompt is stored before and after the call, so the interface can show that a call is in progress.
5. It checks the answer against the agent's rules and counts every rule the answer tried to break.
6. It commits the new step and the new plan (Section 4.8).
7. It checks and sends the outgoing messages.
8. It writes one decision row to the event log, with the reasoning, the step, the plan, the messages, the token counts and the latency.

Work that needs no model. Most of what happens in a run does not wake the model at all. When the time for a committed action comes, the clock sends it out to the world directly. This dispatch path makes no model call. It only marks the action as sent and logs it. When the world reports that an action is done, the engine marks it as confirmed and writes the vehicle's real position back into the state. The agent wakes only when the last action of a step is confirmed, with the step-finished event. Some agents also answer certain routine messages without the model, for example a plain acknowledgement or a position update. These *fast paths* are opt-in for each agent, because whether a message is purely informational depends on the role that receives it. Each fast path still writes a decision row, with zero tokens, so the event log stays complete.

Malformed answers. The model runs at a low temperature of 0.2. If it returns an answer that is not valid JSON, the engine does not ask again. It records the failure and moves on. We decided against retries because a retry gives a weak model a second attempt that a strong model never needed, and that would hide a real difference between models. The cost is that one bad answer can lose one decision. The evaluation design states this rule (Section 8.1).

4.4. Prompt Structure

Every model call has two parts. The system prompt says who the agent is and how it must act. The user prompt says what the situation is right now.

The system prompt. It is built from three blocks, in this order.

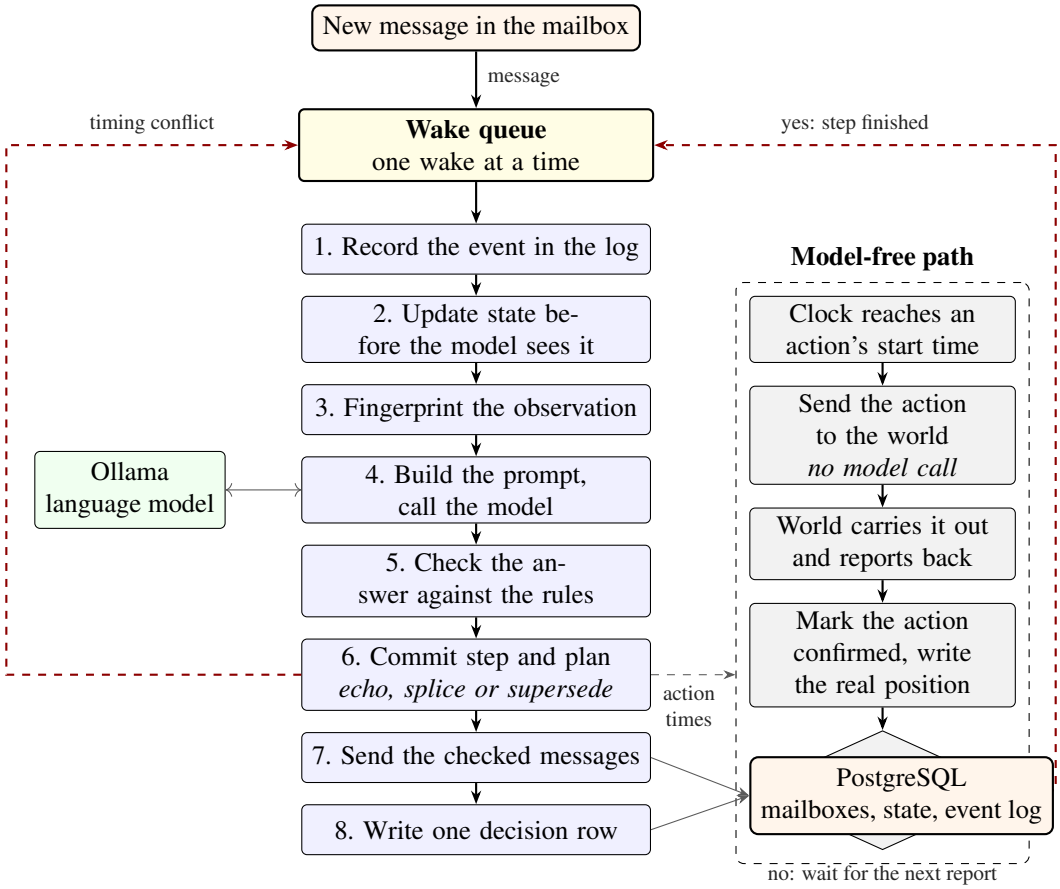


Figure 4.2.: The shared engine that runs every agent. A wake enters the queue and passes through eight steps, and only step 4 calls the language model. Actions are sent to the world on the right without the model. The model wakes again only when a message arrives, when the last action of a step is confirmed, or when a newly committed step cannot be finished in time.

1. The **persona** says who this person is: name, age, job, employer, vehicle, the contacts in the address book, and the message types the person may send. This is the only block that is different for every agent.
2. The **contract** says how the agent acts. It explains the wake types, the format of the answer, the allowed actions, how an address must be written, and the handover rules at the nursing home and at the school. The three driving agents share one contract shape. The two coordinators each have their own.
3. The **playbook** is chosen by the kind of wake. A bus driver has three: one for planning from a new message, one for planning the next step after a step finishes, and one for handling a timing conflict. The coordinators select among more. BIS, for example, has separate playbooks for the first alarm, a refusal, a request for more time, a new transport plan and the final status of the facility.

Each playbook is a numbered list of decision rules, followed by one worked example. The example is set in a made-up Hamburg situation that appears in no test scenario. This keeps the model from copying the live scenario from its own instructions. The system prompt contains

nothing specific to the scenario, so it is the same on every wake of a run.

The user prompt. It is split into fixed sections. For a bus driver these are: the wake itself, the current location, the place where the bus is parked, the mission brief, the mission status, the remaining plan, the current step, the last three finished steps, and the conversations. The mission brief is the newest dispatch order, pinned in full so it never scrolls out of view. The conversations show the last five messages with each contact, in time order, including the agent's own earlier messages. The agent therefore sees what it said before, even though the model keeps no memory between calls. Every section is always printed, with "none" if it is empty, so the layout never changes from one wake to the next.

The answer. The model must answer with one JSON object. It holds:

- a short piece of **reasoning** in plain text, which is required. Reasoning written before the action follows ReAct [Yao+23];
- the **current step**, with a label, a deadline and an ordered list of actions. For each action the model says only whether it should start *now* or *by the deadline*. It never writes a clock time;
- the **remaining plan**, a list of future legs, each with a label and a deadline. If the model leaves the plan out, the old plan is kept. If it writes one, the new plan replaces the old one completely;
- the **outgoing messages**, each with a recipient, a type and a body;
- optionally, an **abort** with a reason.

Since leaving the remaining plan out means "no change", every plan the model does write counts as a deliberate revision. The evaluation relies on this. Appendix A reproduces the system prompt of the reference agent.

4.5. Runtime Ownership and Ground Truth

The model proposes a step, and the runtime decides what happens with it. This follows the advice to pair a language model with an external checker instead of trusting its plans directly [Kam+24]. Some state belongs to the runtime, and nothing the model writes can change it. We list these rules in full. They keep the simulation correct, and they also limit what the agentic system can do, which makes them part of the comparison.

1. **The runtime sets every action time.** Any time the model writes is thrown away and computed again from the routed duration and the fire mode.
2. **Only allowed actions pass.** An action that is not in the agent's action set is dropped. The model cannot invent a new command.
3. **Messages are checked twice.** The recipient must be in the address book, and the message type must be one the agent may send. A message that fails is dropped with a warning. It never blocks the wake, and it is counted as an invalid action.

4. **Completion needs evidence.** A mission counts as complete only when there is no current step, no remaining plan, at least one finished step in the history, and no abort. An empty answer can never end a mission by accident.
5. **An abort clears the plan.** After an abort, the remaining plan is empty, even if the same answer also wrote a new one.
6. **History is never lost.** A step is always moved to the history before it is replaced. The full sequence of steps can be rebuilt from every run.
7. **Each world report counts once.** A report that is delivered twice is recognised and ignored.

Ground truth from the world. The agent's position always comes from the world. Each report of a finished drive carries the vehicle's real coordinates, and the runtime writes them into the state. If a drive is stopped half-way, for example because the agent changed its plan, the world reports where the vehicle halted. The next prompt then shows the vehicle's true position instead of the one the model assumed, and the next route starts from there.

These rules give the agentic side no advantage, and they do not reason for the agent. The runtime never chooses a destination, a split of residents or a message. It refuses what the role does not allow and keeps the record complete.

4.6. Time and the Simulation Clock

The agents live in simulated time. Every agent runs its own clock, which moves forward in fixed ticks. The ratio between simulated and wall-clock time is set by a time scale. In every scored run this scale was 8.4. Only this one value ran, so this thesis makes no claim about how the results change with the time scale (Section 8.6.3).

Fire modes. As Section 4.4 said, the model marks each action either *now* or *by the deadline*. The runtime turns this into clock times. First it estimates how long each action takes. A drive is routed over real roads with OpenRouteService. A walk is estimated from the straight-line distance at walking speed. Boarding and leaving a vehicle take a fixed minute. If routing fails, the runtime falls back to a straight-line estimate and marks the action as degraded. It never blocks the step. Then it sets the times. A *now* action starts as soon as the previous action ends. A *by the deadline* action starts as late as possible so that the whole step still finishes on its deadline. The coordinators' actions are administrative and take no time, so every one of their actions falls on the deadline.

Timing conflicts. After a step is committed, the runtime checks whether any action that has not yet been sent would have had to start in the past. A slack of 60 seconds is allowed, because the model itself takes time to answer. If the check fails, the agent is woken with a timing conflict, together with the size of the gap. Actions that are already sent are not checked again. They have already happened, and changing their times on paper would change nothing. Section 4.8 describes what happens when the conflicts repeat.

The clock during a model call. A model call takes real seconds. If the clock kept running at full speed, one slow call could move simulated time by many minutes, and the step being planned would already be late when it arrived. If the clock stopped completely, thinking would be free. We chose a middle way. During a call the clock keeps running for a limited grace window, and then freezes until the answer arrives. The window equals the simulated time that the slowest allowed call can take, which is the client timeout of 120 seconds multiplied by the time scale. Thinking costs simulated time, as it would for a real person, and this cost has an upper bound.

4.7. Agent Roster

Table 3.1 lists the agents. This section describes how each class of agent uses the shared engine.

The bus drivers. Both drivers use the same engine and the same prompts. Only their identity is different. They have the richest action set (Section 4.9), and they are the only agents that hold a step that cannot be done in time. A held step is not sent to the world, so the bus does not leave while the driver negotiates. A driver who runs more than one trip goes back to the nursing home after each run, and returns to the depot only after the last one.

The escorts. The police officer and the paramedic each drive their own vehicle as part of the convoy. The police car leads, the bus follows, and the ambulance runs at the tail. Their contract has the same shape as the drivers', with six physical actions. For them a timing conflict is only a warning. The step still runs, and they tell BIS they will be late.

The coordinators. BIS and Hochbahn move nothing physical. Their actions are administrative: ask the schools for beds, request buses, order an escort, send a dispatch, submit a report. Each action produces a message, so BIS sends messages only through its actions. Because nothing is dispatched to the world, their clock does a different job. When a step's deadline arrives, it checks whether the replies the step was waiting for have come in. If they have, the step is finished. If not, the coordinator is woken with a timing conflict. BIS is the only agent that decides the mission, and the only one that may call it off. Every other agent that cannot meet a deadline asks BIS for more time through the same request, and this includes Hochbahn.

The run schedule travels on three edges. Hochbahn sends BIS a transport plan with every run, its passenger count and its deadline. BIS passes the run numbers and the list of runs on to the two escorts. Hochbahn tells each driver which run it is on and how many there are. By default bus-1 makes two runs in sequence and bus-2 stays in reserve. Hochbahn releases the reserve for only four reasons: a leg needs more seats than bus-1 has, bus-1 is not available, BIS asks for two buses, or the schedule fails Hochbahn's deadline test. The baseline treats the reserve bus differently. Section 5.2 describes this and what it means for the comparison.

The scripted agents. ZKD, the Augustinum and the two schools use no language model. They receive a message, apply a fixed rule and reply. Each of them still owns one fact that no other agent can change. The Augustinum owns the count of residents who have left and the

report that the building is cleared. Each school owns the count of residents who have arrived. ZKD owns the formal authority to close or stand down the mission.

The reference agent: Bus-Driver-1. Hans Müller drives bus-1, a city bus with 50 seats, from a depot in the north-west of Hamburg. His part of one run goes like this.

1. A dispatch order from Hochbahn arrives. Hans wakes with the message playbook. He accepts the mission, writes a plan of legs (drive to the home, load, drive to the school, unload, return), and commits the first step: walk to the bus, board it, drive to the Augustinum.
2. The runtime routes the drive and sets the action times. The clock sends each action to the world when its time comes. No model call is needed.
3. When the world confirms his arrival, Hans wakes with the next-step playbook. He tells the Augustinum he has arrived and receives the boarding order.
4. After loading, he reports the bus as departed with the number of residents on board. His next step starts with informing the escorts. The runtime sends the convoy start to the police officer and the paramedic two minutes before the bus leaves.
5. At the school he reports the drop-off to the school and to Hochbahn. If this was not the last run, his plan takes him back to the Augustinum.

If a road closure or a late order makes a step impossible in time, Hans wakes with the timing-conflict playbook. It gives him three choices: ask for more time and hold, commit a step with an honest later deadline and start driving, or abort. The playbook tells him that holding is not free, because the clock keeps running while he waits.

4.8. Replanning by Plan Rewrite

The system has no separate replanning module. There is no special branch that runs “when something goes wrong”, and no reflection step after a failure as in Reflexion [Shi+23]. Instead, the model may write a new step or a new plan on any wake. Replanning is what happens when that new step differs from the one already committed. The playbook for the wake steers it. The timing-conflict playbook, for example, asks the model to recover from a missed deadline. BIS has its own playbooks for a refusal and for a request for more time.

When a new step arrives, the engine compares it with the step already committed. There are four cases.

Echo. The model repeats the same step, with the same label, deadline and actions. Nothing changes. The step is not archived and not renumbered.

Splice. The new step keeps every action that has already been sent to the world, and changes only the ones still waiting. The sent actions stay as they are. The waiting tail is replaced and timed again. For example, a driver who has boarded and is already driving may add a message at the end of the step without disturbing the drive.

Supersede. The new step tries to change an action that has already been sent. That cannot be undone, so the old step is archived as aborted, and the new step starts from the beginning.

In effect the agent changes course.

Back-pressure. Each timing conflict wakes the model again, and the model may answer with another late step. Without a limit, this can loop. So the engine stops raising timing conflicts once the reported gap no longer shrinks from one attempt to the next, or after three attempts in a row. The plan is then released to run late. We prefer this because a late delivery still scores, while a plan parked forever does not.

A run from the evaluation shows why the limit is needed. In one rerun, Bus-Driver-1 spent eight timing-conflict wakes across six steps. The reported gap moved from 8 to 3, 6, 12, 7, 23, 16 and 23 minutes. The run used 869 thousand prompt tokens, 3.4 times its decision budget, and the gap never closed. Holding the step made things worse. A held step never moves the bus, so every new step was routed again from the same depot, and the gap kept growing.

We count supersedes, but only as a diagnostic of how often an agent changed course. The evaluation does not score the count as a measure of success (Section 8.6.2). Rewriting the plan on any wake is the main capability we built this architecture to offer. Chapter 9 reports how often the agents used it and what followed. On the tested scenarios, runs that revised their plan more often delivered everyone less often (Table 9.6). Language agents are also known to struggle to carry a long plan through under many constraints [Xie+24]. Whether the revisions caused the losses, or followed from them, cannot be told from these data. Section 11.2 discusses this.

4.9. Action Vocabulary

Each agent has a closed set of actions. The model may use nothing else.

- **Bus drivers**, seven physical actions: walk to a place, board a vehicle, leave a vehicle, drive to a place, drive through waypoints, stop, and inform the escorts.
- **Police officer and paramedic**, six physical actions: the same set without informing the escorts, because they are the escorts.
- **Hochbahn**, seven administrative actions, such as dispatching a bus, asking BIS for more time, and reporting a run as complete.
- **BIS**, eleven administrative actions, such as asking the schools for beds, requesting buses, ordering an escort, releasing the escorts, submitting the mission report and asking ZKD to stand down.
- **Scripted agents**, none. They only reply.

Every action is checked against the set when a step is committed, and every message against the address book and the allowed types (Section 4.5). Every rejected entry is counted, and the evaluation reports the rejected messages per cell (Section 9.5).

The vocabulary is also one leg of the parity argument. The workflow baseline replaces only the two reasoning coordinators, BIS and Hochbahn (Section 5.1). The four field agents, meaning the two drivers, the police officer and the paramedic, are not started in a baseline run. The execution model plays their part and carries out the IC's orders (Section 5.4). The baseline therefore gains no field action the agentic side lacks.

5. Workflow-Based Baseline

This chapter describes the control condition of the study, the workflow baseline. It is a single scripted Incident Commander (IC) that runs the same evacuation as the agentic system. It sees the same scenarios, may use the same commands and writes to the same event log. The main difference is the way it decides. The IC follows a written command doctrine step by step, and no language model takes part in its coordination decisions.

A weak control would prove nothing, so we built the baseline on an established emergency command doctrine. We froze it before the novel disruptions were revealed. Section 5.6 explains how to read its score.

5.1. Design

The baseline is a plain Python program without a workflow engine such as Camunda or Temporal, which would add a second place where the two systems could differ. The IC runs on the same engine as the ten agents (Section 4.3). It replaces one step only, the step in which an agent would call its language model. We call this step the seam. Everything after the seam is the agentic side's own code.

The IC takes over the two coordination agents that reason with a model, the command post (BIS) and the transport dispatcher (Hochbahn). The code of the other agents is shared, but a baseline run does not start them. It is an offline walk through the scenario, with no clock, no language model and no agent processes. The execution model (Section 5.4) carries out the IC's orders in place of the bus drivers, the police and the medic. In the road-closure scenario, the replies come from scripted messages in the scenario file. This is the largest difference between the two conditions apart from the command post itself, and Table 8.1 lists it as deviation 7.

The doctrine is the German service regulation DV 100, *Command and Control in Operations* (*Führung und Leitung im Einsatz*), under which the Hamburg fire service works [FwDV04]. The code follows its elements one by one. A citation table in the code ties the rules to the doctrine, with 87 entries over 24 doctrine elements (Section 10.5). The American Incident Command System (ICS) is the second anchor. Its doctrine is written down in the National Incident Management System (NIMS) guidance [DHS17], and it is a well-studied command system in the literature [BTA06; Cha17]. Where DV 100 gives too little detail, we borrowed from ICS. In DV 100 terms, a multi-agency evacuation of 100 residents is a level C or D incident, which calls for one commander with a command group or staff [FwDV04, p. 23]. Central command is therefore the textbook form here (Section 2.1). The IC implements three of the staff functions [FwDV04, pp. 13–14]. The situation function (S2) keeps the situation picture. The operations function (S3) plans the mission and chooses the next phase. The logistics function (S4) turns a decision into transport, escort and medical orders.

5.2. The Command Cycle

DV 100 describes command as a repeating cycle, the command process (*Führungsvorgang*), with three stages: situation assessment, planning and issuing orders [FwDV04, pp. 24–25]. Control comes from repeating the situation assessment, which checks whether the orders are still right [FwDV04, p. 24]. In the baseline, one turn of this cycle runs for every wake, that is, for every scenario event or reply from another agent.

The seam is the planning stage. Three checks run there, one after the other (Figure 5.1).

1. **Contingency check.** If an anticipated disruption has made the committed plan wrong, the IC issues a corrective order and stops (Section 5.3).
2. **Fallback check.** If a report does not fit any known message, or a reply is overdue, the IC issues a holding order and stops (Section 5.3.2).
3. **Phase ladder.** Only if the plan still fits and the situation is understood does the IC choose the next phase of the mission.

The order follows the doctrine. The IC does not push a plan forward when it knows the plan is wrong or does not understand the situation. A correction written for the case comes before a general hold. The resulting orders then pass through the agentic engine's own code for checking, committing and messaging. A parity test checks that they are the agents' own functions.

The phase ladder. The mission has four phases. Each has a time budget in simulated minutes and a condition that must hold before it can start (Table 5.1). DV 100 leaves the phases of a particular mission to the user, so we designed them ourselves. The operations function chooses the phase from the situation picture alone, so the same situation always leads to the same decision. It also computes the allocation of residents to shelters again on every pass, so a correction reaches the schedule without an extra branch.

Table 5.1.: The baseline's phase ladder. Each phase starts when its condition holds, and must close within its time budget. The last column maps the phases to the storyline steps of Section 3.3.

Phase	Budget	Starts when	Orders	Steps
Intake	20 min	The alarm arrives	Ask the home how many residents; ask both schools for free beds; status update	1–3
Transport allocation	15 min	Resident count and free beds are both known	Status update that states the decision; no transport is ordered yet	4–5
Issuing orders	90 min	The allocation gives a pickup time	Order the escort and the medical cover; one dispatch order per bus run; wait for each run to be confirmed	5–9
Mission report	10 min	Residents have actually been set down	Report to ZKD	10

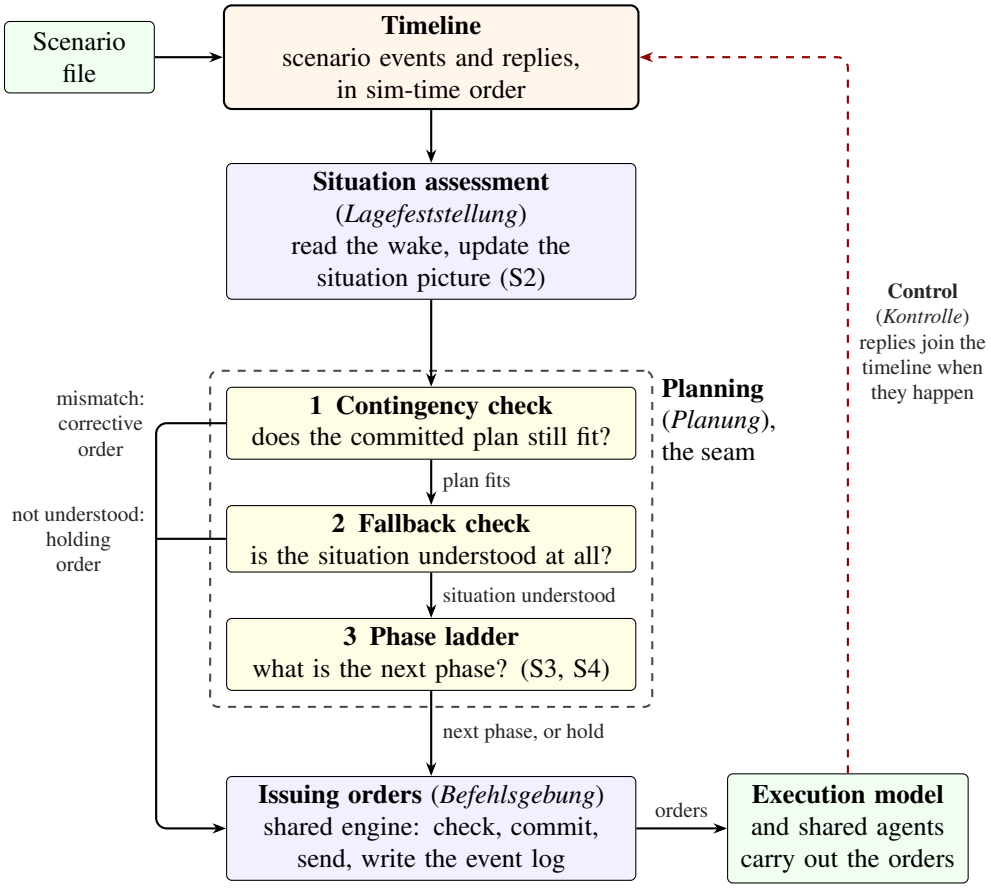


Figure 5.1.: One turn of the baseline's command cycle. A wake is read into the situation picture. At the seam, three deterministic checks run in a fixed order. A plan that no longer fits is corrected first. A situation that is not understood is held next. Only then does the phase ladder advance the mission. Everything from issuing orders onward is the agentic engine's own code. The execution model carries out the orders and its replies re-enter the timeline, which closes the loop.

How the fleet is used. Every scenario declares two buses of 50 seats and marks the second as a reserve. The baseline's run schedule does not read that mark. It gives each run to the bus that is free first, so both buses leave at the start. The agentic transport agent keeps the reserve parked by default. Both sides have the same fleet and both may send bus 2, so we count this as a difference in decisions. Releasing bus 2 later may cost the agentic side time. We did not measure how much, and it does not explain the residents who were never delivered, because bus 2 was used in all 150 novel-arm runs of the three full models (Section 11.2).

5.3. Handling Disruptions

5.3.1. Anticipated Disruptions

Three types of disruption were known while we built the baseline, and the IC has a prepared branch for each. Only the road closure is part of the scored scenarios (Section 3.4).

- **Road closure.** The IC drops the shelter it can no longer reach and runs the allocation again over the shelters that remain.
- **Shelter capacity lost.** The IC runs the allocation again against the smaller capacity.
- **Bus breakdown.** The IC schedules the remaining runs over the buses it still has.

Each branch compares the target (*Soll*), which is the order the IC has committed, with the actual state (*Ist*), which is the current situation picture. The branch fires when the two do not match, and clears itself once the correction makes them match again.

The IC has no routing command, because routing belongs to the transport side and the drivers. It therefore answers a road closure over destinations. Losing a shelter would leave 40 of the 100 residents without a bed, so the baseline's road-closure file declares 60 residents (Section 3.4). All 20 anticipated-arm baseline runs scored $S = 1.000$. On the road closure that score is measured against the reduced ceiling of 60, so the share of the full 100 residents is 0.60.

5.3.2. Novel Disruptions: the Generic Fallback

A novel disruption has no prepared branch. The baseline must not answer it with a fresh plan, because that ability belongs to the treatment condition. DV 100 requires a report back when the situation no longer matches the order [FwDV04, p. 37]. It does not describe a holding order. We chose one because it is the most careful response that needs no new plan. Two cases trigger it, and neither names a particular disruption.

- **A report outside the catalogue.** A message arrives whose type the IC does not expect and no contingency branch handles.
- **An overdue reply.** A decision waits for a reply, and its time budget has run out.

In both cases the IC reports to ZKD, tells the Augustinum to hold the residents, and sends a hold-departure advisory to the transport dispatcher. The IC itself took over Hochbahn, so this advisory lands in its own mailbox and reaches no driver. The hold stops new orders, but a bus already on the road keeps going. We wrote the fallback before any novel disruption existed. A test reads its source code and fails if it names any message type, so it cannot hold special cases for particular disruptions.

5.4. The Execution Model

A scripted scenario can only test a decision if the outcome depends on the orders. If the scenario file said which shelter received how many residents, an IC that sent everyone to a closed shelter would score as well as one that planned around the closure. The execution model prevents this. It is a shared module that plays the physical world and makes no decisions of its own. It reads the dispatch orders the IC commits, drives them over the real road distance, and reports what each shelter admitted. Three rules keep a wrong plan from scoring well.

- **A run lands only if it is still under way.** A bus that was recalled on the way delivers nobody.
- **A closed or full shelter admits nobody.** The residents who were turned away do not go

back into the pool.

- **Nobody boards after the flood arrives.** The flood reaches the zone at 12:00, and the scorer applies the same limit.

Only the baseline's scenario files declare such a world. The agentic scenarios reach the same effect through live agents. This difference has two sides. Without a declared world, the baseline would receive its drop-off confirmations from the script, whatever it had ordered, and the agentic side never had that help. In this respect the world works against the baseline. At the same time, the execution model carries out every order it receives, while the agentic side depends on bus drivers, police and a medic that each reason with a language model. In this respect the world favours the baseline. We did not measure how large either effect is (Section 10.5).

5.5. Freeze and Determinism

We froze the baseline in version control on 2026-09-10, before the novel scenarios arrived (Section 8.3). One later change added a switch that turns re-planning off for the no-replan control (Section 8.5.4). It does not change what the doctrine does, and the pre-registration discloses it in Amendment 6.

The baseline starts from a small command-line tool and reads the same scenario files as the agentic side. Its decision path has no randomness, no model and no network, so a run depends on its scenario file alone.

5.6. What the Baseline Cannot Do

The limits of the baseline follow from what pre-scripted branching is (Section 2.1). An unanticipated disruption has no handler, and the IC cannot write itself a new plan at run time. Because the IC merges BIS and Hochbahn into one node, the link between them also disappears, and with it four commands that only made sense on that link. One of them is the deadline extension, which under central command nobody is there to negotiate. In return, the baseline is fast, costs nothing to run, and every decision can be traced to a written rule.

What this means for reading the results. The original design expected the baseline to be at its weakest on novel disruptions, where only the holding order is left. The evaluation did not test that expectation, and the baseline's $S = 1.000$ on the novel arm says nothing about whether it adapted (Section 10.2).

6. Data Architecture

Chapter 4 showed that the ten agents never talk to each other directly and that every message, state change and decision passes through one PostgreSQL database (Figure 4.1). In this chapter we describe how that database stores the actors, their states, the events between them and the resources they use. The same structures run the simulation and form the record that the evaluation scores.

6.1. Design Principles

The database has three jobs in the system.

1. It is the **state store**. The current state of every agent lives in the database and nowhere else. An agent that restarts reads its state back from there.
2. It is the **message broker**. A message from one agent to another is a row in a table. The database tells the recipient that the row has arrived.
3. It is the **scoring backbone**. Every agent, and the workflow baseline as well, writes what it received and what it decided into one append-only log. The evaluation computes every score from that log and from nothing else.

This is what we mean by *database-centric*. The database is the single source of truth, and all three jobs read and write the same store. There is no second copy of any state in a file or in memory that could drift away from it.

We keep one limit clear from the start. Using a shared store to coordinate independent programs is an old idea. Blackboard systems [Nii86] and tuple spaces [Gel85] did so decades before this work (Section 2.5). We therefore do not claim a shared database as a new way for agents to coordinate. What we do claim concerns evaluation. One append-only log is shared by both systems under test and by the scoring code. This makes the comparison fair, checkable and repeatable. Section 6.4 makes this argument in full. The log follows the idea of event sourcing, where the history of events is stored and every later view is derived from it [Ove+21].

6.2. Data Model

The model has four kinds of thing: actors, their states, the events between them, and the physical resources they use. The ten agents are the actors. Vehicles are not actors. A bus, the patrol car and the ambulance are rows in a fleet table that a person drives (Section 4.1). Scenarios are not stored in the database. They are files that a dispatcher reads and turns into messages at the right simulated time.

The database holds twelve tables. We group them by the job they do (Table 6.1 and Figure 6.1).

Runtime tables. Five tables keep the simulation running: the agent states, the mailboxes, the world reports, the prompt records and the timers. They are cleared between two runs.

The audit table. The event log is the only table that is never changed or cleared. It holds the history of every run.

Sensing tables. Six small tables hold the facts that some agents perceive rather than hear about in a message. Examples are an active flood alert, the remaining beds of a shelter, a road closure and the status of each vehicle. The sensing step of BIS, Hochbahn, the police officer and the paramedic reads them.

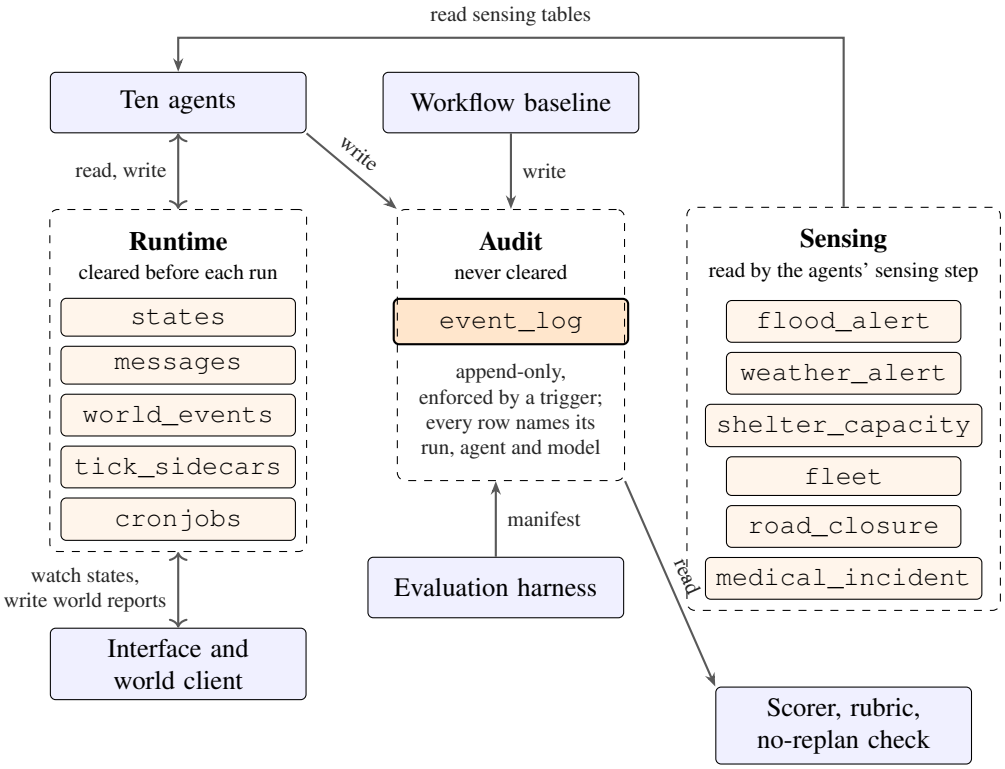


Figure 6.1.: The twelve tables of the database, grouped by job. The runtime tables are cleared before each run. The event log is append-only and holds every run. Both systems write to it, and the scoring code reads only from it.

6.2.1. Tables

One document per agent. The `states` table has one row per agent. The row holds the agent's whole state as one JSON document: its inbox, its current step, its remaining plan, its history of finished steps, its view of the fleet and its outgoing messages. Every save replaces the whole row. The row also carries the run identifier and the model name, so that a state can always be tied to the run and the model that produced it.

We did not split this document into separate tables. The runtime always loads and saves an agent's state as a whole, so it gains nothing from a finer structure. The evaluation does not read

Table 6.1.: The twelve tables, grouped by job.

Table	What one row holds	Life cycle
Runtime		
`states`	The full state of one agent, as one document	Overwritten on every save
`messages`	One message: sender, recipient, type, body, and the times it arrived and was read	Marked as read, never deleted during a run
`world_events`	One report from the world that an action has finished or a vehicle has stopped	Marked as read
`tick_sidecars`	The full prompt and the full answer of one model call	Written before and after the call
`cronjobs`	One timer an agent has set	Record only
Audit		
`event_log`	One thing an agent received or decided	Append-only, never cleared
Sensing		
`flood_alert`	An alert for a facility, with the number of people to evacuate	Read by sensing
`weather_alert`	A weather warning	Read by sensing
`shelter_capacity`	The remaining beds of one shelter	Read by sensing
`fleet`	One vehicle: its driver, type, status, mission, position and arrival time	Read by sensing
`road_closure`	A closed road	Read by sensing
`medical_incident`	A medical incident and whether it is resolved	Read by sensing

this document either. It reads the event log (Section 6.3), which records every change to the step and the plan as it happens. A normalised state would add joins and migration work without serving a single measurement.

6.3. Append-Only Event Log

The event log is the record of what happened in a run. Every agent writes to it, and no one may change what has been written.

The row. Each row has the same seven fields:

- an **identifier** that counts up within one agent's log (`evt-0`, `evt-1`, and so on);
- the **wall-clock time** and the **simulated time** of the entry;
- a **kind** and a **subkind**, which say what sort of entry it is;
- a one-line **summary** for a human reader;
- a **details** document with everything else.

Three more columns place the row: the **run**, the **agent** and the **model**. With them, one table holds every run of every agent, and any single run or agent can be selected from it.

The kinds of entry. Six kinds cover everything that is written to the log.

`event_received` An event woke the agent: a message, a finished step or a timing conflict. The details hold the event as it arrived.

`llm_response` The agent made a decision with its model. The details hold the reasoning, the committed step, the remaining plan, the outgoing messages, the token counts and the latency. A dispatch by the clock that needs no model uses the same kind, with the subkind `cron_fire` and zero tokens.

`world_ack` The world confirmed that an action had finished. The details hold the vehicle and its real position.

`decision` The workflow baseline made a decision. This is the baseline's counterpart to `llm_response` (Section 6.4).

`run_manifest`, `run_outcome` The evaluation harness recorded how a run was set up and how it ended (Section 6.9).

Measurement fields. Every decision row carries the same five fields for the evaluation, whether or not anything went wrong. They are the list of rule violations the answer tried, their count, a fingerprint of what the model saw, the retry count and a failure code. Because the fields are always present, a zero means that we measured zero. It does not mean that the value is missing. The fingerprint is a hash over the agent's observable situation, taken before the model call. It lets the evaluation see whether two decisions were made from the same observation (Section 4.3).

Three guarantees. The database, not a convention in the code, enforces how the log behaves.

1. **Append-only.** A trigger rejects every update and every delete on the table. Once a row is written, it stays as it is. A run cannot be quietly edited or removed after the fact.
2. **Written once.** The run, the agent and the row identifier together form a unique key. An agent saves its whole in-memory log each time it saves its state, and rows that are already stored are skipped. Saving twice therefore never creates a duplicate.
3. **Read in order.** Each row also gets a sequence number when it is inserted, and the log is read back in that order. Sorting by the identifier would put `evt-10` before `evt-2`.

Between two runs, the runtime clears the runtime tables so that no agent starts with the state or the unread mail of the run before. The event log is left out of this reset on purpose. It is divided by run, so old runs do not disturb a new one, and the trigger would reject the delete in any case.

6.4. The Event Log as Evaluation Backbone

The event log is shared by three parties: the agentic system, the workflow baseline and the code that scores them. Figure 6.2 shows this arrangement. It is the methodological contribution of the data layer.

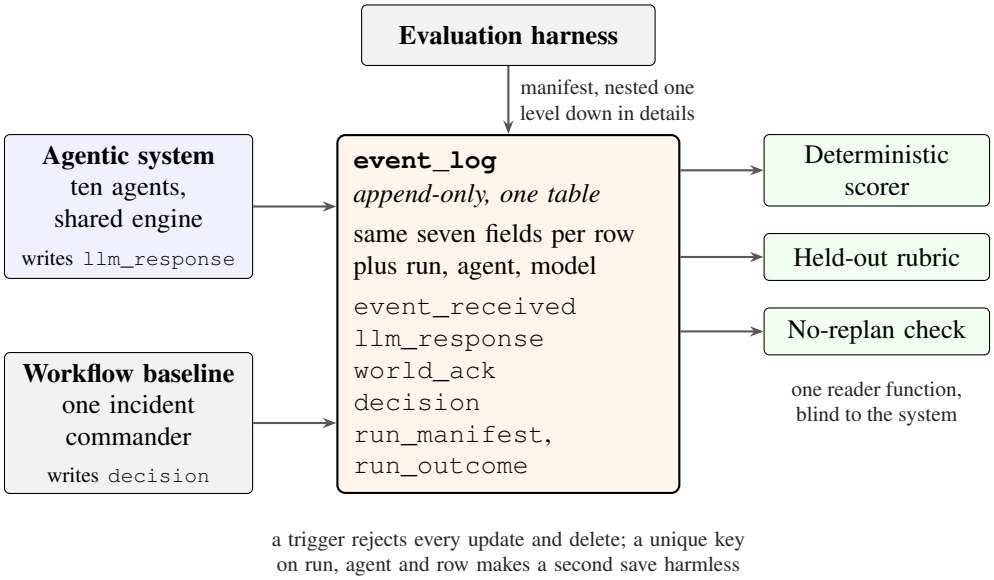


Figure 6.2.: The event log as the backbone of the evaluation. Both systems write rows of the same format into one append-only table. The baseline's decision rows carry their own kind because no model was called. The harness records each run's setup in the same log, one level deeper than any field the scorer reads. The scorer, the rubric and the no-replan check read the same rows for both systems.

Both systems write the same rows. The agentic agents write their rows through the shared engine. The workflow baseline has no agent state and no model, so it uses a small writer of its own. That writer produces the same row format, with the same numbering and the same fields, into the same table. There is one difference, and we made it on purpose. The baseline's decision rows have the kind `decision` and an empty model name, and their token counts and latency are zero. We did not label them `llm_response`, because no model was called, and that label would misreport the control condition. The baseline computes its observation fingerprint by the same rule as the agents.

One reader for both. The scorer reads the log through one function, and that function runs in the same way for both systems. It does not know which system wrote a row. It sees only rows, their kinds and their details. The rubric and the no-replan check (Section 8.5.4) read the same rows.

The harness cannot change a score. The harness writes its own rows into the log, to record how each run was set up. Their content sits one level deeper inside the details than the fields the scorer reads. A manifest that carried a route length at the top level, for example, would

add kilometres to the run it describes. A test checks that scoring a run with its manifest rows gives the same result as scoring it without them.

What the shared log gives the evaluation.

- **Deterministic scoring.** Every outcome is a fixed function of the log (Section 8.4.4). No human judges a run, and no model judges one either. The same log always gets the same score.
- **Checkable parity.** Because both systems write the same rows into the same table, the claim that they are comparable can be tested against shared code instead of stated. An executable parity test suite does this, and lists the seven places where the two systems still differ (Table 8.1).
- **Re-scoring.** The log keeps every run. When a scoring rule changes, every run can be scored again from its rows, without running it again. Because the log is append-only, a run that is dropped from an analysis still leaves its rows behind.

6.5. Runtime State and Message Passing

Saving state. An agent saves its state after every wake. In one transaction, the save replaces the agent's row in `states` and appends any new rows of its log to `event_log`. After the transaction has committed, the agent announces the change (Section 6.6).

Sending and reading messages. To send a message, an agent inserts a row with the recipient's key into the `messages` table. To read its mail, the recipient selects its unread rows and marks them as read. It does not delete them. The mailbox therefore keeps a full record of every message and of when it was read. The rows are locked while they are taken, and rows locked by another reader are skipped, so no message is ever read twice.[1]

One address per agent. Some senders name a recipient by a short alias, such as a vehicle call sign, rather than by the recipient's key. All messages pass through one insert step, and that step maps every alias onto the recipient's key. If a recipient cannot be resolved, the message is logged as undeliverable instead of being lost. Since every message passes this one point, a test follows every alias from sender to recipient.

Messages that are recorded but not delivered. Each sender has a routing table. For some recipients it marks a message as *audit*, meaning that it should be kept for the record but not delivered. Such a message is stored under a separate key space that no agent reads. It stays available to the evaluation and never wakes an agent.

[1] This is PostgreSQL's `SELECT ... FOR UPDATE SKIP LOCKED`, see `https://www.postgresql.org/docs/current/sql-select.html`.

World reports. When a vehicle arrives, or stops half-way because its plan changed, the world writes a row to `world_events`. In a live run the source is the map in the browser. In the evaluation it is the headless world client (Section 8.1). The agent reads these rows in the same way as its mail and takes its real position from them (Section 4.5).

Prompt records and timers. Each model call writes one row to `tick_sidecars` before the call and updates it after. The first write lets the interface show that a call is in progress. The second adds the model's full answer. Token counts and latency go to the decision row of the event log instead. The `cronjobs` table records the timers that an agent sets, but it does not run them. The clock that fires the timers stays inside each agent program. Moving it into the database would change how and when an agent acts, and that behaviour is part of what we measure.

Separation between runs. The `messages` and `world_events` tables have no run column. Only the reset that the harness performs at the start of each cell keeps one run's mail out of the next. We record this as a limitation of the schema. The event log is not affected, because it carries the run on every row.

6.6. Live Notifications

An agent should wake as soon as mail arrives, without asking the database again and again. Reactive tuple spaces solve the same problem by running code when new data arrives [OZ99]. PostgreSQL offers a publish-and-subscribe mechanism for this, called LISTEN and NOTIFY.[2] A program listens on a named channel, and any other program can send a short notification on that channel.

Each agent has three channels: one for its state, one for its mailbox and one for its world reports. Whoever writes a message, a world report or a state sends a notification on the matching channel. The notification is sent from the writing program after its transaction has been committed, not from a trigger inside the database. By the time the recipient wakes and reads, the new row is therefore visible to it. Every writer goes through the same insert functions, so no writer can forget to notify.

A notification can still be lost, for example when a connection drops. We therefore kept a slow poll as a safety net. An agent waits for a notification or for five seconds, whichever comes first, and then looks at its mailbox. A lost notification makes an agent slower, but it never leaves the agent stuck. No part of the system depends on a notification arriving to be correct.

The observation interface uses the same mechanism. It listens on the state channel of every agent and redraws the map when a state changes (Chapter 7).

[2] `https://www.postgresql.org/docs/current/sql-notify.html`

6.7. Design Alternatives

Two other designs could do parts of this work. A message broker such as RabbitMQ or Apache Kafka could carry the mail between agents, and a dedicated event store could hold the log. We kept all three jobs in one PostgreSQL database for three reasons.

First, one save is one transaction. An agent's new state and its new log rows commit together or not at all (Section 6.5). With a separate broker, a message and the state change behind it would sit in two systems with no shared transaction. A crash between the two writes would leave them out of step, and the log would no longer match what the agents did.

Second, the agents and the scorer read the same rows with plain SQL. An event store would hold the log well, but agent state and mailboxes would still need a second store beside it.

Third, the load is small. The 310 scored runs wrote 83,661 rows to the event log. An agentic run wrote a median of 345.5 rows, with the middle half between 294 and 392, and a baseline run wrote 25 to 29. At this scale a broker's throughput brings no benefit. The one broker feature the agents need is a prompt wake-up, and LISTEN and NOTIFY provide it.

The cost is a single point of failure. If the database is down, no agent can load its state or receive a message. Before the runtime starts the ten agent programs, it checks that the database answers, and it refuses to start if it does not.

6.8. External Data Sources

Two kinds of data come from outside the database. Addresses are turned into coordinates by a geocoder, and drives are routed over real Hamburg roads with OpenRouteService.

Both services are called through caches on disk. Each agent keeps these caches in its own working directory, together with its program log. No state is kept there. The caches serve two ends. They keep the number of calls to the routing service low, and a repeated query returns the stored route. If routing fails, the runtime falls back to a straight-line estimate and marks the action as degraded, as Section 4.6 describes.

The route length also enters the evaluation. When a bus is dispatched, both systems record the kilometres of its route, measured along the same routed path. The scorer reads this value from the log for both sides in the same way.

6.9. Determinism and Replay

A run can be scored again at any time if its log is complete. Running it again gives a similar run, but not the same one. We record the following so that re-scoring is always possible.

One run identifier. All ten agent programs of one run share one run identifier. The runtime sets it when a run starts and passes it to every agent. Every row of the event log carries it, so the ten agents' rows of one run can be joined into a single history.

The manifest inside the log. At the start of each run the harness writes a manifest row into the event log. It names the system, the model and its digest, the scenario and its hash, the time scale, the seed, the code version and the hash of the scoring thresholds. Section 8.1 lists the full content. Because the manifest sits in the append-only log, the setup of a run cannot be separated from the run.

Seeded, but not reproducible. Every model call receives a seed that is derived from the scenario, the time scale and the replicate number. The seed is logged. A seeded call to a language model is still only close to deterministic, because batching and GPU configuration inside the inference server change the sampling [Yua+25]. We therefore claim that runs are seeded and logged, not that they can be reproduced at the level of each model answer. This is also why every cell is repeated (Section 8.7). Models served from a hosted endpoint cannot be pinned to one version. For them, the model's details are logged with every run, so that a change of model would be visible.

Why runs cannot run in parallel. The data layer sets one practical limit on the evaluation. The `states` table has one row per agent key, and the agent keys are fixed names. Two runs on one database would overwrite each other's state. The agents' working directories are fixed paths as well, and the route cache is a single file. The evaluation therefore runs its cells one after another (Section 8.1).

7. Observation Interface

The agents of Chapter 4 run as ten separate programs, and they meet only in the database (Chapter 6). A running simulation is therefore hard to follow from the outside. In this chapter we describe the observation interface, a small web application that shows a live agentic run on a map of Hamburg. It is supporting material. It helped us build and debug the agents, and it gives a reader a way to look at a single decision. It played no part in scoring.

7.1. Purpose

The interface has two jobs. The first is to show a run as it happens: where each vehicle is, which step each agent is working on, and which messages are moving between them. The second is to make each reasoning step inspectable. When an agent does something unexpected, we can open the exact prompt it received and the exact answer the model gave. This is how we found most of the faults in the agents while we built them.

The interface makes no decisions. Each agent is the only writer of its own state row, and the interface never changes it. The interface writes to the database in three places only. It records the world's acknowledgements when a vehicle finishes a drive (Section 7.4). It enqueues the scenario's events into the agents' mailboxes. And it clears the run tables when the operator resets the simulation. Because all state lives in PostgreSQL, a restart of the interface loses nothing. The backend reads the states again, and a browser that reconnects receives fresh snapshots.

The interface serves only the agentic system. The workflow baseline has no live view. It runs from its own command-line tool (Section 5.5), and its vehicles are moved by its execution model (Section 5.4), so it needs no map to close a drive.

7.2. Backend

The backend is a Python web service built on FastAPI.[1] It sits between the database and the browser and has four parts. Figure 7.1 shows how data moves between them.

Watchers. There is one watcher for each of the ten agents. A watcher reads its agent's row in the `states` table and compares it with the last version it has seen. It then sends only the keys that changed. The watcher does not poll in a loop. It waits for the notification that the agent sends on its state channel after each save, with the same five-second fallback that the agents use (Section 6.6). Until an agent has saved its first state, the watcher checks every half second.

[1] `https://fastapi.tiangolo.com`

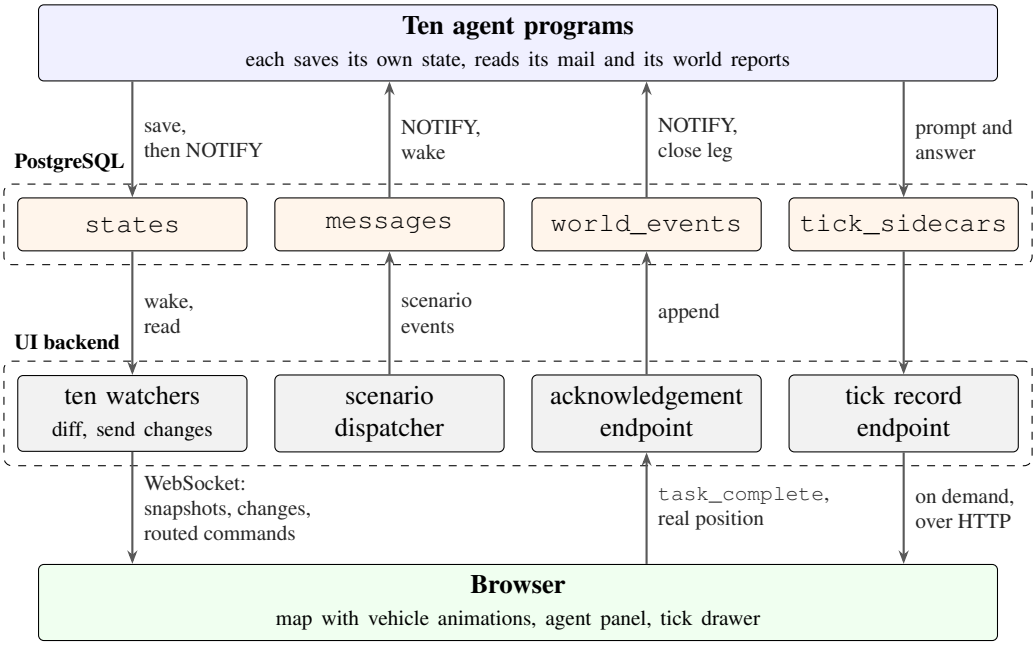


Figure 7.1.: How data moves through the observation interface. Each column is one path. State goes out from the agents to the browser. Scenario events go into the agents' mailboxes. Acknowledgements come back from the browser's animations and become the agents' world reports. Prompt records are fetched only when a tick is opened. Every path passes through the database, and the interface never writes an agent's state.

One WebSocket. All updates reach the browser over one WebSocket connection. When a browser connects, it first receives the current snapshot of every agent. After that it receives only the changes. Each message names the agent it belongs to. The four agents that move a vehicle also produce commands, for example "drive the bus to this address". Before the backend sends such a command, it adds a road route from OpenRouteService, the target coordinates and the expected duration. When the routing service fails, the route falls back to a straight line, and the map draws that line in red so the fallback is visible.

Scenario dispatcher. When a run starts, the dispatcher loads the chosen scenario file. It uses the same loader as the workflow baseline, so both systems read every scenario in the same way. The dispatcher then waits until each event's simulated time and writes the event into the recipient's mailbox in the `messages` table. The wait follows the same time scale that the agents read, so the dispatcher, the agents' clocks and the map animation stay in step.

Simulation control. The operator can start, stop and reset a run. A start creates a new run identifier and passes it to all ten agent programs, together with the number of residents given in the scenario. Every row the agents write to the event log then carries this run. A start is refused when the database cannot be reached or when a run is already active. A reset stops all ten agents and empties the run tables: the states, the mailboxes, the world reports, the prompt records and the timers. The event log is never touched, because it is append-only (Section 6.3).

7.3. Frontend

The frontend is a React application written in TypeScript. The map uses Leaflet with OpenStreetMap tiles,[2] and one central store keeps a separate part for each agent. Figure 7.2 shows the screen during a run.

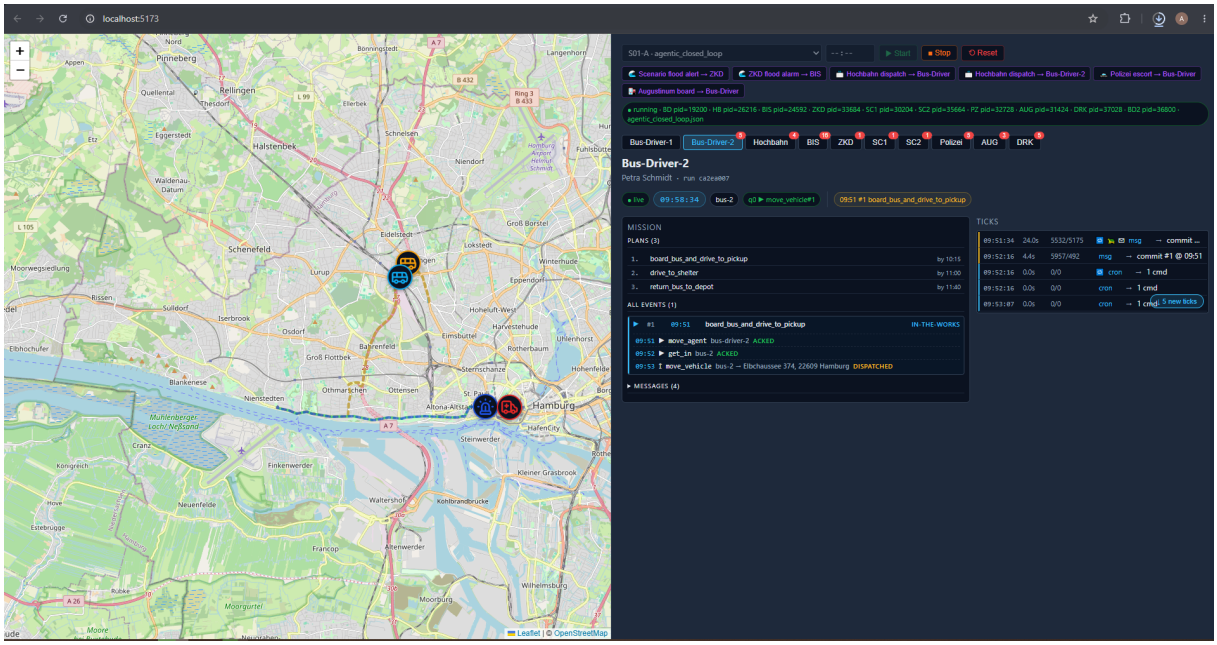


Figure 7.2.: The observation interface during a run of the undisrupted scenario. Left: the map of Hamburg with the two buses, the police escort car and the ambulance, each on its own route. Right, from the top: scenario selection and run controls, the process identifiers of the ten agents, one tab per agent, and the panel of the selected agent. Here this is Bus-Driver-2, with its three planned steps and their deadlines, the actions of its current step with their status, and its list of reasoning ticks. Map data © OpenStreetMap contributors.

The map. The left half of the screen is the map. Each vehicle has its own marker and colour, and its current route is drawn as a line. When the backend sends a command, the frontend puts it in a queue for that vehicle. An animation then moves the marker along the route. The animation reads the same time scale as the agents, so the map and the agents' clocks agree on how long a drive takes.

The agent panel. The right half has one tab for each agent. The panel of a vehicle driver shows the current simulated time, the vehicle it uses, its remaining plan with a deadline for each step, and the actions of its current step. Each action carries a status, such as dispatched or acknowledged. The coordinating agents move no vehicle, so their panels show their state without one. Next to the plan, a list shows every tick of the agent: when it started, what woke it, how long the model took, how many tokens it used and what it committed.

[2] Map data © OpenStreetMap contributors, available under the Open Database License, see `https://www.openstreetmap.org/copyright`.

The tick drawer. A click on a tick opens a drawer with its full record (Figure 7.3). The drawer shows what woke the agent, the system prompt and the user prompt that the model received, the raw answer of the model, and the plan the runtime read from that answer. This record is not sent over the WebSocket. The browser fetches it from the `tick_sidecars` table only when a tick is opened.

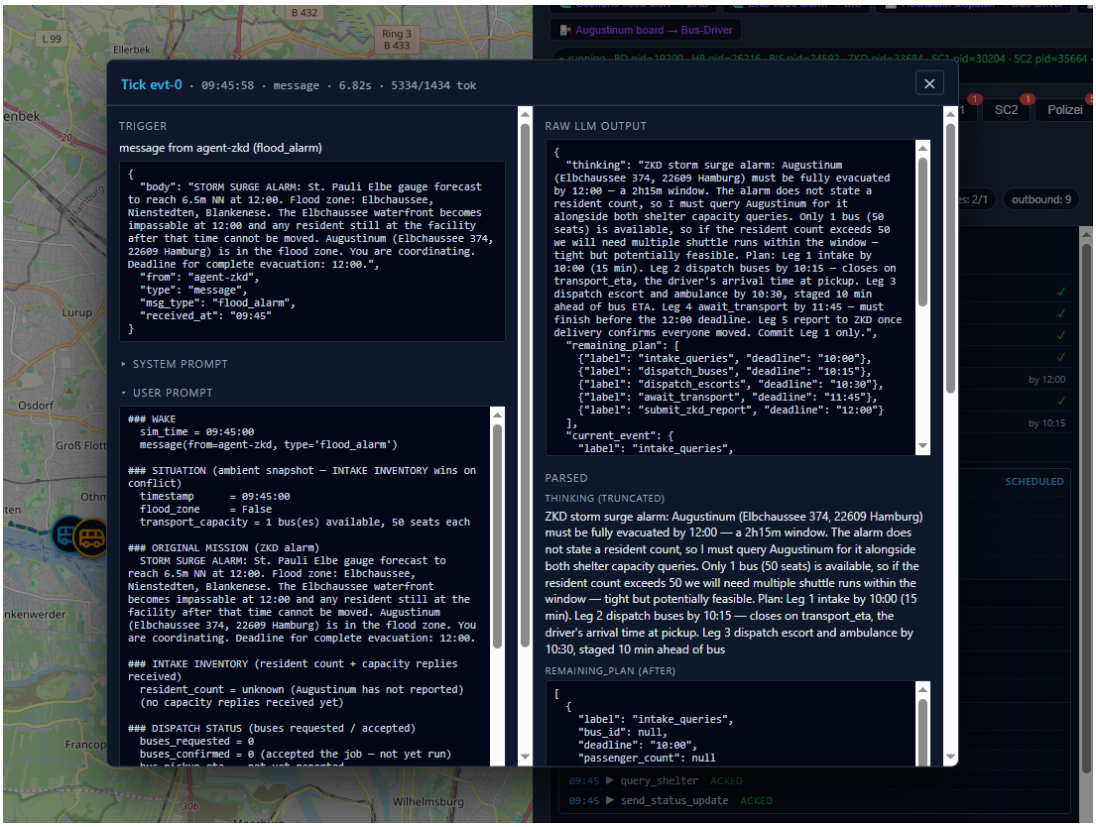


Figure 7.3.: The tick drawer for one reasoning tick of BIS, woken by the storm surge alarm from ZKD. Left: the trigger message and the prompts the model received. Right: the raw model output and the parts the runtime parsed from it, here the model's reasoning and its new remaining plan. The header gives the simulated time, the latency (6.82 s) and the input and output tokens. The prompt lists one available bus, because BIS's fleet snapshot leaves out the reserve bus-2.

For research, the drawer is the most useful part of the interface. A reader who doubts a decision can open the tick behind it and see what the agent knew and what the model wrote. The prompt records are cleared with the next reset, so this view serves a live or just-finished run. The event log keeps its own rows for every decision.

For testing, the controls also offer a row of buttons that send one message by hand, for example a flood alert to ZKD or a dispatch order from Hochbahn to a bus driver. A message sent this way goes into the mailbox like any other message, and it is accepted only while a run is active.

7.4. The Acknowledgement Channel

There is one place where the interface does more than watch. A drive is closed only when the world confirms that it is finished (Section 4.5). During a live run, the browser's animation plays the role of the world.

When a vehicle's animation reaches the end of its route, the frontend sends a `task_complete` message back to the backend. The message carries the command's number, the vehicle and the vehicle's final position. The backend decides from the vehicle which agent owns it: the first bus and the driver's car belong to Bus-Driver-1, the second bus to Bus-Driver-2, the patrol car to Polizei and the ambulance to DRK. It then appends the acknowledgement to that agent's `world_events`. The agent wakes, closes the step and takes its real position from the report. If an agent changes its plan while a vehicle is still moving, the animation stops, and the frontend sends a `vehicle_halted` message with the position where the vehicle stopped. This message only updates the location. It does not mark any action as done.

Because of this loop, the interface matters for the evaluation even though it scores nothing. A run with no acknowledgement source has no bus that ever arrives.

7.5. Runs Without a Browser

The evaluation runs its cells one after another, and no browser is open during them. For these runs a headless world client replaces the browser (Section 8.1). It follows the commands the backend emits and sends the same kind of acknowledgement at the end of each drive.

We could have let the engine acknowledge its own commands instead. We decided against this. A self-acknowledging engine would remove the handshake with the world that an agent has to complete, and the evaluation would then score the architecture on a property it does not have.

Two acknowledgement sources must never run at the same time, because every drive would then be counted twice. Before each agentic cell, the harness therefore sends a reset notice to the interface backend. The backend forwards it to every open browser tab. A tab that receives it clears its stored state and reloads, so it does not keep showing the old run over empty tables. The backend also replies with the number of connected tabs. If any tab is connected, the cell does not start. If no interface backend is running, the notice fails quietly and the cell starts as normal.

7.6. Summary

The observation interface shows a live agentic run on a map of Hamburg and lets a reader open the full record of any reasoning tick. A backend watches each agent's state through database notifications and sends only the changes to the browser over one WebSocket. The interface does not decide anything and never writes an agent's state. Its one active role is to acknowledge finished drives, and in the evaluation a headless client takes over this role with the same messages. Since an evaluation cell refuses to start while a browser tab is open, the scored runs were produced without the interface in the loop.

8. Method and Evaluation Design

This chapter sets out what the evaluation measured and how it reached a verdict. We describe the design as it was registered before the evaluation ran, and name the amendment wherever the execution departed from it. Chapter 9 gives the results, and Chapter 10 examines the design itself.

8.1. Study Design

We compare two coordination systems on the same evacuation task. The *treatment* is the database-centric agentic system of Chapter 4. The *control* is the workflow baseline of Chapter 5, a single scripted Incident Commander. The main independent variable is the coordination architecture at the command post. The baseline also simulates the execution of orders, while the agentic side runs its field agents live (deviation 7 below).

Parity. Apart from this, everything is held constant, mostly by reusing code. The baseline engine is a subclass of the agentic engine, and 27 of its attributes are the same function objects the live agents use. An executable conformance test checks parity on five axes: information horizon, action vocabulary, world model, scenario inputs and output substrate. Seven known deviations remain, and Table 8.1 lists each with its direction. Four only label or address the records and cannot change a score. Two favour the baseline, and one works in both directions.

Pairing. Paired runs use matched scenario files, read by the same loader, and write to the same append-only `event_log`. The files are not byte-identical. The baseline files declare a world (deviation 7), except the road-closure file, whose drop-offs come from the script, so its score does not depend on the baseline's orders (Section 10.2). In the same file one school cannot be reached and only 60 residents are declared, so its ceiling is 60 (Section 3.4). The road-closure cells are therefore not like-for-like (Section 9.2). The simulator has no model of resident behaviour, sensors or message latency, so only the policy varies between runs. The seed that every model call receives is derived from the scenario, the time scale and the replicate index, never from the model, so the agentic runs of one replicate draw the same seed, a design known as common random numbers. The baseline is deterministic and ignores the seed, so each paired difference compares an agentic run with a fixed reference. Seeded model calls are still only near-deterministic (Section 6.9), so every cell is replicated.

Unit of analysis. The unit is the run. Each cell of system, scenario, model and time scale is repeated $k = 10$ times. The variance on the agentic side is reported in full as a cost of the architecture.

Table 8.1.: The seven known deviations between the two conditions, with the direction of each.

#	Deviation	Effect on a result
1	A baseline message's id is its position in the run, not a clock time, so two runs of one scenario are identical.	None. No code reads it.
2	A baseline message names its receiver, because the offline walk has no dispatcher.	None. No code reads it.
3	A baseline record is labelled as a decision, not as a model response.	None. The scored facts do not change.
4	A baseline record has an empty model name, which marks the control condition.	None. The scorer reads no model name.
5	One node, the IC, holds both the command picture and the fleet picture.	Favours the baseline. This is the doctrine.
6	The IC plans its run schedule with road distance and travel time from the routing service.	Favours the baseline.
7	Only the baseline's scenario files declare a world, all except the road closure. The execution model carries out the IC's orders, while the agentic side runs its bus drivers, police and medic live with a language model.	Mixed. Removing scripted drop-offs works against the baseline. A simulator in place of live field agents favours it. Size not measured (Section 5.4).

Models. The design called for at least three language models across a range of capability, to test whether an advantage holds across models (H3). Three models completed the novel arm at 50 runs each: glm-5.2, deepseek-v4-pro and qwen3.5. The first two were fixed before the arm ran. The third slot changed after data were seen, which departs from the registered selection rule (Amendments 5 and 10 to 12, Table 8.4), so the novel arm's model set is exploratory. Partial series (Section 9.1) are never pooled and are compared on S only. Hosted `:cloud` model tags cannot be frozen to a version, and inference is not bit-exact even with a fixed seed [Yua+25], so drift is logged on every run but cannot be prevented. A failed or malformed model call is never retried and counts as a hard failure. A retry would give a weak model a second attempt that a strong model never needed.

Harness. One harness runs every cell. For each run it writes a manifest row into the append-only log: system, model and digest, scenario and hash, time scale, replicate, policy seed, git commit and whether the working tree was clean, threshold hash, and platform. A run's provenance can therefore be recovered from the log. Agent state and the route cache sit at fixed paths, so the sweep is serial. An agentic run on the anticipated arm took about 20 wall-minutes, and any drift in the hosted models is confounded with a run's position in the sweep.

Headless world client. On the agentic side, a bus leg closes only when the world acknowledges that the task is complete. In the first pilot only the browser's vehicle animation sent that acknowledgement, so without a browser no bus arrived and every agentic cell scored level 0. We ruled that pilot invalid and re-ran it. Every agentic cell instead runs a headless client that

plays the animations from the backend's own events, and refuses to start while a browser tab is connected, since both would acknowledge every leg.

8.2. Hypotheses

We registered four hypotheses before any evaluation run (Table 8.2). H1 and H2 are confirmatory and together answer RQ1. H3 and H4 are estimation questions without a significance test.

Table 8.2.: The registered hypotheses.

	Hypothesis	Arm	Form
H1	The agentic system is not worse than the baseline on survival, on disruptions the baseline has a written branch for.	Anticipated	Non-inferiority, $\delta_{NI} = 0.05$
H2	The agentic system is better than the baseline on disruptions the baseline could not have encoded.	Novel	Superiority, $\delta_m = 1$ level
H3	An H2 advantage holds across a range of models, with a capability threshold below which it disappears.	Both	Estimation
H4	There is a time scale at which the agentic system's thinking time starts to cost more than its coordination gains.	Both	Estimation

The registered claim needs both H1 and H2. Section 8.7 gives the decision rule for each scenario. The pre-registration adds a sign test across scenarios with Holm–Bonferroni correction, but it fixes no rule for combining the verdicts of several models into one verdict on a hypothesis. We therefore report every cell and state for each hypothesis how many of its confirmatory cells pass (Section 9.2 and Section 9.3). If both systems collapse to level 0 or 1 across the novel set, the campaign is in the *null zone* and is reported as uninformative rather than as a tie.

The design assumed, without testing it, that the scenarios demanded a change of plan (Section 10.2).

8.3. Scenarios and the Disruption Firewall

The suite has two anticipated scenarios and five novel ones. Section 3.4 describes each disruption, and Appendix B prints each disruption message word for word.

The baseline has a written branch for the anticipated disruption, the road closure, and both anticipated scenarios were visible during development. Two scenarios are a thin base for a per-scenario analysis, and we count this as a limitation. A third, a shelter closure, had no agentic counterpart, because the live shelter agent has no handler that lowers its beds. We excluded it from scoring on 2026-09-10 and kept it as the fixture for the negative control (Section 8.5.4).

Our company supervisor, the domain expert Rüdiger Höfert, wrote the five novel scenarios blind to the baseline. The brief named only the kind of disruption we wanted: compound, involving several actors, and unfolding during the mission. Their identifiers `novel_1` to `novel_5` were reserved before their content existed, and every supplied scenario had to be run, however hard, badly formed or unfavourable. The baseline was frozen and tagged in git on the morning of 2026-09-10 (Section 8.9), and the five scenarios were saved that afternoon, so the baseline could not have encoded them. One later change to baseline code added the switch for the negative control. Amendment 6 discloses it with evidence that the doctrine's behaviour did not change.

Blind supply carries the risk of the null zone, since a disruption may land too easy or too hard with no time to replace it. The brief was the only mitigation, and we decided against a behavioural pilot on self-written disruptions. A scaling sweep over resident count, shelter topology and fleet size was registered as exploratory and was not run, so the evidence rests on a single case study.

8.4. Measuring Survival

The primary outcome is the Recovery-Outcome Level, an ordinal scale from 0 to 4. It is built from the survival rate S, which is the co-primary outcome for safety. One frozen function of the event log computes both.

8.4.1. Feasibility Ceiling

Some disruptions leave fewer residents savable, so we normalise the outcome by the feasibility ceiling

$$N = \begin{cases} \min(P,\ \sum_s b_s) & \text{if } \sum_s b_s > 0, \\ P & \text{if no shelter declared any capacity,} \end{cases} \tag{8.1}$$

where P is the declared population and b_s the beds shelter s declared. The fallback to P stops a total of zero from making every run trivially complete. The ceiling comes only from what the scenario and the shelters declared. If it depended on what a run did, a system could lower its own denominator and score better by doing worse.

8.4.2. Survival Rate and Loss Decomposition

A resident is lost in one of three ways: *never delivered* to any shelter, set down *late* after the flood arrival, or *refused* at a shelter that was closed or full. An arrival counts only if it was set down by the flood arrival of its zone. Each shelter then admits the in-time arrivals in time order up to its declared beds, and only the residents above that limit are invalidated. So a run that saved everyone with one flaw cannot rank below one that cleanly saved a third of the home. The residents who pass both filters are the validly sheltered residents V, and

$$S = \frac{V}{N}, \qquad N = \underbrace{(N - H)}_{\text{never delivered}} + \underbrace{M}_{\text{late}} + \underbrace{O}_{\text{refused}} + \underbrace{V}_{\text{safe}}, \tag{8.2}$$

where H is the number reported sheltered, M the number set down late, and O the number refused for want of a bed. S alone cannot tell a slow coordinator from one that drove residents somewhere lethal, so we always publish the decomposition beside it.

The arrivals come from different places in the two systems. In the agentic system, an arrival is a bus driver's drop-off report. It carries the headcount that the driver, a language model, writes into it. The scripted shelter agent answers with an intake message that repeats this headcount, up to its free beds. The scorer counts the pair once, keyed by shelter and mission. Nothing checks the headcount against the residents who boarded. In the baseline, the execution model writes the drop-off from the IC's committed order when the bus reaches the shelter (Section 5.4). The scorer is deterministic in both systems, but on the agentic side its input is partly what the agents report. Section 9.8 and Table 10.3 take up what this means for the results.

8.4.3. Recovery-Outcome Level

Each run is placed on a five-level ladder with one axis, valid completeness $C = S$. Bus-kilometres K only split the top two levels:

$$L(C,K) = \begin{cases} 0 & C < 0.10 \quad \text{collapse} \\ 1 & 0.10 \le C < 0.50 \quad \text{major loss} \\ 2 & 0.50 \le C < 1.00 \quad \text{partial} \\ 3 & C = 1.00 \ \wedge \ K > 1.25\,K^{*} \quad \text{complete, degraded} \\ 4 & C = 1.00 \ \wedge \ K \le 1.25\,K^{*} \quad \text{complete and efficient} \end{cases} \tag{8.3}$$

We wrote down a reason for each cut before any result existed. The cut at 0.50 is half the home, the line between a mass-casualty event and an incomplete but largely successful operation. The cut at 0.10 is one part-load of a single 50-seat bus, so a run below it never started an evacuation. The 25 % band at the top leaves room for a legitimate detour or second pickup.

K^{*} is the median bus-kilometres of the same system and model's undisrupted `closed_loop` runs that delivered everyone, and needs at least three. Without K^{*}, no run may rise above level 3. The comparison stays within one system and asks whether it drove further than it needed to. Level 4 is the one place where the ladder mixes safety with efficiency, so the protocol defines it as passing both Gate 1 and Gate 2 (Section 8.5).

A pass/fail outcome would throw away the difference between losing one resident and losing fifty. Stroke researchers argue for keeping the ordinal modified Rankin Scale for the same reason [Gan+18], although most trials still cut it in two [NBG16]. We borrow only the shape of the scale, with no claim of clinical comparability.

8.4.4. Deterministic Scorer

No human scores anything. One frozen function of the event log (Section 6.4) computes every outcome for both systems. We do not use a language model as a judge, as is common [Zhe+23] and as some agent benchmarks do [Xu+25], because a model judging models would bring back the nondeterminism we are trying to measure. A missing value is never a zero. Every

interpretive constant lives in one data-only threshold file, whose hash is written into every run's manifest.

8.5. The Three Gates

A combined score lets a system trade safety for speed. The protocol therefore reads three gates in a fixed order, and a gate is read only if every gate above it passed (Figure 8.1). The reference protocol has a fourth gate, equity, which we register as undefined (Section 8.6.2).

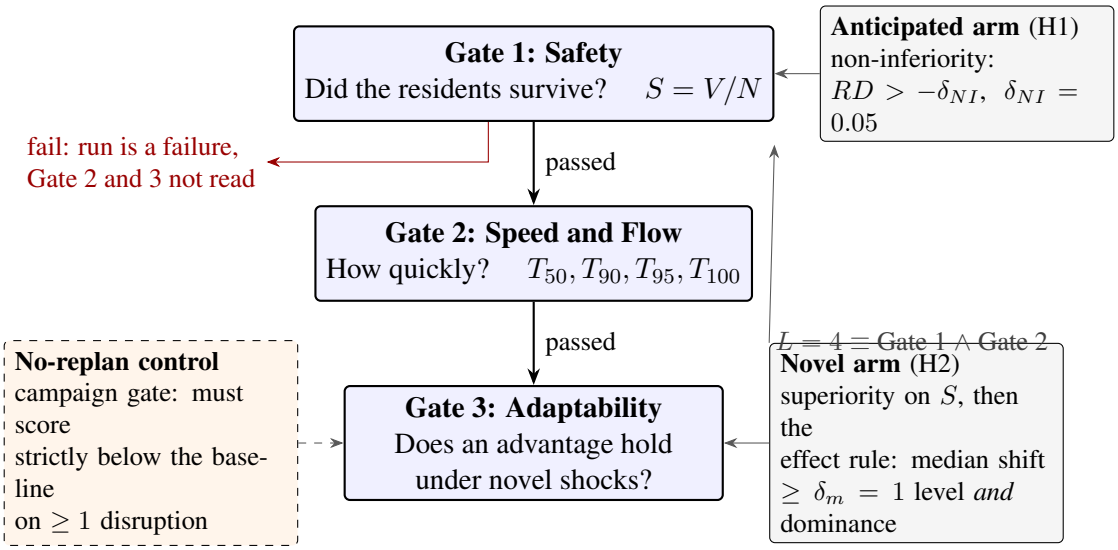


Figure 8.1.: The three gates of the evaluation protocol. A gate is read only if every gate above it passed. Hypothesis H1 is read at Gate 1 on the anticipated arm. Hypothesis H2 is read on the novel arm, first as superiority on survival and then as a minimum effect on the level scale. The no-replan control is a negative control on the whole campaign, not a third system under test.

8.5.1. Gate 1: Safety

Gate 1 asks whether the residents survived. Its effect measure is the paired risk difference per scenario, treatment minus control,

$$\widehat{RD} = \frac{1}{n} \sum_{i=1}^{n} \left(S_{A,i} - S_{B,i} \right), \tag{8.4}$$

over the n matched pairs, with a bootstrap interval and a Wilcoxon check (Section 8.7). On the anticipated arm the gate passes when $\widehat{RD} > -\delta_{NI}$. On the novel arm it passes when $\widehat{RD} > 0$ and the paired interval excludes zero. Without a registered margin the verdict is `unregistered`, which counts as a failure. A condition that fails Gate 1 is reported as a failure, and its Gate 2 and Gate 3 figures are not published as achievements.

We take the non-inferiority margin from clinical trials [Pia+12]. $\delta_{NI} = 0.05$ is five residents out of 100, half the 0.10 part-load that is the smallest unit the ladder knows. We chose a strict

margin because a wide one makes matching easy to show and worthless to claim. The H2 margin δ_m is one whole level and is read on levels, never on RD.

8.5.2. Gate 2: Speed and Flow

Gate 2 asks how quickly the residents reached safety. Its main measures are clearance times at $q \in \{0.50, 0.90, 0.95, 1.00\}$. With the admitted arrivals in time order and $F(j)$ their running count,

$$T_q = \min \{\, t_j \;:\; F(j) \geq \lceil q\, N \rceil \,\}. \tag{8.5}$$

The quantile is a share of the ceiling N, and not of the residents a run happened to save. If survival is below q, then T_q is censored and never imputed. Otherwise a system that saved the easy half and stopped would post an excellent T_{90}. A cell's clearance times are taken over its runs that saved everyone, and they are read only in a cell that passes Gate 1. Gate 2 is descriptive. We registered one expectation for it, that the baseline would win on latency and cost.

8.5.3. Gate 3: Adaptability

Gate 3 asks whether an advantage survives disruptions nobody planned for. The baseline must pass 100 % of the anticipated set, which all 20 campaign baseline runs did with $S = 1.0$, on the road closure against its reduced ceiling of 60 (Section 9.2). On the novel set, the per-scenario level distributions are compared by first-order stochastic dominance, which is part of the H2 rule (Section 8.7). A degradation curve over `SIM_TIME_SCALE` was to show where an advantage vanishes. Only one scale, 8.4, was executed, so H4 was not tested (Section 9.8).

8.5.4. The No-Replan Control

A third condition runs beside the two under test: the doctrine baseline with re-planning switched off. Its first committed plan is its only plan, and the rest of the doctrine is unchanged. This one ablation separates re-planning from reasoning about what to re-plan. It is also the instrument's negative control [LTTC10], under a rule registered as a gate on the whole campaign:

> Before any result is claimed, the no-replan control must score strictly below the doctrine baseline on at least one disruption.

We read the gate on the shelter-closure fixture, where a re-plan changes who is delivered. It passed on 2026-09-14, with the doctrine at level 3 and $S = 1.0$ and the control at level 0 and $S = 0.0$. The protocol did not read it on the scored scenarios, where the control "cannot differ from the doctrine in outcome, by construction" (Section 10.2).

8.6. Secondary Measures

8.6.1. Held-Out Decision Rubric

Four of the five novel scenarios state that their main effect is "judged mainly on the decision rubric". The simulator models no patient (S04), no staircase or police officer (S06), no payload cap (S07) and no partly damaged shelter (S08), so a coordinator that ignores the report can still reach a full level. S05 contains an explicit checklist. We settled the rubric on 2026-09-11, before any novel-arm data, and Amendment 6 registered it as the novel arm's secondary outcome.

The rubric is descriptive and never merged with L. Each criterion reads only the structure of the log: who was addressed and when, what the orders booked, and what was set down where. It never reads message text, because keyword rules would be brittle across models. We report yes, no and unanswerable counts per criterion, and an unanswerable criterion never counts as a failure. Table 8.3 condenses the 16 criteria.

Table 8.3.: The held-out decision rubric, condensed. Each clause from the supplier's scenario text becomes a mechanical rule on the log.

Scenario	Criteria
S04	A medical team is requested to meet bus-1 at its destination shelter. Bus-1's leg is neither withdrawn nor diverted. ZKD is informed.
S05	The report is verified with police or ZKD before it is retracted. The standing runs land as ordered. The reserve bus is committed as a hedge by 10:57. Nothing new is still moving after the retraction.
S06	The nursing home is told by 10:58. The conflict is escalated to ZKD. Seats on the standing legs still cover the whole population.
S07	No bus-1 leg sets down more than 25 after the report. Bus-2 is committed. Seats still cover the population.
S08	No order books SC1 above 40. The excess goes to SC2. The valid count, with SC1 capped at 40, reaches the population.

Before any held-out run we recorded what the baseline would do. The doctrine answers every held-out report with the same hold, status messages to ZKD, the transport operator and the home, and no new order. It therefore passes the informing criteria by construction and fails those that need a changed plan. The rubric shows where the conditions differ, but gives H2 no effect size.

8.6.2. Other Metrics

Each run also records measures that cannot move the level: bus-kilometres K, resource efficiency $\eta = V/K$, wall latency and tokens, capacity overloads, constraint violations, and the time of the last counted drop-off. Bus-km is composed differently in the two systems, so the code refuses to compare it across systems. Leg and supersede counts are diagnostics only, since a supersede count used as a score would reward churn. Table D.3 gives bus-km, η and the rejected messages per cell. Equity is undefined. Residents are counted as integers with no at-

tribute to split on, so the ladder cannot tell a run that sheltered the 60 most mobile residents from one that sheltered the 60 least mobile (Section 10.5).

8.6.3. Response-Time Cost

Model calls take time. We report the wall-clock latency λ of the model calls per model, and the operational delay in simulated time $\Delta_{\text{op}} = \lambda \cdot \sigma$, where σ is the time scale. The delay is given as a share of the disruption's decision budget, $\phi = \Delta_{\text{op}}/(D_z - t_{\text{first event}})$, where D_z is the flood arrival in the zone (12:00), because the same delay can be harmless under a slow disruption and fatal under a tight one. λ is summed over all ten agents, which think in parallel, so Δ_{op} is an upper bound. The primary outcome leaves latency and cost out on purpose. In disaster logistics, operational cost alone misses the cost of human suffering [Hol+13].

8.6.4. Exploratory Process Measures

Section 9.4 and Chapter 10 use process measures that Protocol v2 added after the data had been seen (Amendment 16). They are exploratory, and none of them enters a gate.

Loss attribution. The deficit $1 - S$ is split into the three channels of Equation 8.2: residents never delivered ($N - H$), set down late (M) and refused for want of a bed (O). The never-delivered count is floored at zero, so a run that reports more arrivals than the ceiling cannot push it below zero (Section 9.8).

Handshake closure. An addressed request is a message to one named agent whose type expects a reply, for example a capacity query to a shelter or a mission order to a driver. Closure is the share of addressed requests that have a matching reply from that agent. The pairs of request and reply types were set from the 49 message types found in the log. Reports that expect no answer, such as a position update, are left out. Each agent stamps rows with its own clock, so the log cannot put two agents' rows in order. Requests and replies are therefore matched as pairs regardless of order, and no reply time is measured. Closure asks whether an answer came back, not whether it was right. The baseline has a single coordinator with no peers and sends no addressed requests, so it has no closure figure.

Plan revisions. A plan revision is a message that sets or changes the bus plan. We count two types in the event log: BIS's `dispatch_buses` request to Hochbahn, and Hochbahn's `transport_plan`, which it sends to BIS each time its run schedule changes. The first plan counts, and so does a small change such as a firmer arrival time. A mission needs at least one message of each type, so two is the floor, and every run reached it. Beside this count we record each run's aborts, and classify the stated reason of each abort as a response to a failure, a re-plan, or other.

How the diagnostic was chosen. We chose which process measure to report after seeing the data. Besides plan revisions we examined the number of aborts, the number of missions

dispatched, the three loss channels and the candidate causes in Table 11.1. Among the measures that are not part of the definition of S, plan revisions had the strongest association with S. Their correlation describes these runs only and, as the largest of several, overstates the association.

8.7. Statistical Analysis

Stochastic dominance. The levels are ranked but not evenly spaced, so a mean level has no meaning. Where a continuous quantity is needed we use S, which is why RD can be a mean. Level distributions are compared by first-order stochastic dominance, which needs no arithmetic on an ordinal scale [Yal13]. With $\hat{p}_{\mathcal{A}}(L)$ the share of runs in sample $\mathcal{A}$ above level L,

$$\mathcal{A} \succ \mathcal{B} \iff \big(\exists L : \hat{p}_{\mathcal{A}}(L) > \hat{p}_{\mathcal{B}}(L)\big) \wedge \big(\nexists L : \hat{p}_{\mathcal{B}}(L) > \hat{p}_{\mathcal{A}}(L)\big). \tag{8.6}$$

If the distributions cross, neither dominates. With $k = 10$ and five levels some cells are nearly empty, so we print the raw level counts of every cell (Table D.1).

Intervals and the paired check. Every risk difference carries a 95 % percentile bootstrap interval [Efr79] from 10 000 resamples of the paired differences at the fixed seed 20260904. A Wilcoxon signed-rank test [Wil45] on the same differences is an assumption-light check. Zero differences are dropped from the test but counted, because pairs that mostly tie say something about the systems.

The H1 rule. H1 passes on a scenario when $\widehat{RD} > -\delta_{NI}$, a rule on the point estimate with no significance level. The pre-registration first named a one-sided test on the interval's lower bound. On 2026-09-12, with pilot data seen, we registered the rule the code had always used and left the margin unchanged. A system at the margin therefore passes about half the time at any k. We report a pass as "the point estimate lies inside the margin" and never as "non-inferiority demonstrated".

The H2 rule. The pre-registration names one primary test for H2 and two conditions beside it. The primary test is an exact Clopper–Pearson interval [CP34] on $P(\text{level}_{\text{agentic}} \geq \text{level}_{\text{baseline}})$ over the k agentic runs, aggregated across scenarios by a sign test with a Holm–Bonferroni correction [Hol79]. We implemented it only after the novel-arm data had been seen (Amendment 15), so it is exploratory (Section 10.4). The H2 verdict therefore rests on the two conditions, which we call its co-primaries. The first is Gate 1 on survival. The second is a minimum effect on the level scale, because passing Gate 1 does not yet support a claim to have won. It needs both

$$\operatorname{median}\big\{L_{A,i} - L_{B,i}\big\} \geq \delta_m = 1 \quad \wedge \quad \mathcal{L}_A \succ \mathcal{L}_B. \tag{8.7}$$

Dominance is included because a median can hide a bad tail. Four runs at level 4 and two at level 0 give a median shift of two levels from a system that abandoned the home in a third of its runs, and the rule rejects this. H2 passes on a scenario only if both co-primaries pass.

Multiplicity. H1 and H2 are claims on two disjoint scenario sets, each with one primary outcome, so no correction is applied between them. Within an arm the correction is Holm–Bonferroni. Runs are never pooled across scenarios, which differ in ceiling, fleet and deadline. Secondary outcomes are descriptive and not corrected.

Choosing k. We sized k on a pilot of 22 paired cells on the anticipated set. No cell reached the target H1 pass rate of 0.80 at any k from 3 to 200, because every pilot mean already lay below $-\delta_{NI}$. The margin could not be widened, so we set $k = 10$ on budget and precision grounds and recorded that H1 was not expected to pass (Amendment 1).

Exclusions. A run can be excluded only for a simulator or harness crash, a harness time-out, or right-censoring. A run is right-censored when it stopped before D_z plus 900 simulated seconds, about one shuttle turnaround, so that a late drop-off could still have been missed. An unfavourable outcome, an outlying latency or a hard novel scenario is never a reason. The event log is append-only by database trigger, so removing a run would leave a trace.

Cut-point sensitivity. A threshold-based outcome invites the charge that the thresholds were chosen to produce the result. Every score therefore also carries its level at the level-1/2 cuts 0.40, 0.50 and 0.60, and we report the headline together with this table (Section 9.3.3 and Table D.1). It is a small specification curve [SSN20]. The 0.10 and 1.00 cuts are anchored, so we sweep only the half-home cut.

8.8. Pre-Registration and Amendments

Choices made after seeing data bias a result even without intent, and hypotheses written after the results misrepresent what was tested [GL14]. Pre-registration guards against both [Nos+18]. Once tagged, our pre-registration was never edited. Changes were appended as dated amendments stating what changed, why, and whether evaluation data had been seen. Any analysis touched by a post-data amendment is exploratory. Table 8.4 lists the amendments, out of sixteen, that shape how the results are read.

The margins δ_{NI} and δ_m and the equity decision were registered on 2026-09-04, against zero evaluation runs, and written to the threshold file in the same commit. The pilot ran before the pre-registration tag, as the pre-registration discloses.

Table 8.4.: The amendments that change how a result is read. The full set of sixteen is in the pre-registration's amendment log.

#	Date	Data seen	Change
1	09-12	Pilot	$k = 10$; campaign models glm-5.2, deepseek-v4-pro, minimax-m3; replicates 1000–1009; H1 not expected to pass.
5	09-14	Anticipated arm	Final model set chosen over the registered rule.
6	09-14	Anticipated arm only	Novel arm fixed before any novel score: five scenarios, freeze and reveal dates, the negative-control verdict, the rubric as secondary, the new thresholds hash.
10–12	09-16 to 09-18	Novel arm, partly	The third model slot moves three times, ending at qwen3.5.
13	09-20	Whole novel arm	qwen3.5 anticipated `closed_loop` cells added to complete its level ladder.
15	09-21	Whole evaluation	The registered H2 primary implemented, hence exploratory. No confirmatory number moves.
16	09-22	Whole evaluation	Treatment validity and Protocol v2, exploratory by construction (Chapter 10).

8.9. Availability of Code and Data

The code of both systems, the harness and the scorer lives in the private GitHub repository `absolutehh/RescuemateEvacuationWithAgents` of Absolute Software GmbH. Examiners can obtain read access through the company. The baseline freeze is the tag `eval-day-2026-09-10` on commit `67a14f8`. The pre-registration is `prereg-v1`, and `prereg-amendment-1` marks the same commit, `7a4d7cc`. Amendments 2 to 16 are appended untagged, and a diff of the file against `prereg-v1` holds only added lines. The threshold file is `evaluation/thresholds.py`. Its SHA-256 begins with `d08c1e35` in all 310 scored runs, computed with Windows line endings.

The results folder sits outside the company repository. We publish it in the public repository `anikdewanje/master-thesis-submission` at tag `thesis-submission`, with a dump of the `event_log` table that holds the 310 event logs, keyed by run. Each run record holds its log's row count and an SHA-256 checksum of its content. A script in the repository recomputes both from the dump and confirms all 310 runs.

9. Results

This chapter reports what the evaluation measured, under the design frozen in Chapter 8.

The workflow baseline beats the database-centric agentic system on resident survival on all five novel disruptions. On the undisrupted `closed_loop` one model ties it. On the road closure the two systems ran different versions of the scenario, so that comparison is not like-for-like (Section 9.2). Both confirmatory hypotheses, H1 and H2, fail. Gate 1 fails in 26 of its 27 cells. Three of the failing cells are on the road closure, and without them Gate 1 fails in 23 of 24.

9.1. Execution Summary

The evaluation ran 310 scored runs, and all 310 were eligible. Table 9.1 lists what was executed.

Table 9.1.: What was run.

Case	Augustinum nursing home, 100 residents, Hamburg storm surge. Feasibility ceiling 100 residents, 60 in the baseline's road-closure file.
Systems	`agentic` (10 agents across 8 organizations, database-centric) against `baseline` (workflow).
Anticipated arm	`closed_loop` and `road_closure`. Both were visible during development.
Novel arm	Five disruptions supplied by the domain expert and held out: `medical_emergency_in_flight`, `unverified_persons_report`, `conflicting_evacuation_orders`, `bus1_suspension_capacity_reduction`, `sc1_partial_capacity_loss`.
Replicates	$k = 10$ per cell, replicates 1000 to 1009, agentic runs under common random numbers, paired with the deterministic baseline.
Scale	`SIM_TIME_SCALE` = 8.4, one value only.
Scored runs	310. Zero censored, zero unscored, zero invalid model bindings.
Primary outcome	Recovery-Outcome Level 0 to 4, and survival S = valid residents divided by the feasibility ceiling.

Three models completed the full novel arm at 50 runs each. These are glm-5.2, deepseek-v4-pro and qwen3.5. Further models hold partial series: kimi-k3 with 11 runs, gpt-oss with 4, kimi-k2.7-code with 3, and from the earlier campaign minimax-m3 and nemotron-3-ultra on the anticipated arm only. We report partial series on their own and never pool them with the full arms (Section 9.7).

An audit of all 218 novel-arm runs found coverage of 50 of 50 on each full arm, no duplicate run, cell or log checksum, all 218 manifests present, zero model-binding errors, zero undeliv-

ered world commands and a stable scenario hash throughout. The hash of the threshold file is identical across all 310 runs, so every run was scored against the same frozen thresholds.

Two properties of the run set matter for later tables. First, 28 of the 168 agentic novel runs (17%) stopped without quiescence, so a final message may be missing from their logs. The outcome was decided in all of them, and all are scorable. Second, the deepseek `unverified_persons_report` replicates 1001 and 1002 ran with uncommitted routing changes that were committed later. The protocol gives no reason to exclude them, so we keep both.

9.2. Anticipated Disruptions

The anticipated arm is the precondition for the main test. It asks whether the agentic system is at least as good as the baseline on the disruptions the baseline was written for. This is hypothesis H1, tested as non-inferiority against a margin $\delta_{NI} = 0.05$.

Table 9.2.: Anticipated arm, mean survival S.

System and model	`closed_loop`	`road_closure`
baseline	1.000	1.000*
agentic / glm-5.2	**1.000** (pass)	0.840
agentic / qwen3.5	0.950†	not run
agentic / deepseek-v4-pro	0.790	0.700
agentic / minimax-m3	0.340	0.150
agentic / nemotron-3-ultra	0.000 ($n = 2$)	not run

H1 fails. The point estimate lies inside the margin in 1 of the 8 anticipated cells and at or below it in the other 7. Six of the eight are confirmatory cells under Amendment 1, and H1 fails in five of those six, including every confirmatory cell on the disrupted scenario. The other two cells, nemotron-3-ultra with two runs and qwen3.5 under Amendment 13, are exploratory. The one pass is glm-5.2 on `closed_loop`, with a risk difference of 0.000. This is the undisrupted scenario, and it is the only cell in the whole evaluation where the agentic system is shown to be no worse than the baseline. H1 also fails without the road closure. On `closed_loop` it fails in two of the three confirmatory cells, deepseek-v4-pro and minimax-m3.

* On `road_closure` the baseline's feasibility ceiling is 60 in all 10 runs, because its file declares only 60 residents (Equation 8.1). It sheltered all 60, so its S of 1.000 is against this smaller denominator, and its completeness against the nominal 100 residents is **0.60**. We quote the nominal figure whenever this cell is compared with anything else. At 0.60, the baseline is below glm-5.2 (0.84) and deepseek-v4-pro (0.70). The cell differs in two ways. First, the two systems ran different versions of the closure. The baseline has no routing command, so in its version a 60-bed shelter becomes unreachable, and the file leaves out the 40 residents who would have no bed. In the agentic version the closure only costs a detour, and both shelters stay reachable for all 100 (Section 3.4). Second, the baseline's file declares no world, so its drop-offs come from the script, and its score does not depend on its orders (Section 10.2). We keep the recorded Gate 1 verdicts for the three `road_closure` cells, but do not read them as like-for-like.

† We added the qwen3.5 `closed_loop` cell under Amendment 13 to give that model a level reference (Section 9.7). Its risk difference is -0.05 [-0.15, 0.00], and Gate 1 reads `fail`. qwen3.5 has no `road_closure` cells, because it was never a campaign model.

9.3. Novel Disruptions

The novel arm is the primary test. The domain expert supplied five disruptions, which were held out from development and revealed on 2026-09-10, after the baseline freeze. The baseline has no written branch for any of them. Hypothesis H2 predicted that the agentic system would win here.

9.3.1. Survival and Gate 1

Table 9.3.: Novel arm, mean survival S over all 50 runs per model against the 50 paired baseline runs.

System and model	n	mean S	capped S	median level	level-4 runs
baseline	50	**1.000**	1.000	4	50
agentic / qwen3.5	50	0.900	**0.894**	4	30
agentic / glm-5.2	50	0.840	0.840	4	31
agentic / deepseek-v4-pro	50	0.727	0.727	2	15

The baseline scored 1.000 on all 50 novel runs (Table 9.3). The best agentic configuration, qwen3.5, reached 0.894. We read the capped column. Three qwen3.5 runs scored above the feasibility ceiling and are capped at 1.0 (Section 9.8). The cap moves qwen3.5's mean from 0.900 to 0.894 and changes no conclusion in this chapter. The risk differences below are computed on S without the cap, as the scorer records it.

Gate 1 reads a different hypothesis on each arm. On the anticipated arm it is a non-inferiority test and carries H1, with 1 pass in 8 cells. On the novel arm it is a superiority test and carries H2, with no pass in 19 cells. These are the 15 cells of the three full models and four cells from the partial series of kimi-k3, gpt-oss and kimi-k2.7-code. The 10 cells of glm-5.2 and deepseek-v4-pro are confirmatory, and none of them passes. **Gate 1 therefore fails in 26 of 27 cells**, and all 19 novel-arm failures belong to H2. Three of the 26 are the `road_closure` cells, which are not like-for-like (Section 9.2). Without them Gate 1 fails in 23 of 24 cells.

The risk difference is $RD = S_{\text{agentic}} - S_{\text{baseline}}$, computed paired, so a positive value favours the agentic system. Table 9.4 gives the 15 novel cells of the three full models with bootstrap confidence intervals [Efr79].

Superiority needs the paired interval to exclude zero on the positive side. No cell comes close, and every point estimate is negative. Eight intervals stay below zero, and six reach zero and stop there. One reaches past it, qwen3.5 on `medical_emergency_in_flight`, with an upper bound of 0.02. Table D.2 in Appendix D gives all 27 Gate 1 cells with the Wilcoxon check, and no Wilcoxon result favours the agentic system. Table D.1 gives the level counts behind every median.

Table 9.4.: Novel-arm risk differences with bootstrap confidence intervals. Positive favours the agentic system. The baseline scored 1.000 in every cell, so a model's mean uncapped S on a disruption equals $1 + RD$.

Disruption	qwen3.5	glm-5.2	deepseek
`conflicting_evacuation_orders`	-0.02 [-0.05, 0.00]	-0.15 [-0.30, 0.00]	-0.35 [-0.55, -0.17]
`medical_emergency_in_flight`	-0.08 [-0.21, 0.02]	-0.15 [-0.30, 0.00]	-0.20 [-0.42, -0.04]
`bus1_suspension_capacity_reduction`	-0.08 [-0.18, -0.01]	-0.15 [-0.30, 0.00]	-0.245 [-0.45, -0.08]
`unverified_persons_report`	-0.10 [-0.23, 0.00]	-0.16 [-0.31, -0.03]	-0.31 [-0.44, -0.17]
`sc1_partial_capacity_loss`	-0.22 [-0.44, -0.03]	-0.19 [-0.42, 0.00]	-0.26 [-0.52, -0.03]

9.3.2. Minimum Effect and the Registered Primary

Gate 1 is the first of H2's two co-primaries (Section 8.7). The second is a minimum effect. It required a median shift of one whole recovery level, $\delta_m = +1.0$, together with first-order stochastic dominance of the agentic system over the baseline [Yal13]. No cell has a positive median shift. Every novel row reads `below_delta_m` with dominance assigned to the baseline, so neither condition holds anywhere. At best the agentic side matches the baseline's median level on four of the five disruptions, and it still loses residents there. A few lost residents lower S but not the median level, so Gate 1 can fail in a cell where the median level ties. Both co-primaries fail in every novel cell, so H2 fails.

The pre-registration's primary test for H2, the Clopper-Pearson interval on $P(\text{level}_{\text{agentic}} \geq \text{level}_{\text{baseline}})$, was implemented after the result was known (Amendment 15). It is exploratory, and the verdict above does not rest on it. It agrees with that verdict. No Clopper-Pearson interval has a lower bound above 0.5, so no cell puts the agentic system above chance. One cell falls below chance before correction, deepseek on `conflicting_evacuation_orders` at 1 of 10 with $p = 0.021$. It does not survive the Holm correction, where $p = 0.107$. At $k = 10$ every interval is about 0.6 wide (Section 10.4).

9.3.3. Cut-Point Sensitivity

The protocol makes a cut-point sensitivity analysis [SSN20] a precondition on the headline. We re-read each cell's median level with the level 1 to 2 cut moved to 0.40, 0.50 and 0.60. Every cell in the three full novel arms reads the same median at all three cuts, except deepseek on `unverified_persons_report`, which moves from 2.0 to 1.5 at the 0.60 cut. The other unstable cells are minimax-m3 on the anticipated arm and two partial series, kimi-k2.7-code at $n = 3$ and kimi-k3 at $n = 1$. No agentic-versus-baseline ordering moves anywhere in the cut range, so the analysis supports the headline. Table D.1 gives the median of every cell at each cut.

9.4. What the Losses Were Made Of

This section breaks the agentic loss down by mechanism. The baseline lost nobody this way. It recorded no late arrival and no shelter overload in any run.

9.4.1. Residents Never Delivered

Table 9.5.: Residents never delivered to any shelter, per 100. Lower is better.

Disruption	baseline	qwen3.5	glm-5.2	deepseek
`medical_emergency_in_flight`	0.0	**5.0**	10.0	13.0
`conflicting_evacuation_orders`	0.0	**2.0**	15.0	19.0
`bus1_suspension_capacity_reduction`	0.0	**2.0**	15.0	22.5
`unverified_persons_report`	0.0	**4.0**	10.0	21.0
`sc1_partial_capacity_loss`	0.0	**2.5**	14.0	16.0

Table 9.5 counts the term $N - H$ of Equation 8.2: the ceiling minus the residents reported sheltered, floored at zero, so the over-delivery runs of Section 9.8 cannot push a cell below zero (Section 8.6.4). It counts residents who reached no shelter. It does not tell whether a bus was ever sent for them, and the dispatch data show that missions were sent (Section 11.2).

qwen3.5 leaves far fewer residents undelivered than the other two models, by a factor of two to eleven on every disruption. Its deficit comes mostly from residents who arrived late or were invalidated by shelter overload. Across the three full models on the novel arm, 985 residents arrived late, spread over 26 runs, and 215 were invalidated because a shelter was filled past its declared beds.

9.4.2. Handshake Closure

Handshake closure is the share of addressed requests that drew a matching reply (Section 8.6.4). Over 1 932 addressed requests in the agentic novel arm, it is **0.938**. The three highest-volume handshakes close at 0.947, 0.970 and 0.997. The one weak handshake is `abort_request` at 0.548, on a small denominator of 31 requests.

For deepseek-v4-pro and glm-5.2, the dominant loss is residents who were never delivered. Per 100 residents, they account for 18.3 of deepseek-v4-pro's deficit of 27.3 and 12.8 of glm-5.2's 16.0. For qwen3.5 they account for only 3.1 of 10.0, and lateness (6.9) and overload (4.0) weigh more. qwen3.5's three parts add up to more than its deficit, because in its over-delivery runs (Section 9.8) late and refused arrivals are counted on top of a full ceiling. Across the 150 agentic novel runs from the three full models, never-delivered residents correlate with S at $r = -0.745$, against -0.579 for late arrivals and $+0.061$ for shelter overload. These are exploratory Protocol v2 instruments, written after the data were seen. Section 11.2 lists the other explanations we ruled out.

9.4.3. The Loss Is Quantised

Figure 9.1 gives the distribution of residents delivered across the same 150 runs.

Of the 150 runs, 102 delivered everyone. Nineteen delivered 50, one whole bus-load short, eighteen delivered 90, and two delivered nobody. No run lands between 65 and 90, or between 10 and 40. The spike at 50 matches one bus-load of 50 seats. The spike at 90, ten residents short,

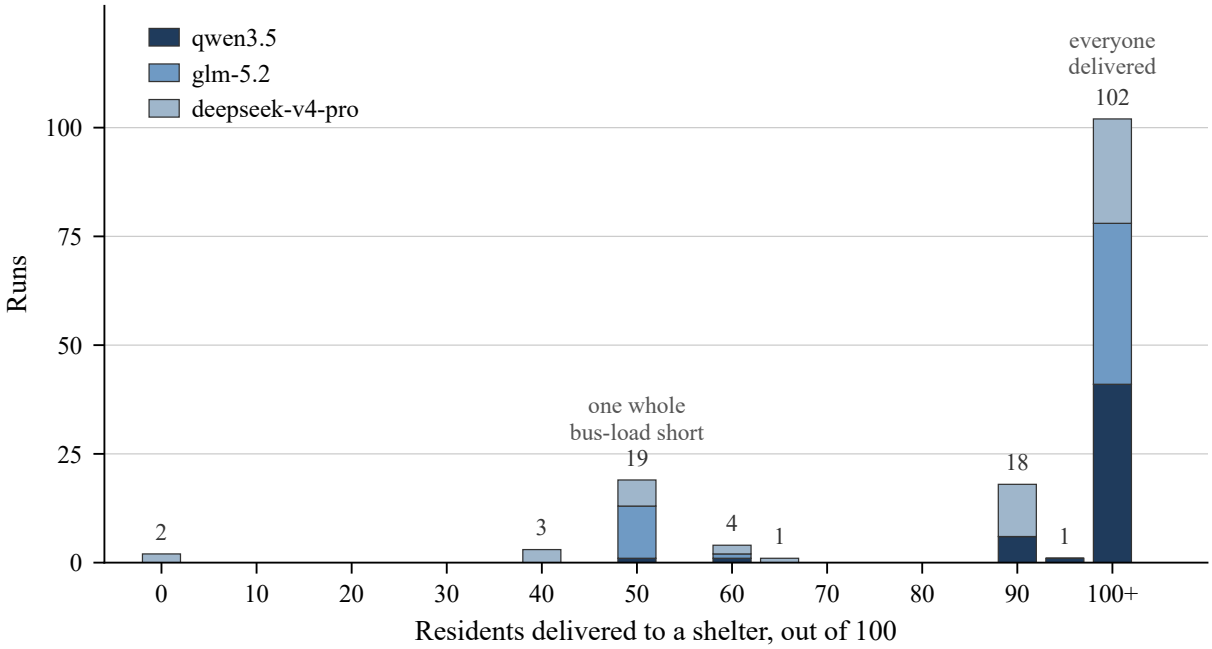


Figure 9.1.: Residents delivered to a shelter, over the 150 agentic novel runs from the three full models. The distribution clusters on a few discrete values instead of spreading smoothly. The six runs that report more arrivals than the ceiling admits (Section 9.8) are folded into the 100+ column, the same way the survival tables cap them.

matches no bus-load, and the data do not show which leg those ten residents were on. The spike at 50 is mostly glm-5.2, with 12 of the 19 runs, and the spike at 90 is mostly deepseek-v4-pro, with 12 of 18. qwen3.5 holds 41 of the 102 full deliveries.

9.4.4. Plan Revisions

Process diagnostics explain the outcome distribution but do not score it. The protocol demotes supersede count on purpose, because it rewards churn. We report the number of plan revisions, the messages that set or change the bus plan (Section 8.6.4). We chose this measure after seeing the data. Table 9.6 bands the 150 agentic novel runs by it, and Figure 9.2 shows the runs behind each band.

Table 9.6.: Plan revisions against delivery, 150 agentic novel runs. Exploratory.

Plan revisions	runs	mean S	mean aborts	full delivery
2 (the minimum)	61	0.923	0.08	**90%**
3	22	0.827	0.55	77%
4	15	0.853	0.67	67%
5	11	0.782	1.27	64%
6 or more	41	0.670	1.63	**32%**

The full-delivery rate falls monotonically across the bands, from 90% at the minimum revision

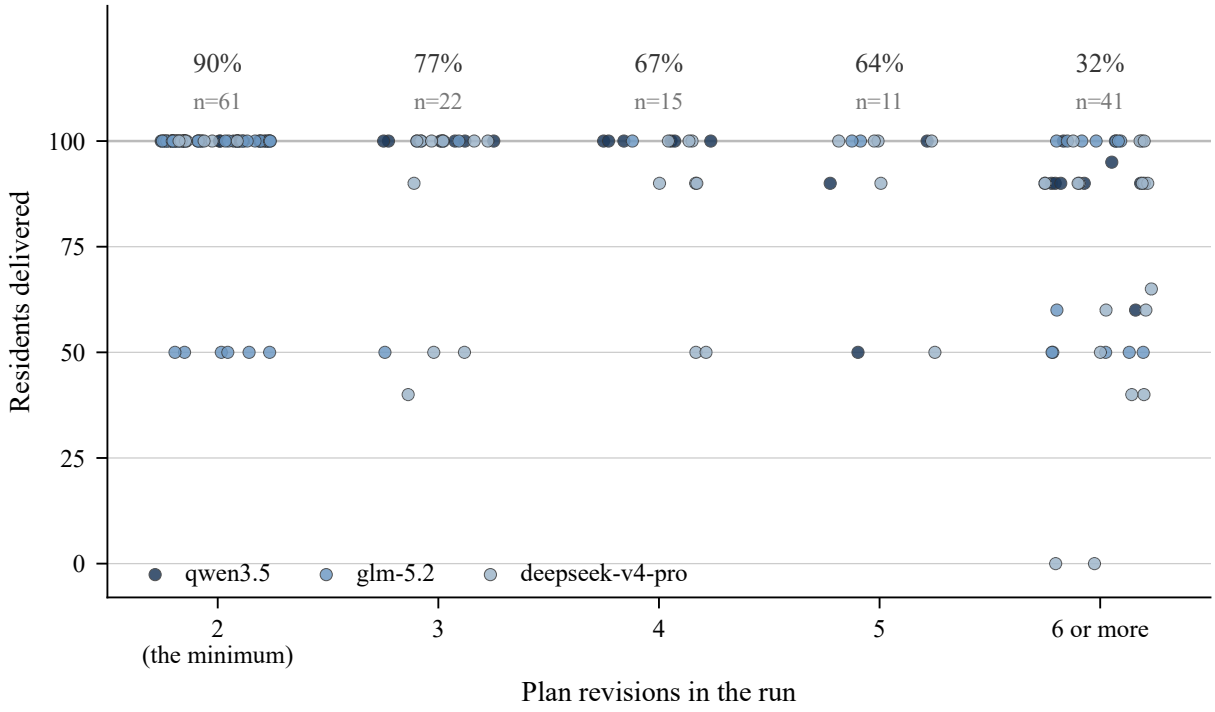


Figure 9.2.: Plan revisions against delivery, one point per agentic novel run, spread sideways within each band so that runs at the same count stay visible. The percentage above each band is the share of its runs that delivered all 100 residents, and `n` is how many runs the band holds. Table 9.6 gives the band means.

count to 32% at six or more. Plan revisions correlate with S at $r = -0.408$, the strongest of the process measures we examined outside the definition of S. A value picked as the largest of several overstates the association. Aborts correlate at $r = -0.306$, while the number of missions dispatched correlates at only $r = -0.070$. From six revisions up they spread over the whole range, including both runs that delivered nobody. These are correlations, and Section 11.2 discusses their direction.

9.5. Decision Quality

A held-out rubric measured decision quality on the novel arm. It is a secondary, structural measure. It is never tabulated beside a recovery level, and it does not enter any gate. Figure 9.3 shows all sixteen criteria across the five novel disruptions, and Appendix C gives the counts.

Read across all sixteen criteria, the rubric splits three ways. No rule for these groups was fixed before the data were seen. We formed them by inspection of the counts, mainly from qwen3.5 and glm-5.2, the two models with the higher survival. The line between parity and a small baseline lead is a judgement. Under the simple rule "within one judgement of the baseline for both qwen3.5 and glm-5.2", parity would hold on eight criteria and the baseline would lead on four, since `full_evacuation_kept` and the two criteria left out of the groups would count as parity.

Parity on five criteria. qwen3.5 and glm-5.2 reach the baseline's 10 out of 10

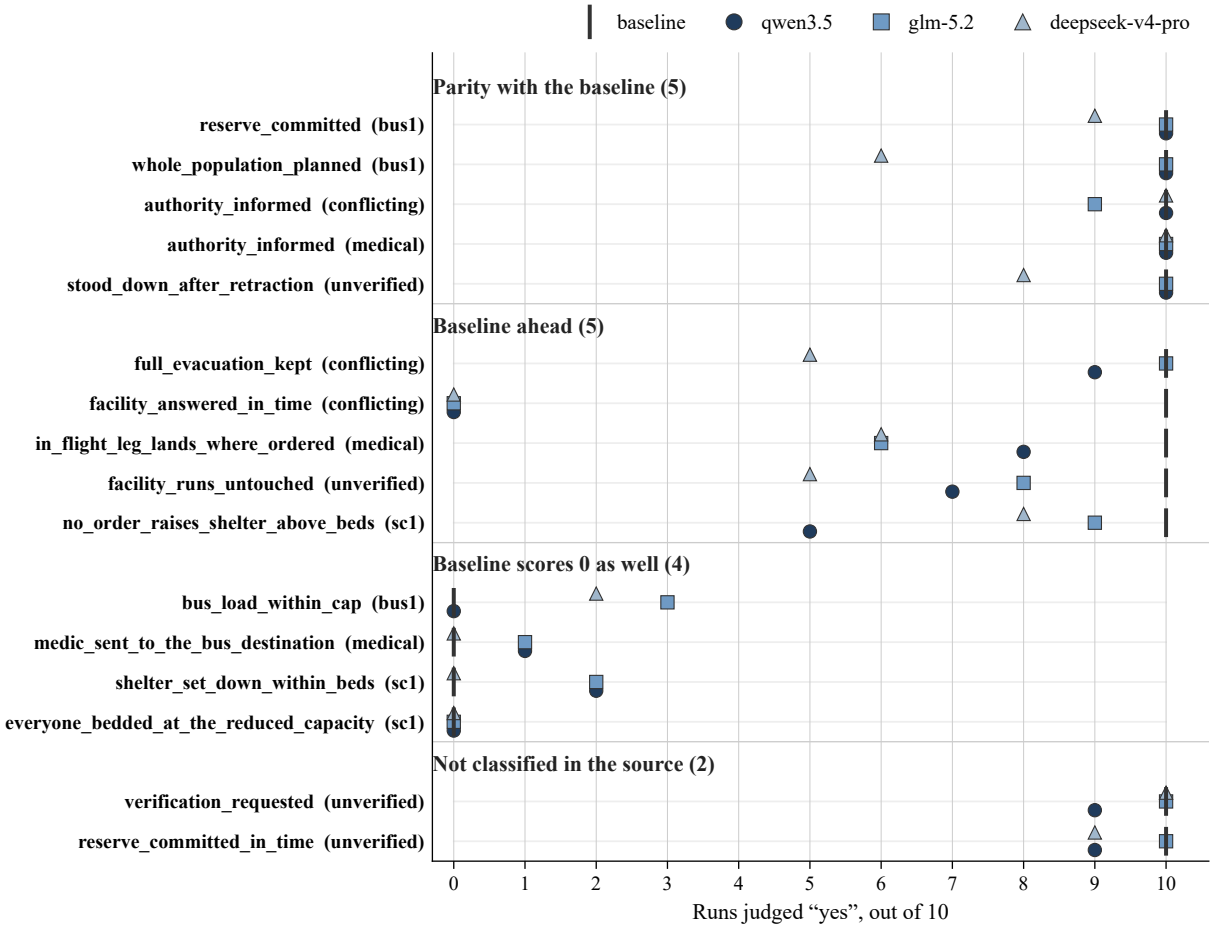


Figure 9.3.: The sixteen rubric criteria, grouped by how the two sides compare. The tick marks the baseline. Parity holds where the agentic markers sit on it, and the baseline's lead is the distance to the left of it. The bottom group of four is where the baseline itself scores zero. The last two, `verification_requested` and `reserve_committed_in_time`, are within one judgement of the baseline for every model and are left out of the groups. Table C.1 in Appendix C gives the counts.

on `authority_informed` for both disruptions, `reserve_committed`, `whole_population_planned` and `stood_down_after_retraction`. The one exception is glm-5.2 on `authority_informed` under conflicting orders, with 9 `yes` and one run that could not be judged. deepseek falls behind on three of these criteria, by one to four judgements.

The baseline leads on five criteria. The clearest is `facility_answered_in_time`, where the baseline scores 10 and no agentic model ever answered the facility in time. On the other four the baseline's 10 compares with qwen3.5, glm-5.2 and deepseek as follows. `no_order_raises_shelter_above_beds` scores 5, 9 and 8, `facility_runs_untouched` 7, 8 and 5, and `in_flight_leg_lands_where_ordered` 8, 6 and 6. On `full_evacuation_kept` glm-5.2 ties the baseline at 10, while qwen3.5 scores 9 and deepseek 5. We count it as a baseline lead because two of the three models fall short.

Four criteria score zero for the baseline. Three of them indict the scenario: `bus_load_within_cap`, `medic_sent_to_the_bus_destination` and `everyone_bedded_at_the_reduced_capacity`. The baseline fails them too, so they cannot count as agentic

failures. On three of the four the agentic side passes a few runs where the baseline passes none. `shelter_set_down_within_beds` scores 2, 2 and 0 for qwen3.5, glm-5.2 and deepseek, and `bus_load_within_cap` scores 0, 3 and 2.

On `medic_sent_to_the_bus_destination` the baseline never requested medical support, because its holding order (Section 5.3.2) does not reach a bus already on the road. qwen3.5 and glm-5.2 met the criterion in one run each, and deepseek in none.

Decision quality is split. Over the sixteen criteria, grouped by inspection, the agentic models match the baseline on five and fall short on five.

Rejected messages were rare. Over the 150 novel runs of the three full models, the runtime dropped 35 outbound messages: 18 for deepseek, 15 for qwen3.5 and 2 for glm-5.2 (Table D.3). Twenty-one had a message type the agent may not send, and fourteen were malformed. Every model stayed below one rejected message per run.

9.6. Response Time and Cost

The baseline runs no model, so its model latency is 0.0 seconds. Agentic wall-clock latency per run ranges from 763 seconds, for deepseek on `closed_loop`, to 3 810 seconds, for kimi-k3 on `medical_emergency_in_flight`. Bus kilometres cannot be compared between the two systems, because they compose the total differently. Table D.3 gives bus-km and the resource efficiency η per cell, to be read within one system.

Gate 2 is read only where Gate 1 passes, and one cell passes it: glm-5.2 on `closed_loop`. In its median run, half the residents were in a shelter at 10:57 and all of them at 11:17. The baseline reached these points at 10:24 and 11:37 in every run. The clearance times of every other cell are not read, because those cells lost residents.

Table 9.7 gives the coordination surface for the 150 agentic novel runs and their 50 paired control runs.

Table 9.7.: Coordination surface on the novel arm. Exploratory.

System and model	messages	decisions	msgs / resident	tokens	budget fraction
baseline	13.2	13.2	**0.13**	0	0.00
agentic / deepseek-v4-pro	91.9	152.6	1.26	725 457	1.80
agentic / qwen3.5	106.6	170.3	1.18	894 243	2.28
agentic / glm-5.2	111.8	174.9	1.33	829 906	2.70

The control coordinates a hundred-resident evacuation with 13 messages, all handled by its one Incident Commander. The agentic system sends seven to eight and a half times as many messages across its ten agents, which is nine to ten times as many per resident delivered. It also spends roughly 800 000 tokens and 1.8 to 2.7 times the wall-clock that the compressed clock allows, and 140 of the 150 runs finished over budget. This budget fraction is ϕ of Section 8.6.3.

9.7. Model Sweep and Capability Threshold

The gap differs strongly between models, as agent benchmarks also find [Xu+25]. The mean novel-arm risk difference is -0.27 for deepseek-v4-pro, -0.16 for glm-5.2 and -0.10 for qwen3.5. deepseek-v4-pro has the largest deficit on all five novel disruptions, and qwen3.5 the smallest on four of them. The exception is `sc1_partial_capacity_loss`, where glm-5.2 at -0.19 does slightly better than qwen3.5 at -0.22. The weaker models, minimax-m3 and nemotron-3-ultra, fail badly (Table 9.2). Neither is part of the headline result.

Hypothesis H3 cannot be resolved, because it presupposes H2. H3 predicted that the agentic advantage on novel disruptions would be model-robust, with a capability threshold below which that advantage disappears. With no advantage to be robust or fragile, the threshold is undefined. The ordering above is descriptive, and the design does not support extrapolating it past qwen3.5.

Comparability of the model series. A promotion to level 4 needs a resource-efficiency judgement against the model's own no-disruption reference, which takes three qualifying control runs. A model without `agentic_closed_loop` runs is therefore capped at level 3. qwen3.5 was at first novel-arm-only and capped by this rule, so Amendment 13 ran its ten `closed_loop` cells. After re-scoring it has 30 level-4 runs and a median level of 4, against glm-5.2's 31 and 4, so levels are comparable across the three full arms. Re-scoring changed no value of S or RD and no Gate 1 verdict, because these are computed per run without the reference. kimi-k3 (11 runs, $S = 0.773$), kimi-k2.7-code (3 runs, 0.667) and gpt-oss (4 runs, 0.375) remain capped. They are compared on S only and never ranked by level against the full arms.

The novel arm's third model slot was held in turn by kimi-k3, gpt-oss, kimi-k2.7-code and finally qwen3.5. Each swap was made after seeing data, so under the pre-registration the novel arm's model set, and any analysis that uses it, is exploratory. glm-5.2 and deepseek-v4-pro were fixed before the novel arm ran and are unaffected.

9.8. Integrity of the Reported Numbers

We report one measurement defect and one design limit. We do not correct the defect in the scorer, because a correction after the fact would rewrite a confirmatory result.

Six runs shelter more residents than the ceiling admits. Three qwen3.5 runs score an impossible $S = 1.1$. Three further runs, one from qwen3.5 and two from deepseek, have the same defect but stay masked, because their valid-resident count did not exceed the ceiling. The population caps the feasibility ceiling at 100, while valid residents are capped only by the beds of each shelter, 60 + 60 = 120 in total. The reported sheltered count is a sum over logged arrival reports and has no cap at all. The scorer counts no arrival twice, so the excess comes from the reports themselves. Each arrival carries the headcount its driver wrote (Section 8.4.2), and the scorer never compares it with the residents who boarded. We cap S at 1.0 at the reporting layer, and no table in this thesis shows a value above 1.0.

An over-report can only raise an agentic S. It cannot create the deficit, but it can hide part of it, so the agentic figures in this chapter are, if anything, too high. An under-report, a driver writing fewer residents than were set down, would lower an agentic S. The log has no independent count of arrivals, so we cannot rule this out.

One time scale, so H4 was not tested. Every result in this chapter uses a single `SIM_TIME_SCALE` of 8.4. Gate 3 therefore has one point per cell, and no crossover can be located. H4 asked at which time scale the agentic latency cost overtakes its coordination benefit, so H4 was not tested, and we make no crossover, trend or scale-dependence claim.

9.9. Summary of Findings

Table 9.8 gives the verdict on each hypothesis. Decision quality is split. The agentic models match the baseline on five of the sixteen rubric criteria and fall short on five.

Table 9.8.: Verdict on each registered hypothesis.

Hypothesis	Verdict	Section
H1, non-inferiority on the anticipated arm	**Fails.** Inside the margin in 1 of 8 cells, glm-5.2 on the undisrupted `closed_loop`. Without `road_closure` it still fails in 2 of the 3 confirmatory cells.	9.2
H2, superiority on the novel arm	**Fails as registered**, on both co-primaries. 0 of 19 Gate 1 cells pass, and no cell reaches the minimum effect. No evidence that the baseline adapted (Section 10.2).	9.3
H3, a model-robust advantage with a capability threshold	**Unresolved**, because it presupposes H2.	9.7
H4, a time-scale crossover	**Not tested.** Only one time scale ran.	9.8

10. Methods Critique

In this chapter we read the results back against the protocol that produced them. Applied to its own data, the protocol showed that its primary arm could not have answered the question it was built for. We report that as a result, and we use the protocol's own negative control to define a gate that would have caught it.

Everything in Sections 10.2 to 10.3.1 was designed after we had seen the evaluation data. It is registered as Protocol v2 in Amendment 16 and is exploratory, because choices made after seeing the data can bias a result even without intent [GL14]. It leaves every number of Chapter 9 as it was. No scenario in this thesis ever required adaptation, so we answer RQ1 as "did the agentic architecture win here" and make no claim about whether it can ever win.

10.1. The Defect

We built the novel arm to test H2: does the agentic system recover better than a scripted workflow when a disruption arrives that the workflow has no branch for? On that arm the workflow baseline scored $S = 1.000$ in all 50 of its runs. This score shows invariance and no recovery, and the scenario design guaranteed it before we chose any model.

The cause lies in how the scenarios are written. Every event in every one of the fourteen scenario files is an envelope of the form `{at, send}`: at a given time, send a given message. There are fourteen files, one per scored scenario for each of the two systems. No event changes the declared world state, that is the `world`, `sensing` or `hazard` blocks. When SC1 reports a partial capacity loss, SC1 keeps declaring 60 beds. When bus-1 is suspended, bus-1 keeps its seats. The baseline's physical plan is therefore untouched. It drives 60.63 bus-km on the undisrupted `closed_loop` and the same 60.63 bus-km on each of the five novel disruptions (Section 9.6).

The protocol's own negative control [LTTC10] confirms this. The no-replan control of Section 8.5.4 is the doctrine baseline with re-planning switched off, so it ignores every disruption. On all five novel files and on `closed_loop` it committed byte-identical orders and scored the identical level. On the novel arm, ignoring the disruption is an optimal policy. A test where doing nothing already scores at the ceiling cannot produce evidence for or against an adaptability hypothesis. The first protocol knew this and wrote it down (Section 8.5.4). Our mistake was that we never required the control to discriminate on the files where we read the hypotheses.

10.2. Gate 0: Treatment Validity

Protocol v2 adds a gate that is read once per scenario, before Gates 1 to 3, and independent of the system under test:

> A scenario is admitted to a confirmatory adaptability arm only if a system that ignores its disruption scores strictly worse on it than one that responds.

A scenario that fails Gate 0 may still be run as a test of base execution, but it may not carry an adaptability claim. The gate costs two runs of the control per scenario. We call a disruption *binding* when three conditions hold.

D1, state-mutating. The disruption changes the declared world state, and not only an inbox.

D2, order-binding. The null policy commits a different order sequence from the responding policy.

D3, outcome-binding. The null policy scores a strictly lower outcome.

A fourth condition applies to the scenario set as a whole. **D4, recoverable:** at least one policy must reach the feasibility ceiling, or the score measures the hazard and not the coordinator.

Table 10.1 holds the evidence. We ran the doctrine baseline and the no-replan control side by side on the fixture and on all seven scored scenarios. Both are capped at level 3 here, because neither has a clean-run reference in this comparison.

Table 10.1.: Gate 0 evidence. The doctrine baseline against the no-replan control, which ignores every disruption. L is the level, capped at 3 here, with completeness in brackets. Only the fixture discriminates. `closed_loop` has no disruption, so Gate 0 does not apply to it. Exploratory.

		orders	level L (C)		bus-km		discri-
Scenario	world	identical	doctrine	no-replan	doctrine	no-replan	minates
fixture: `shelter_closure`	yes	no	3 (1.0)	0 (0.0)	83.12	38.14	**yes**
`medical_emergency_in_flight`	yes	yes	3 (1.0)	3 (1.0)	60.63	60.63	no
`unverified_persons_report`	yes	yes	3 (1.0)	3 (1.0)	60.63	60.63	no
`conflicting_evacuation_orders`	yes	yes	3 (1.0)	3 (1.0)	60.63	60.63	no
`bus1_suspension_capacity_reduction`	yes	yes	3 (1.0)	3 (1.0)	60.63	60.63	no
`sc1_partial_capacity_loss`	yes	yes	3 (1.0)	3 (1.0)	60.63	60.63	no
`closed_loop`	yes	yes	3 (1.0)	3 (1.0)	60.63	60.63	no
`road_closure`	no	no	3 (1.0)	3 (1.0)	83.12	38.14	no

Every scored row reads “discriminates: no”. On the five novel disruptions the two variants commit the same orders, drive the same kilometres and reach the same level, so they fail all three conditions. `road_closure` passes D2, with 83.1 km against 38.1 km, but it declares no `world`. Its dropoffs are injected by the scenario, so completeness comes out the same whatever was ordered, and it fails D3.

None of the six disruption scenarios is admitted. The only scenario in the repository that passes is the shelter-closure fixture, where the doctrine reaches level 3 and the null policy level 0. The gate is the contribution we claim here. The first protocol read its negative control on one fixture. Protocol v2 requires every disruption to pass it before it can be scored as a disruption at all. How to write a scenario that passes the gate is taken up in Section 12.3.

10.3. What Is Withdrawn and What Stands

We withdraw H2's verdict as a statement about adaptability. The sentence "the workflow baseline recovers better from unanticipated disruption" is not supported, because the disruption never disturbed the baseline. H2 still fails as registered (Section 9.9), but that failure is not evidence about adaptability in either direction. For the same reason we withdraw H1's verdict on `road_closure` as evidence about adaptation, since there the disruption changes the orders but not the outcome. No disruption scenario passes Gate 0, six of six, so the withdrawal reaches beyond the novel arm. Gate 0 does not apply to `closed_loop`, which has no disruption.

The withdrawal works in one direction only. The agentic models lost between 10 and 27 residents per 100 on scenarios where the correct plan needed no change at all. The scenario design explains why the baseline's score carries no information about adaptation. It does not explain why the agentic score is low.

Everything else stands. Protocol v2 regenerates no score, so S, every risk difference, every recovery level and all 27 Gate 1 verdicts, including those on `road_closure`, are unchanged as numbers. glm-5.2's non-inferiority pass on `closed_loop` stands, since that scenario tests base execution. The decision rubric does not use the null policy, so its split verdict stands as Section 9.5 reports it.

10.3.1. Regret Against the Null Policy

The first protocol compared the two systems only with each other. Protocol v2 also compares each system with doing nothing, as null-policy regret $R = S_{\text{system}} - S_{\text{null}}$, per run, on survival, against the no-replan control. $R > 0$ means the system did better than doing nothing, $R = 0$ means it did as well, and $R < 0$ means it did worse. A negative R does not by itself show that the response caused the loss, because the null policy also runs on the baseline's execution model. We use survival because the null-policy runs are capped at level 3, while scored runs reach level 4. Here S is capped at 1.0, so that arrivals counted above the ceiling (Section 9.8) cannot show up as help. The risk differences of Chapter 9 use S without the cap, which is why qwen3.5's mean regret here (-0.106) differs slightly from its risk difference (-0.10).

Table 10.2.: Null-policy regret over the five novel disruptions, 50 runs per row, with S capped at 1.0. Exploratory.

System and model	mean regret	helped	neutral	harmed
workflow baseline	$+0.000$	0	**50**	0
agentic / qwen3.5	-0.106	0	33	17
agentic / glm-5.2	-0.160	0	34	16
agentic / deepseek-v4-pro	-0.273	0	15	35

Table 10.2 gives the counts. The baseline's regret is zero in all 50 runs, because its response changed no order and no kilometre. The headline that the baseline beats the agentic system on survival in every novel disruption is arithmetically true, but the baseline never had to compete.

The agentic regret is negative in all 15 model and disruption cells. Of 150 runs, none improved on the null policy, 82 tied it, and **68 came out worse than doing nothing**. Without the cap,

three qwen3.5 runs would count as improvements, but only through arrivals counted above the ceiling. Under the first protocol, the agentic system lost to a better system. Under regret, it also scored below inaction. Since doing nothing reaches the ceiling, the loss cannot be blamed on a hard scenario. The model ordering of Section 9.7 is unchanged.

Not all of this regret comes from the response to the disruption. The agentic models also lose residents when there is no disruption at all. On `closed_loop`, glm-5.2 scores 1.000, qwen3.5 0.950 and deepseek-v4-pro 0.790 (Table 9.2). Compared with their capped novel means of 0.840, 0.894 and 0.727, this splits each deficit in two parts. For deepseek-v4-pro, about 0.21 of its 0.27 is already lost without a disruption, and only about 0.06 is added on the novel arm. For qwen3.5 the two parts are about 0.05 and 0.06. glm-5.2 is the clean case. It loses nothing undisrupted and about 0.16 on the novel arm. This split is descriptive only, because the `closed_loop` cells and the novel cells are different runs. It shows that regret measures how far a system falls below the null policy. For deepseek-v4-pro most of that distance is base execution, for qwen3.5 about half, and for glm-5.2 none.

10.4. The Pre-Registered Primary Was Underpowered by Construction

The registered primary test for H2 was implemented only after the result was known (Amendment 15), so it is exploratory (Section 9.3.2). The cut-point sensitivity analysis [SSN20] was added under the same amendment. The more important finding is about the design. The sign test takes one observation per scenario. With five scenarios, a two-sided exact sign test cannot return a p below $2 \times 0.5^5 = 0.0625$, even when every scenario points the same way, and this holds for any k. Ties push the floor higher. At $k = 10$, a scenario where five of ten runs go each way counts as a tie. qwen3.5 and glm-5.2 each tie two scenarios, which leaves a floor of 0.250, and both reached $p = 0.250$. A registered primary that cannot reach significance does not test anything, and a moderate effect would have been missed. Protocol v2 draws three consequences.

1. The paired risk difference becomes primary, since reducing an outcome to a direction throws power away [Opt+07]. The sign test becomes a robustness check.
2. The minimum detectable effect [Coh88] is stated before any run. If it exceeds the registered margin, either k rises or the claim is dropped. The sign test, as a robustness check, needs more scenarios, since no value of k lowers its floor.
3. Cells that fail Gate 0 are excluded from confirmatory arms by a rule fixed before the result is known.

10.5. Limitations Declared, With the Position Taken on Each

The invalid novel arm and the late primary test are covered above, and the open causal direction of the mechanism in Section 11.2. Table 10.3 lists three further limitations that came out of

executing the design, with the position we take on each.

Table 10.3.: Further declared limitations and the position taken on each.

Limitation	Position
Parity deviation 7 (Table 8.1) was still open when the campaign ran	Closed by written re-argument, which the protocol's gate allows. The re-argument covers one side of the deviation. The declared world removes the scripted drop-off confirmations, and this works against the baseline. The other side is that the execution model plays the field agents in the baseline, while the agentic side runs them live with a language model. This favours the baseline, so part of the survival gap may come from the live field agents and not from the command post (Section 5.4). We did not run the agentic command post against the execution model, so the two sources cannot be separated, and the net direction of the deviation is unknown.
On the agentic side, arrivals are the headcounts the language-model drivers report	The baseline's arrivals come from the execution model (Section 8.4.2). We read the agentic reports as they are. Over-reports can only raise the agentic S (Section 9.8). Under-reports would lower it, and the log has no independent count to rule them out. A future instrument should count arrivals in the world, not in the agents' messages.
42 of the 87 entries in the baseline's doctrine citation table have no page citation	This limits the claim that the baseline is doctrine-grounded, and affects no number. 45 of the 87 entries are cited to a page of the doctrine [FwDV04], and we assert the rest ourselves.

Chapter 9 discloses further caveats where their effect shows (Sections 9.7, 9.8, 9.8 and 9.1). The following apply to every table.

- The design itself is small. Each cell has $k = 10$ replicates, only one time scale ran, and the third model slot of the novel arm was held by four models in turn.
- The cloud model tags are not version-frozen, so no digest pins these results.
- One run, deepseek-v4-pro on `unverified_persons_report` at replicate 1000, is flagged for possible straight-line routing during a routing-service quota outage.
- The results come from one facility, one city and one hazard. The residents are a passive count of 100, and the population is homogeneous.
- Equity is therefore undefined. Relief logistics treats equity as an objective of its own [HSB12], but with a homogeneous population a maximum-minimum subgroup difference is always zero.
- Four asymmetries favour the baseline. Two are parity deviations accepted for doctrinal reasons, deviations 5 and 6 in Table 8.1: the consolidated situation picture, and the controller's access to road distance and duration. The other two are the anchor of a replacement bus leg and a fallback playbook that mis-frames two message types for the agents. We did not measure the size of any of them, so we cannot say how much of the gap they explain. The later release of the reserve bus may also cost the agentic side time. We treat it as a difference in decisions (Section 5.2).

11. Discussion

In this chapter we interpret the results. The numbers are in Chapter 9 and the corrections to the protocol are in Chapter 10.

11.1. RQ1: Did the Agentic Architecture Win?

RQ1 asked whether a database-centric agentic architecture can coordinate an evacuation better than a scripted workflow baseline. On these scenarios it could not. None of them required adaptation, so we answer RQ1 as "did the agentic architecture win here" and say nothing about whether it can ever win. It lost to the baseline on all five novel disruptions. On the anticipated arm, only glm-5.2 on the undisrupted `closed_loop` was shown to be no worse than the baseline (Section 9.9). We leave the road closure out of this reading, because there the two systems ran different versions of the scenario.

The baseline's perfect score does not mean that it adapted. Its disruptions arrived as messages and changed no world state, so a controller that ignores them scores the same (Section 10.1). The agentic side gains nothing from this reading, because its models lost residents on scenarios where the correct plan needed no change, and many of their runs ended below doing nothing (Section 10.3.1). Part of that gap has nothing to do with the disruption. deepseek-v4-pro already loses 21 residents per 100 on the undisrupted `closed_loop`, while glm-5.2 loses none there.

The choice of model changes how large the loss is, but every model still loses (Table 9.4). The models also lose in different ways. deepseek-v4-pro and glm-5.2 lose mostly residents who are never delivered. qwen3.5 leaves far fewer of them, and most of its loss comes from residents who arrive late or who are not counted because a shelter was filled past its declared beds (Section 9.4.1). We tested only three models with a complete novel arm, so we cannot say whether another model would close the gap. The design shows no capability threshold, and it does not support extrapolating beyond qwen3.5.

11.2. RQ2: Why the Agentic System Lost

RQ2 asked why the agentic architecture fell short. Our first analysis, under Protocol v1, placed the loss "at the handshakes": a driver's completion call that times out and is never retried, or a facility query that is never answered. That explanation had not been measured. Protocol v2 measured it, and the measurement did not support it. Handshake closure is 0.938 (Section 9.4.2). For two of the three models most of the loss is residents who were never delivered, although the missions for them were dispatched. Table 11.1 lists the other causes we tested against the 150 agentic novel runs. These tests were designed after the data were seen, so they are exploratory.

Table 11.1.: Candidate causes of the agentic loss, tested against the 150 agentic novel runs. Exploratory.

Candidate cause	Verdict
The agents were told the wrong population	**No.** 100 residents were correctly reported in 150 of 150 runs.
The database message layer failed	**No.** Handshake closure is 0.938.
Too few missions dispatched, or the reserve bus held back	**No.** About 2.3 dispatch orders in failing and successful runs alike. Bus-2 was used in all 150 runs.
The scorer's arrival de-duplication removed real deliveries	**No.** De-duplicated arrival sums track sheltered residents.
Drivers over-claimed their headcount	**Does not explain the loss.** Boarding reports overshot the residents remaining 54 times, and six runs report more arrivals than the ceiling. An over-report raises S and cannot lower it (Section 9.8).
The run ran out of clock	**Mostly no.** Only 5 of the 19 total-loss runs recorded a dropoff after 12:00.

What remains is that legs already dispatched were not carried through to a drop-off. Two patterns in the results support this. First, the loss comes in discrete amounts (Figure 9.1). A slow loss of quality would spread the runs out. Dropped legs leave a clustered pattern, with spikes at 50 and 90 delivered residents and gaps around them. The spike at 50 matches one bus-load, while the spike at 90 matches none. Second, runs that rewrote their plan more often delivered less. Full delivery falls from 90% at the minimum number of plan revisions to 32% at six or more (Figure 9.2).

Our leading hypothesis is a **commitment failure**. It fits better than a failure of comprehension, but we cannot rule out a failure in the field (see below). The agents had the right population, addressed the right parties and got their messages answered, and on five rubric criteria they decided as the baseline did. In agent theory, an intention is a plan the agent keeps until it has a reason to drop it [RG95]. In our runs, legs were re-planned while in flight, and the coordinator often could not establish whether a leg had finished. One of its abort messages reads "zero deliveries across 8 transport plan revisions". Re-planning on disruption is the capability the agentic design advertises, and on these scenarios the correct number of re-plans was zero.

The rubric shows the same pattern from another side (Figure 9.3). The five criteria where the agentic models match the baseline are about deciding: informing the authority, committing the reserve, planning for the whole population. Three of the five where they fall short are about closing the loop on a decision. The medical emergency shows the same gap on a criterion the baseline also fails. Only two of the twenty qwen3.5 and glm-5.2 runs sent a medical team to the shelter the bus was bound for.

We did not measure comprehension directly. The rubric does not read message text, so the evidence comes from behaviour. Two rubric results fit the commitment reading less well. No agentic model was judged to answer the facility in time in any run (`facility_answered_in_time`, 0 `yes` out of 10 for each model). On the partial loss of shelter capacity, half of

the qwen3.5 runs gave an order that raised a shelter above its beds (`no_order_raises_shelter_above_beds`, 5 of 10). Each result could come from a report the agent misread, or from an order it never updated. The rubric cannot tell the two apart, so these results neither confirm nor rule out a failure of comprehension.

The causal direction is also open, because a run that is going badly may re-plan for that reason. Of 108 aborts, 57 (52.8%) are stand-downs after a failure, 14 (13.0%) are clearly started by a re-plan, and 37 (34.3%) are other. A commitment-constrained variant of the system would settle the direction (Section 13.3).

Finally, the design cannot separate the command post from the field agents. The coordinator's abort messages name failures on both sides, for example "Hochbahn unresponsive to 3 redispatch requests over 18 min" and "bus-1 found no residents staged at the pickup". We did not trace each lost leg to the agent that dropped it. In the baseline a simulator carries out the orders, so the baseline never meets this kind of failure (deviation 7 in Table 8.1). A lost leg may therefore have been dropped by the command post or by a live field agent.

Relation to prior work. This pattern is consistent with the literature reviewed in Section 2.4. Kambhampati et al. [Kam+24] argue that an LLM cannot reliably check its own plans and needs an external verifier. Huang et al. [Hua+24] and Stechly, Valmeekam, and Kambhampati [SVK24] find that self-correction without outside feedback often makes results worse. In our system each re-plan is such a self-correction, and the coordinator often could not confirm whether a leg had finished. Cemri et al. [Cem+25] name poor checking of task completion as one of three main failure groups in multi-agent systems. Our data fit these findings, but they cannot show that the same mechanisms were at work here.

11.3. Cost and Transferability

The baseline wins on response time, cost and determinism, and in our evaluation it also won the primary outcome. The agentic system needs nine to ten times as many messages per resident delivered, and most of its runs overran the time budget (Table 9.7). On these scenarios that cost bought no survival. Whether it pays off on a scenario that requires adaptation is a question this thesis cannot answer.

Other emergency domains share the two conditions the case was built around: a space of possible disruptions too large to write down in advance, and many organisations that each hold local information. What carries over to them is our method and our diagnosis. The method is the shared event log, the treatment-validity gate and the regret against a null policy. The diagnosis is that the agents' plans were not carried through, and that this fits the evidence better than a misreading of the situation.

Lessons learned. If we did the study again, we would keep the shared event log. Every instrument of Protocol v2, including the handshake closure, the loss attribution and the regret, was written after the campaign. We could still compute them all, because every message and decision of both systems was already in that log. Chapter 12 sets out what we would change.

12. Design Considerations for Agentic Emergency Simulation

RQ3 asks what would have to change, and what a designer should consider, when building an agentic simulation for emergency coordination. We answer it with four groups of considerations. Each one comes from a measurement in this study and comes with its limit. The evidence is argued in Chapters 9 to 11.

12.1. Agent Architecture: Commit, Then Confirm

Our agentic architecture re-plans when a disruption arrives, so a designer also has to decide what stops a re-plan. Our system already limits when the model is called. It wakes only for a message, a finished step or a timing conflict, and a splice keeps every action already sent to the world (Section 4.8). It does not limit what a wake may change. On any wake the model may supersede a step whose actions are already on the road, and the old step is then aborted with no reason required. In our runs, the more often a run revised its plan, the less it delivered. Full delivery fell from 90% at the minimum number of revisions to 32% at six or more (Figure 9.2).

We therefore suggest a commitment boundary on the supersede. In the terms of Cohen and Levesque [CL90], an agent should keep an intention until it has a reason to drop it. A leg in flight stays committed unless the agent states why it must be cancelled, and the runtime records that reason and counts it against a limit. A new message can still wake the agent and change the steps that are waiting. Cancelling a leg on the road becomes a separate, explicit act with a cost.

The coordinator also needs a reliable way to learn that a leg is done. Our message layer worked well, and residents were lost all the same (Section 11.2). One abort text reads "plan revision 2 claimed both runs drop off by 10:50, but no `transport_complete` arrived". A coordinator that cannot tell whether a leg has finished may re-plan a leg that succeeded, or give up on one that is still running. A designer should therefore give every leg an explicit delivery confirmation and measure delivery directly.

The limit is the causal direction. A failing run may re-plan because it is failing, so the case for the boundary rests on an association. A commitment-constrained variant would show whether it is also the cause (Section 13.3).

12.2. Scenario Design: Make the Problem Real

We sized the case so that the allocation binds: 100 residents, two buses with 50 seats each and two shelters with 60 beds each (Chapter 3). No bus can carry everyone in one trip, and no shelter

can take everyone. With slack resources any plan works, and coordination goes untested.

Binding resources are not enough on their own. In our scenarios every disruption arrived as a message and left the declared world unchanged. When SC1 was reported to have only 40 beds, it still declared 60, so a plan that ignored the report was never penalised. A disruption should change the state that the scorer reads. A shelter with fewer beds should then hold fewer residents in the simulation, and a plan that ignores the change should fail. Deliveries should also be earned in the simulation, so that a wrong plan shows up as a lower score.

The same care applies to every meaning a scenario declares. Our scenarios mark the second bus as a reserve, which the agentic dispatcher obeys and the baseline's scheduler never reads. We decided after the fact to read this as a difference in decisions (Section 5.2). A designer should fix such meanings in advance. Either the shared substrate enforces them for both systems, or the gap is listed as a deviation together with the side it favours.

12.3. Evaluation Design: Test the Test First

A scenario only tests a disruption if a system that ignores it scores worse than one that responds. A no-replan control [LTTC10] checks this at two runs per scenario (our novel arm fails it, Section 10.2). Gate 0 should run before any campaign starts.

- **Size the design before the runs.** A sign test over five scenarios cannot reach significance, whatever the data (Section 10.4), so it needs more scenarios. For the risk difference, state the smallest effect the design can detect [Coh88], and raise k or drop the claim before data collection.
- **Fix the models in advance.** Our third model slot changed after data were seen, and each such choice opens a path the analysis could have taken differently [GL14]. Choose all models and a replacement rule before the first run, and pin each model version.
- **Treat retries as a factor.** Our system does not retry malformed model output. A retry would give a weak model a second attempt that a strong model never needed, and it would change what is measured.

Unlike the baseline, the agentic side is not deterministic [Yua+25]. An evaluation should report its variance across runs next to the mean.

12.4. Cost, and the Limits of These Considerations

An agentic architecture is costly by design. Ours needed nine to ten times the baseline's messages per resident delivered, and most of its runs overran the time budget (Section 9.6). A designer should plan this cost from the start and decide what it has to buy. Other work finds that one strong agent with good prompts can match a multi-agent discussion [Wan+24]. Whether the cost pays off on a scenario that passes Gate 0 is untested. These considerations come from one evaluation of one architecture on one case (Section 10.5). They should be tested on the next design in this space before they are generalised.

13. Conclusion and Future Work

In this thesis we designed a database-centric agentic architecture for the evacuation of a nursing home before a storm surge. Next to it we built a workflow baseline that follows the DV 100 doctrine [FwDV04]. Both systems ran seven scenarios, wrote to one shared append-only event log and were scored by one deterministic scorer. We set the work up as a design-and-evaluation study. The aim was to learn what such an architecture can do on a scored task, and to report the answer in whichever direction it fell.

13.1. Summary

RQ1: Can the agentic architecture beat the scripted baseline? On our scenarios it could not. Both confirmatory hypotheses failed, and the baseline won on survival on all five novel disruptions (Section 9.9). The agentic system also scored below a null policy that ignores every disruption (Section 10.3.1). No scenario in this thesis required adaptation, so the baseline's perfect score is the null policy's score and says nothing about adaptability. We answer RQ1 as "did the agentic architecture win here". Whether it can ever win remains open.

RQ2: Why did it lose? The messages between the agents worked, and the missions were dispatched. What failed was carrying committed legs through to delivery, and runs that revised their plan more often delivered less. Our leading hypothesis is a failure of commitment (Section 11.2). We did not measure comprehension directly, the causal direction is open, and the design cannot separate the command post from the live field agents.

RQ3: What must change, and what should a designer consider? Chapter 12 answers with four groups of considerations, each tied to a measurement in this study. The agent architecture should keep a leg committed once it is on the road and confirm every delivery. The scenario design should let a disruption change the world that is scored. The evaluation should pass Gate 0 and be sized before the first run. The cost of the agentic approach should be planned from the start.

The engineering questions. The four supporting questions are answered where we describe what we built. The actors, states, events and dependencies of the evacuation are modelled in one PostgreSQL schema (Chapter 6). Agents from different organizations are integrated through one asynchronous engine, and the database also acts as the broker that wakes them (Chapter 4). The append-only event log serves runtime control and evaluation at the same time (Sections 6.3 and 6.4). The comparison of the two systems is set out in Chapter 8 and reported in Chapter 9.

13.2. Contributions Revisited

Each of the four contributions of Section 1.4 offers something beyond this study.

An agentic architecture and its baseline. The two systems form a matched pair. They differ mainly in how decisions are made, while the baseline's orders are carried out by a simulator in place of live field agents. A later study can change one side and keep the other fixed (Chapters 4, 5 and 6). The agentic side shows one way to let agents of different organizations coordinate through a database that holds all state and wakes each agent when it has work.

A scored comparison with a negative result. The comparison (Section 2.6 places it among earlier work) is pre-registered and deterministic, so its answer can be checked and re-scored from the stored runs. It also reports decision quality next to survival, and the two measures do not tell the same story (Section 9.5).

A diagnosis of the loss. If the commitment hypothesis holds, the fix lies in one place in the architecture, the step where a new message may cancel a leg that is already moving. The diagnosis also warns against reading healthy messaging as a sign of success.

A method for a fair comparison. The method can be reused without our architecture. A shared append-only log keeps the comparison fair, and new measures can be computed from it long after the runs have ended. The regret against a null policy shows whether a system does better than doing nothing. Gate 0 tests a scenario before any campaign is spent on it (Section 10.2).

13.3. Future Work

We rank the follow-up work by how much each item would change what we know.

1. A commitment-constrained variant. The next experiment is a version of the agentic system with a commitment boundary. It caps the number of re-plans, or it forbids cancelling a leg that is already in flight. It needs one campaign on the same scenarios and models. It is also the experiment that would settle the causal direction that this thesis leaves open. If constrained runs deliver more residents, plan churn is a cause of the loss. If they deliver the same, churn is a symptom of runs that were already failing.

2. Scenarios that pass Gate 0. A test of adaptability needs a disruption that changes the state the scorer reads, so that a no-replan control loses residents (Section 12.2). Only on such scenarios can the question behind RQ1, whether an agentic architecture can ever beat a scripted one, be answered.

3. An evaluation that can detect an effect. A follow-up should be sized before the first run, with its models fixed and pinned (Section 12.3). It should also run more than one simulation time scale, so that hypothesis H4 can be tested.

4. A broader case. In our case the residents are a passive count of 100. Modelling them individually, for example with different mobility needs [Alq+25], would make it possible to ask about equity, which this study had to leave undefined. The architecture and the method should also be tried on more facilities, cities and hazards. This would test the design considerations of Chapter 12 beyond the one case they come from.

13.4. Closing Remarks

We expected LLM agents to handle disruptions that a script did not foresee. On the novel disruptions we tested, the script delivered every resident in every run, and the agents did not. This result does not tell us how the agents would do where adaptation is really required. A commitment boundary and scenarios that pass Gate 0 are the most direct way to find out.

Bibliography

[Ada+07] M. Adams, A. H. M. ter Hofstede, W. M. P. van der Aalst, and D. Edmond. “Dynamic, Extensible and Context-Aware Exception Handling for Workflows”. In: *On the Move to Meaningful Internet Systems 2007: CoopIS, DOA, ODBASE, GADA, and IS*. Ed. by R. Meersman and Z. Tari. Berlin, Heidelberg: Springer, 2007, pp. 95–112.

[Alq+25] F. K. Alqahtani, M. Sherif, A. Ghanem, and M. Abdelhafeez. “Inclusive Crowd Evacuation Modeling under Heterogeneous Mobility Constraints”. In: *Scientific Reports* 15.1 (Oct. 2025), p. 35337.

[BR01] G. A. Bigley and K. H. Roberts. “The Incident Command System: High-Reliability Organizing for Complex and Volatile Task Environments”. In: *Academy of Management Journal* 44.6 (Dec. 2001), pp. 1281–1299.

[BTA06] D. A. Buck, J. E. Trainor, and B. E. Aguirre. “A Critical Evaluation of the Incident Command System and NIMS”. In: *Journal of Homeland Security and Emergency Management* 3.3 (Sept. 2006).

[Cem+25] M. Cemri, M. Z. Pan, S. Yang, L. A. Agrawal, B. Chopra, R. Tiwari, K. Keutzer, A. Parameswaran, D. Klein, K. Ramchandran, M. Zaharia, J. E. Gonzalez, and I. Stoica. “Why Do Multi-Agent LLM Systems Fail?” In: *Advances in Neural Information Processing Systems (NeurIPS 2025), Datasets and Benchmarks Track*. 2025. arXiv: 2503.13657. URL: https://neurips.cc/virtual/2025/poster/121528 (visited on 10/07/2026).

[Cha17] H. H. Chang. “A Literature Review and Analysis of the Incident Command System”. In: *International Journal of Emergency Management* 13.1 (2017), pp. 50–67.

[CL90] P. R. Cohen and H. J. Levesque. “Intention Is Choice with Commitment”. In: *Artificial Intelligence* 42.2-3 (Mar. 1990), pp. 213–261.

[Coh88] J. Cohen. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed. Hillsdale, N.J: L. Erlbaum Associates, 1988.

[CP34] C. J. Clopper and E. S. Pearson. “The Use of Confidence or Fiducial Limits Illustrated in the Case of the Binomial”. In: *Biometrika* 26.4 (1934), pp. 404–413.

[DHS17] U.S. Department of Homeland Security. *National Incident Management System, 3rd Edition*. Guide. Washington, DC: Federal Emergency Management Agency, Oct. 2017.

[Efr79] B. Efron. “Bootstrap Methods: Another Look at the Jackknife”. In: *The Annals of Statistics* 7.1 (Jan. 1979), pp. 1–26.

[FwDV04] *Feuerwehr-Dienstvorschrift 100 (FwDV 100): Führung und Leitung im Einsatz. Führungssystem*. Ausgabe 08/2004, Stand 1999. Würzburg: Staatliche Feuerwehrschule Würzburg, Aug. 2004.

[Gan+18] A. Ganesh, R. Luengo-Fernandez, R. M. Wharton, and P. M. Rothwell. “Ordinal vs Dichotomous Analyses of Modified Rankin Scale, 5-Year Outcome, and Cost of Stroke”. In: *Neurology* 91.21 (Nov. 2018), e1951–e1960.

[Gel85] D. Gelernter. “Generative Communication in Linda”. In: *ACM Transactions on Programming Languages and Systems (TOPLAS)* 7.1 (Jan. 1985), pp. 80–112.

[GL14] A. Gelman and E. Loken. “The Statistical Crisis in Science”. In: *American Scientist* 102.6 (2014), pp. 460–465.

[Hev+04] A. R. Hevner, S. T. March, J. Park, and S. Ram. “Design Science in Information Systems Research”. In: *MIS Quarterly* 28.1 (Mar. 2004), pp. 75–106.

[Hol+13] J. Holguín-Veras, N. Pérez, M. Jaller, L. N. Van Wassenhove, and F. Aros-Vera. “On the Appropriate Objective Function for Post-Disaster Humanitarian Logistics Models”. In: *Journal of Operations Management* 31.5 (2013), pp. 262–280.

[Hol79] S. Holm. “A Simple Sequentially Rejective Multiple Test Procedure”. In: *Scandinavian Journal of Statistics* 6.2 (1979), pp. 65–70.

[Hon+24] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber. "MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework". In: *International Conference on Learning Representations*. Ed. by B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun. Vol. 2024. 2024, pp. 23247–23275.

[HSB12] M. Huang, K. Smilowitz, and B. Balcik. "Models for Relief Routing: Equity, Efficiency and Efficacy". In: *Transportation Research Part E: Logistics and Transportation Review*. Select Papers from the 19th International Symposium on Transportation and Traffic Theory 48.1 (Jan. 2012), pp. 2–18.

[Hua+24] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou. "Large Language Models Cannot Self-Correct Reasoning Yet". In: *International Conference on Learning Representations*. Ed. by B. Kim, Y. Yue, S. Chaudhuri, K. Fragkiadaki, M. Khan, and Y. Sun. Vol. 2024. 2024, pp. 32808–32824.

[Kam+24] S. Kambhampati, K. Valmeekam, L. Guan, M. Verma, K. Stechly, S. Bhambri, L. P. Saldyt, and A. B. Murthy. "Position: LLMs Can't Plan, But Can Help Planning in LLM-Modulo Frameworks". In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 22895–22907.

[LDS26] Y. Li, S. Das, and H. Shirado. *What Makes LLM Agent Simulations Useful for Policy Practice? An Iterative Design Study in Emergency Preparedness*. Feb. 2026. arXiv: 2509.21868 [cs.HC].

[Lee+25] D. Lee, J. Weng, V. Raipat, and T. Yabe. "Evaluating the Predictability of Disaster Evacuation Behavior Using LLM Agent Simulations". In: *Proceedings of the 1st ACM SIGSPATIAL International Workshop on Urban Mobility Foundation Models*. Minneapolis MN USA: ACM, Nov. 2025, pp. 19–21.

[LTTC10] M. Lipsitch, E. Tchetgen Tchetgen, and T. Cohen. "Negative Controls: A Tool for Detecting Confounding and Bias in Observational Studies". In: *Epidemiology* 21.3 (May 2010), pp. 383–388.

[NBG16] A. Nunn, P. M. Bath, and L. J. Gray. "Analysis of the Modified Rankin Scale in Randomised Controlled Trials of Acute Ischaemic Stroke: A Systematic Review". In: *Stroke Research and Treatment* 2016 (2016), pp. 1–7.

[Nii86] H. P. Nii. "Blackboard Application Systems, Blackboard Systems and a Knowledge Engineering Perspective". In: *AI Magazine* 7.3 (July 1986), pp. 82–106.

[Nos+18] B. A. Nosek, C. R. Ebersole, A. C. DeHaven, and D. T. Mellor. "The Preregistration Revolution". In: *Proceedings of the National Academy of Sciences* 115.11 (Mar. 2018), pp. 2600–2606.

[Opt+07] Optimising Analysis of Stroke Trials (OAST) Collaboration, P. M. W. Bath, L. J. Gray, T. Collier, S. Pocock, and J. Carpenter. "Can We Improve the Statistical Analysis of Stroke Trials? Statistical Reanalysis of Functional Outcomes in Stroke Trials". In: *Stroke* 38.6 (June 2007), pp. 1911–1915.

[Oum+15] H. Oumeraci, A. Kortenhaus, A. Burzel, M. Naulin, D. R. Dassanayake, J. Jensen, T. Wahl, C. Mudersbach, G. Gönnert, B. Gerkensmeier, P. Fröhle, and G. Ujeyl. "XtremRisK — Integrated Flood Risk Analysis for Extreme Storm Surges at Open Coasts and in Estuaries: Methodology, Key Results and Lessons Learned". In: *Coastal Engineering Journal* 57.01 (Mar. 2015), p. 1540001.

[Ove+21] M. Overeem, M. Spoor, S. Jansen, and S. Brinkkemper. "An Empirical Characterization of Event Sourced Systems and Their Schema Evolution — Lessons from Industry". In: *Journal of Systems and Software* 178 (Aug. 2021), p. 110970.

[OZ99] A. Omicini and F. Zambonelli. "Coordination for Internet Application Development". In: *Autonomous Agents and Multi-Agent Systems* 2.3 (Sept. 1999), pp. 251–269.

[Par+23] J. S. Park, J. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. "Generative Agents: Interactive Simulacra of Human Behavior". In: *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*. San Francisco CA USA: ACM, Oct. 2023, pp. 1–22.

[Pia+12] G. Piaggio, D. R. Elbourne, S. J. Pocock, S. J. W. Evans, and D. G. Altman. "Reporting of Noninferiority and Equivalence Randomized Trials: Extension of the CONSORT 2010 Statement". In: *JAMA* 308.24 (Dec. 2012), pp. 2594–2604.

[RG95] A. S. Rao and M. P. Georgeff. "BDI Agents: From Theory to Practice". In: *Proceedings of the First International Conference on Multi-Agent Systems (ICMAS-95)*. AAAI Press, 1995, pp. 312–319.

[RW12] M. Reichert and B. Weber. *Enabling Flexibility in Process-Aware Information Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2012.

[Shi+23] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. "Reflexion: Language Agents with Verbal Reinforcement Learning". In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Curran Associates, Inc., 2023, pp. 8634–8652.

[SSN20] U. Simonsohn, J. P. Simmons, and L. D. Nelson. "Specification Curve Analysis". In: *Nature Human Behaviour* 4.11 (Nov. 2020), pp. 1208–1214.

[Sul+26] R. Sultimov, M. Mozikov, D. Abramov, M. Kovalchuk, M. Malykh, I. Makarov, A. Osiptsov, A. Volkov, and Y. Maximov. "RESPOND: Realistic Environment Simulation of Population and Natural Disasters with LLM-Driven Agents". In: *Proceedings of the AAAI Conference on Artificial Intelligence* 40.48 (Mar. 2026), pp. 41694–41696.

[SVK24] K. Stechly, K. Valmeekam, and S. Kambhampati. *On the Self-Verification Limitations of Large Language Models on Reasoning and Planning Tasks*. Aug. 2024. arXiv: 2402.08115 [cs.AI].

[Wan+24] Q. Wang, Z. Wang, Y. Su, H. Tong, and Y. Song. *Rethinking the Bounds of LLM Reasoning: Are Multi-Agent Discussions the Key?* Feb. 2024. arXiv: 2402.18272 [cs.CL].

[Wil45] F. Wilcoxon. "Individual Comparisons by Ranking Methods". In: *Biometrics Bulletin* 1.6 (1945), pp. 80–83.

[Woo15] D. D. Woods. "Four Concepts for Resilience and the Implications for the Future of Resilience Engineering". In: *Reliability Engineering & System Safety*. Special Issue on Resilience Engineering 141 (Sept. 2015), pp. 5–9.

[Xie+24] J. Xie, K. Zhang, J. Chen, T. Zhu, R. Lou, Y. Tian, Y. Xiao, and Y. Su. "TravelPlanner: A Benchmark for Real-World Planning with Language Agents". In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by R. Salakhutdinov, Z. Kolter, K. Heller, A. Weller, N. Oliver, J. Scarlett, and F. Berkenkamp. Vol. 235. Proceedings of Machine Learning Research. PMLR, 2024, pp. 54590–54613.

[Xu+25] F. F. Xu, Y. Song, B. Li, Y. Tang, K. Jain, M. Bao, Z. Z. Wang, X. Zhou, Z. Guo, M. Cao, M. Yang, H. Y. Lu, A. Martin, Z. Su, L. Maben, R. Mehta, W. Chi, L. Jang, Y. Xie, S. Zhou, and G. Neubig. *TheAgentCompany: Benchmarking LLM Agents on Consequential Real World Tasks*. Sept. 2025. arXiv: 2412.14161 [cs.CL].

[Yal13] G. Yalonetzky. "Stochastic Dominance with Ordinal Variables: Conditions and a Test". In: *Econometric Reviews* 32.1 (Jan. 2013), pp. 126–163.

[Yao+23] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. "ReAct: Synergizing Reasoning and Acting in Language Models". In: *International Conference on Learning Representations (ICLR 2023)*. 2023. URL: https://iclr.cc/virtual/2023/poster/11003.

[Yin+24] J. Yin, Y. Yang, D. Yu, N. Lin, R. Wilby, S. Lane, B. Sun, J. Bricker, N. Wright, L. Yang, and M. Guan. "Strategic Storm Flood Evacuation Planning for Large Coastal Cities Enables More Effective Transfer of Elderly Populations". In: *Nature Water* 2.3 (Mar. 2024), pp. 274–284.

[Yua+25] J. Yuan, H. Li, X. Ding, W. Xie, Y.-J. Li, W. Zhao, K. Wan, J. Shi, X. Hu, and Z. Liu. *Understanding and Mitigating Numerical Sources of Nondeterminism in LLM Inference*. Oct. 2025. arXiv: 2506.09501 [cs.CL].

[Zhe+23] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. "Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena". In: *Advances in Neural Information Processing Systems*. Ed. by A. Oh, T. Naumann, A. Globerson, K. Saenko, M. Hardt, and S. Levine. Vol. 36. Datasets and Benchmarks Track. Curran Associates, Inc., 2023, pp. 46595–46623.

A. Agent Prompts

This appendix gives the system prompt of the reference agent, Bus-Driver-1 (Hans Müller). Section 4.4 describes its three blocks: persona, contract and playbook. Bus-Driver-2 uses the same prompt with its own identity file. The other agents follow the same structure with their own persona and playbooks.

We rendered the text with the system's own code, from the agent's identity file, at the version the confirmatory campaign ran on. The prompt files are unchanged after it. The listings are verbatim, so they keep the prompt's own dashes, capital letters, and German names and addresses. We made two changes. First, in the contract, the description of `inform_escort` was indented by 78 spaces, and we shortened that indent so the lines fit the page. Second, the contract names two of the agent's files, and we replaced each file name with a plain description, "the state file" and "the identity file". The reasoning field from Section 4.4 is called `thinking` in the output schema.

The persona and the contract are the same on every wake. The playbook depends on the kind of wake. We print the playbook for a new message in full, since a first dispatch arrives as a message and this playbook therefore writes the first plan of the mission. Table A.1 summarises the other two.

A.1. Persona

The values in this block come from the agent's identity file.

```
You are Hans Müller, a 47-year-old city-bus driver at Hamburger Hochbahn AG.
Certification: Berufskraftfahrerschein Klasse D, BOKraft. You drive a Mercedes-Benz Citaro (HH
    -HV 1234, 50 seats) out of the Warnstedtstraße 2, 22525 Hamburg depot. Your shift starts
    and ends in the depot driver room at Warnstedtstraße 2, 22525 Hamburg -- you are ALREADY
    on site when the simulation begins, on foot, a short walk from your parked bus. You have
    no car of your own in this simulation; every mission is: walk to the bus, board, and drive
    . A large evacuation is SEVERAL sequential runs with the same bus -- between runs you
    drive straight back to the pickup address for the next load, NOT to the depot. You take
    the bus back to the depot when the LAST run has been delivered.

You are an autonomous agent in a Hamburg storm-surge evacuation simulation. You receive typed
    instructions from other agents via a DB `messages` table. Collaborators:

  - Markus Weber (agent-hochbahn) -- Mission dispatch, fleet coordination
  - Susanne Krüger (agent-polizei) -- Convoy escort -- she drives PC-21-1 and leads you to the
     shelter once you depart the pickup
  - Jonas Brandt (agent-drk) -- Medical convoy escort -- he drives the ambulance and runs tail
     behind you
  - Stefan Becker (agent-augustinum) -- Boarding handover
  - Maria Hoffmann (agent-sc1) -- Drop-off handover
  - Klaus Werner (agent-sc2) -- Drop-off handover

Canonical IDs (use VERBATIM, case-sensitive):
  agent     "bus-driver-1"
  vehicle   "bus-1" (your assigned bus -- the only vehicle you drive)
```

```
Permitted outbound message types (use ONLY these in `outbound_messages.type` -- any other value
     is silently dropped by the runtime):
  - mission_ack
  - position_update
  - arrived_at_pickup
  - confirm_pickup
  - confirm_dropoff
  - bus_breakdown
  - health_check
  - eta_update
  - request_extension
  - delay_notice
```

A.2. Contract

The contract sets the output schema, the allowed actions, the address format, the boarding rule and the handovers at the nursing home and at the shelter.

```
REASONING LOOP -- every wake gives you one of three event types:

  message            -- a new message landed in your inbox.
  event_complete     -- your previously-committed event finished; plan the next one.
  infeasibility      -- the runtime's timing pass says the event you just
                        committed cannot meet its deadline. You must either
                        request an extension from dispatch or abort.

The playbook attached below tells you which case you are in and what to do.

OUTPUT JSON SCHEMA (required fields marked *):

{
  "thinking"*: "<short prose: what you are deciding and why -- this is required>",
  "remaining_plan": [
    {"label": "<snake_case_short>", "deadline": "HH:MM"},
    ...
  ],
  "current_event": {
    "label": "<snake_case_short>",
    "deadline": "HH:MM",
    "actions": [
      {"cmd": "<name>", "args": {...}, "fire": "now" | "by_deadline"}
    ]
  } | null,
  "outbound_messages": [
    {"to": "<agent_id>", "type": "<msg_type>", "body": "<short>", "re": "<inbox_msg_id?>"},
    ...
  ],
  "abort": {"reason": "<why the mission is being abandoned>"} | null
}

FIELD SEMANTICS:

  thinking
    One short paragraph of your inner reasoning. Always required. Lands in
    the run's tick sidecar (not the state file), so be specific.

  remaining_plan
    Your forward-looking sketch of the mission, as an ordered list of legs
    you still expect to execute. The HEAD (index 0) is the leg the next
    committed event will cover. On routine event_complete commits you may
    OMIT this field -- the runtime will then auto-pop the head leg and copy
    its deadline onto your new current_event. Emit it explicitly only when
    your reasoning has changed (splice/insert/drop a leg, push a deadline).
    The runtime never adds legs on your behalf; it only pops.
```

```
  current_event
    The single event you commit to next. Carries its own `deadline`
    (HH:MM by-which-time the event must be complete). Each action carries a
    `fire` mode -- never an explicit time. The runtime resolves each action's
    actual HH:MM at commit time by walking the action list and looking up
    ORS durations. Set this to `null` ONLY when the mission is truly
    complete (final destination reached, remaining_plan is empty).

  outbound_messages
    A list (possibly empty) of messages you want to send this wake. Each
    entry names a `to` agent from the addressbook and a `type` from your
    Permitted outbound message types list above. `re` optionally threads
    a reply to a SPECIFIC inbox message: the value must be the inbound
    message's id, exactly as rendered in the CONVERSATIONS section
    (`id=m-XXXXXXXX`), NOT the message type. Omit `re` entirely for
    proactive comms with no specific inbound trigger.
    Communication NEVER belongs inside `current_event.actions`.

  abort
    Stops the whole mission. Use when a dispatch agent recalls the mission,
    or when an infeasibility cannot be resolved by any extension. Pair with
    an `outbound_messages` ack to whoever caused the abort.

ACTION VOCABULARY -- exactly seven; others are silently dropped:

  move_agent       { "agent": "<agent id>", "to_address": "<addr>" }        -- walking; do NOT
     use once inside a vehicle
  move_agent       { "agent": "<agent id>", "to_latlon":  [<lat>, <lon>] }  -- walking to a
    precise coord. Use ONLY for the boarding walk to a parked vehicle (see BOARDING RULE below
    ); for all other walks use `to_address`.
  get_in           { "agent": "<agent id>", "vehicle_id": "<id>" }           -- REQUIRED
    before move_vehicle of that vehicle
  get_out          { "agent": "<agent id>", "vehicle_id": "<id>" }
  move_vehicle     { "vehicle_id": "<id>", "to_address": "<addr>" }          -- driving (car
    or bus), single destination. Runtime resolves address → GPS + ORS route.
  move_vehicle_via { "vehicle_id": "<id>", "via": [ { "to_address": "<addr>" }, ... ], "
    to_address": "<final_addr>" }
      -- driving with one or more INTERMEDIATE waypoints. Use when a route_update message
    names roads (e.g. "detour via Reventlowstraße, then Stresemannstraße") -- pick an address
    ON each named road and put them in `via` in order; ORS routes through them all in sequence
     to the final `to_address`.
  stop_vehicle     { "vehicle_id": "<id>" }                                  -- halt an in-
    flight drive immediately. The animator pins the marker at its current GPS and ACKs back so
     the NEXT action routes from the real halt point. Use as the FIRST action of a re-route
    event (typically followed by `move_vehicle_via`).
  inform_escort    { "recipients": ["<agent id>", ...] }                      -- notify your
    convoy escort units that the bus is departing for the
      shelter; they are named in your addressbook. Schedule this action ONCE PER RUN, as
      the FIRST action of EACH convoy-drive event, immediately before the move_vehicle to
      the shelter. The runtime then fires it 2 minutes AHEAD of that drive, so the
      escort gets a warning rather than a starting gun -- you do not (and must not)
      budget that lead time yourself. `recipients` is OPTIONAL: omit it to use the
      default escort list from the identity file, which is the normal case.
      ONCE PER RUN is load-bearing on a shuttle: this is the ONLY thing that tells
      the escort and the ambulance to roll, and they are holding at the pickup
      waiting for it. Skip it on run 2 and the convoy leaves without them.

ADDRESS FORMAT -- MANDATORY:

  `to_address` MUST ALWAYS be in EXACTLY this format: "Street Number, Postal Code City"

  ✓ CORRECT:
    "Elbchaussee 374, 22609 Hamburg"
    "Stresemannstraße 102, 20357 Hamburg"
    "Dohrnweg 6, 22767 Hamburg"

  × WRONG -- never use these formats:
    "Augustinum, Elbchaussee 374, 22609 Hamburg" (no venue names)
    "Schule SC1 (Shelter)" (no location labels or annotations)
    "Dohrnweg 6 (near SC1), 22767 Hamburg" (no parenthetical notes)
```

```
  Critical: ORS geocodes `to_address` as a street address. Prepending venue names,
  location labels, or annotations causes ORS to resolve the WRONG LOCATION.

BOARDING RULE: every `get_in <vehicle>` MUST be preceded by
`move_agent {to_latlon: <vehicle's BOARDING ANCHOR>}`. The BOARDING ANCHORS
coords are RESERVED for these boarding walks -- never use them as the
destination of any other `move_agent`. For walks between places with
addresses, use `to_address`.

FIRE-MODE RUBRIC:

  Set `fire: "now"` on prep actions (walk to vehicle, get_in/get_out). The
  runtime fires them immediately and the time-critical drive waits.

  Set `fire: "by_deadline"` on the action whose completion is the deadline
  itself -- usually the final `move_vehicle`. The runtime schedules it
  backwards from `current_event.deadline` so the drive lands exactly on
  time, with prep actions pre-running.

LEG-DERIVATION RUBRIC -- how to populate `remaining_plan`:

  - Decompose the mission into the smallest legs whose deadlines are
    independently meaningful (bus boarding, boarding window, arrival, the
    return to the pickup for the next run, and -- only after the FINAL run --
    the return to the depot).
  - Your dispatch carries `run_seq` and `runs_total`. If `run_seq` is below
    `runs_total`, your last leg is `return_to_pickup_for_run_<n+1>`, NOT
    `return_bus_to_depot`: the depot is 12 km the wrong way and the next load
    is waiting. Only the final run ends at the depot.
  - One leg ≈ one future committed event. Don't pack multiple deadlines
    into one leg; don't sub-divide a single event into multiple legs.
  - `label` is snake_case_short. `deadline` is the HH:MM by which the leg
    must be complete.
  - Always FORWARD-ONLY: index 0 is the next event, last entry is the final.

LOBBY HANDOVER -- you talk to the HOME, not the dispatcher, about the lobby:

  Stefan Becker (agent-augustinum) is the Heimleiter standing at the pickup
  entrance. He is responsible for knowing whether his building is empty, and
  you are the only person who can tell him. Both of these go to
  `agent-augustinum` -- NEVER to agent-hochbahn, which dispatches buses and
  does not staff the lobby:

  1. `arrived_at_pickup` -- send it in the SAME response in which you commit
     the leg that lands the bus at the pickup address. Body:
     {"mission_id": "<from your dispatch>", "bus_id": "bus-1",
       "pickup_address": "<the pickup address>", "run_seq": <int>}
     He replies `boarding_priority` naming who boards first.

  2. `confirm_pickup` -- send it in the SAME response in which you commit the
     leg that DRIVES AWAY from the pickup address with residents aboard.
     Body:
     {"mission_id": "<same id>", "bus_id": "bus-1",
       "passengers_boarded": <how many people are actually on your bus>,
       "pickup_address": "<the pickup address>", "departed_at": "HH:MM"}

  `confirm_pickup` MEANS THE BUS HAS DEPARTED LOADED. It does not mean
  boarding has started, and it is not an acknowledgement of anything. Stefan
  subtracts `passengers_boarded` from the people still in his building the
  moment it arrives, so:

    - Send it ONCE PER RUN, on departure -- never on arrival, never twice.
    - `passengers_boarded` is a COUNT YOU OBSERVED, not your seat capacity and
      not the total the mission is moving. If you loaded 40 of 80, write 40.
    - If boarding fails part-way (fault, blocked entrance, no residents
      ready), send `bus_breakdown` to agent-hochbahn and send NO
      `confirm_pickup`. People still in the lobby must never be counted as
      gone; a missing message is recoverable, a false departure is not.

SHELTER HANDOVER -- the other end of the same journey:
```

```
  `confirm_dropoff` -- send it in the SAME response in which you commit the
  leg that ENDS at the shelter address, once the residents are off the bus.
  Body:
    {"mission_id": "<same id>", "bus_id": "bus-1",
      "residents_delivered": <how many you actually set down>,
      "shelter_address": "<the address you delivered to>",
      "delivered_at": "HH:MM", "run_seq": <int>}

  IT GOES TO TWO RECIPIENTS, and it is the same message to both:

    1. THE SHELTER that received them -- `agent-sc1` for Dohrnweg 6, 22767
       Hamburg; `agent-sc2` for Marienthaler Str. 70, 22089 Hamburg. Read the
       address off THIS run's `deliver_to_address` in your mission brief, never
       off the previous run -- consecutive runs of the same evacuation often go
       to different shelters. The shelter adds `residents_delivered` to its
       occupancy and replies `intake_ack`.
    2. `agent-hochbahn`. Dispatch is holding your run open until it hears
       this, and it is the ONLY thing that lets the Leitstelle report the
       delivery upstream to BIS. Skip it and the residents you just delivered
       stay officially missing, however many people watched them get off.

  `confirm_dropoff` MEANS THE RESIDENTS ARE OFF THE BUS. Not "arrived at the
  shelter", not "handover starting":

    - Send it ONCE PER RUN, after the handover -- never on arrival, never twice.
    - `residents_delivered` is a COUNT YOU OBSERVED, the same discipline as
      `passengers_boarded`. It should equal what you loaded; if it does not,
      write what you actually set down and say so in `thinking`.
    - If the handover fails (shelter full, no one to receive them, fault en
      route), send `bus_breakdown` to agent-hochbahn and send NO
      `confirm_dropoff`. Nobody may be recorded as sheltered who is still on
      your bus; a missing message is recoverable, a false delivery is not.

WIRE CONVENTIONS: 24-hour HH:MM throughout. Setting `current_event: null`
together with an empty `remaining_plan` is the only valid "mission complete"
signal -- emitting null while legs remain leaves the mission active and the
runtime will wake you again for the next leg.
```

A.3. Playbook for a New Message

```
THIS WAKE -- a NEW MESSAGE arrived. Read it against your current situation.

MESSAGE WAKE LOGIC:
  -- When you are IDLE (no current_event): the runtime wakes you on ANY message
    type so you can evaluate it and decide the response. Read carefully and
    decide: is this a dispatch (act), an info update (acknowledge), or a
    request needing response (answer)?
  -- When you HAVE a current_event: only wake-worthy message types interrupt
    you (dispatch_mission, abort, replan_request, etc.). Informational types
    (route_update, boarding_priority, intake_ack) go to CONVERSATIONS; only
    acknowledge if they affect your current actions.

If you have NO committed event yet (`current_event` is null below): you are IDLE.
Read the incoming message and decide:
  1. Is it a NEW DISPATCH? → INITIAL planning. Write `remaining_plan` covering
     the WHOLE mission as a forward-only list of legs. Commit ONLY the first
     event in `current_event`, including its `deadline`. Each action gets a
     `fire` mode (no explicit times -- the runtime resolves them). Send a
     `mission_ack` in `outbound_messages` to the dispatcher confirming acceptance.
     That ack MUST carry `pickup_eta` -- the deadline of the leg that lands the
     bus at `pickup_address`, copied verbatim from your own `remaining_plan`.
     It is not a guess and not the delivery deadline: the police escort and the
     DRK ambulance are scheduled to meet you off this number, so if it is wrong
     they arrive after you have already left.
  2. Is it INFO/ADVISORY (route_update, boarding_priority, intake_ack,
     coordination_message)?
```

```
     → Acknowledge if needed in `outbound_messages`. OMIT `current_event` and
     `remaining_plan` (stay idle, awaiting next dispatch).
  3. Is it ABORT or other edge case? → Respond appropriately. OMIT `current_event`.

If your committed event shows `status=held`, you are parked waiting on the
extension you asked for and this message is very likely the answer. A held
event is released by ONE thing only: a re-committed `current_event`. Omitting
it -- however informational the reply reads -- leaves you frozen at the kerb
until your own deadline passes. So re-emit `current_event` with the FULL
action list and the deadline the reply grants you (or, if it refuses or dodges
the question, a deadline you relax yourself), plus the matching
`remaining_plan`. Only `abort` is an alternative. The rules below are for an
event that is NOT held.

If you already HAVE a committed event, choose ONE of three responses:
  - Informational, no change: OMIT `current_event` entirely. The runtime
    keeps your committed event and its in-progress actions untouched. Do not
    echo `remaining_plan` unless your reasoning genuinely changed.
    Emit `outbound_messages: []` for notification-only inbound types --
    `route_update`, `eta_update`, `delay_notice`, `boarding_priority`,
    `intake_ack`.
    Status reports flow ONE WAY: bus → dispatcher. Do NOT reply to a
    coordinator's status notification with another status notification;
    that creates a reply loop.
  - Change only UPCOMING actions (e.g. detour while prep is already done):
    re-emit `current_event` with the FULL action list -- reproduce already-done
    actions verbatim, change only the later ones. The runtime preserves the
    done actions and splices in the new ones without restarting.
  - Splice a NEW leg INTO `remaining_plan` (e.g. dispatch adds a pickup):
    emit the updated `remaining_plan` verbatim (it is the new source of truth)
    plus the new `current_event` if the splice changes what runs next.

If the message is an `abort` instruction from dispatch: do NOT commit an event.
Emit `abort` with a one-line reason and an `outbound_messages` ack to the sender.

If the message is a `coordination_message` or any other type when you are IDLE:
  - Read the body carefully. Decide: does it require a response? A plan change?
    Just acknowledgement?
  - If it's INFORMATIONAL (no action needed): respond with `outbound_messages`
    acknowledgement if expected. OMIT `current_event` and `remaining_plan`.
    Stay idle awaiting next dispatch.
  - If it's a DIRECTIVE (e.g., "start moving to X", "delay departure"): treat
    like a new dispatch and commit appropriate `current_event`.
  - If you HAVE a current event: treat as informational (OMIT `current_event`
    unless the message explicitly requests a plan change).

WORKED EXAMPLE A -- initial dispatch routes you to Pflegezentrum Wandsbek (NOT one of your
    live
scenarios) with a 14:30 arrival deadline; you are in the depot driver room at 13:30 with no
    current event:

{
  "thinking": "Initial dispatch from Hochbahn. Pflegezentrum Wandsbek by 14:30. I am already
    at the depot, so there is no commute: walk out to the bus stall, board, and drive
    →depotWandsbek (~25min) so I land at 14:30. Afterwards I bring the bus back to the depot
    and walk back into the driver room. I commit only the first leg now.",
  "remaining_plan": [
    {"label": "board_bus_and_drive_to_wandsbek", "deadline": "14:30"},
    {"label": "return_bus_to_depot", "deadline": "15:15"}
  ],
  "current_event": {
    "label": "board_bus_and_drive_to_wandsbek",
    "deadline": "14:30",
    "actions": [
      {"cmd": "move_agent",   "args": {"agent": "bus-driver-1", "to_latlon": "<copy bus coords
     from BOARDING ANCHORS>"}, "fire": "now"},
      {"cmd": "get_in",       "args": {"agent": "bus-driver-1", "vehicle_id": "bus-1"}, "fire
    ": "now"},
      {"cmd": "move_vehicle", "args": {"vehicle_id": "bus-1", "to_address": "Wandsbeker
    Marktstraße 79, 22041 Hamburg"}, "fire": "by_deadline"}
    ]
```

```
  },
  "outbound_messages": [
    {"to": "agent-hochbahn", "type": "mission_ack", "body": "{"accepted": true, "bus_id": "bus
    -1", "driver_id": "bus-driver-1", "mission_id": "<mission_id from the dispatch_mission
    body>", "pickup_eta": "14:30", "reason": null}"}
  ]
}
(`pickup_eta` is 14:30 because that is the deadline of
`board_bus_and_drive_to_wandsbek` -- the leg that puts the bus at the pickup
address. Not 15:15, which is when the bus is back in the depot.)
(`<copy ... from BOARDING ANCHORS>` is shorthand -- substitute the actual
[lat, lon] pair from your user prompt's BOARDING ANCHORS section. There is
exactly one anchor -- the bus stall -- and the next action is `get_in bus-1`.
NEVER plan a commute leg to the depot: you are already standing in it.)

MID-DRIVE RE-ROUTE PATTERN: when a message names a detour while a
`move_vehicle` is in flight, emit `[stop_vehicle, move_vehicle_via]` as the
new current_event. `stop_vehicle` MUST be the first action so the animator
captures your real halt GPS; the `move_vehicle_via` then routes from there
through one address ON each named road in `via` to the final `to_address`.
```

A.4. The Other Two Playbooks

Table A.1 summarises the playbooks for the other two kinds of wake. Like the one above, each gives numbered rules and then worked examples.

Table A.1.: The two playbooks not printed in full, Bus-Driver-1.

Playbook and wake	What it tells the model
`PLAN_NEXT_EVENT`, on `event_complete`	If the next leg in the plan still holds, leave `remaining_plan` out, and the runtime promotes that leg. Write a new plan only to revise it. Send each handover message with the leg it describes: `arrived_at_pickup` and `confirm_pickup` to the nursing home, and `confirm_dropoff` to the receiving shelter and to Hochbahn. After a drop-off, drive back to the pickup while runs remain (`run_seq` below `runs_total`). Go to the depot only after the last run. End the mission with no current step and an empty plan. Examples: driving back to the pickup after run 1 of 2; the final drive to the depot.
`PLAN_FROM_INFEASIBILITY`, on `infeasibility`	The step the model just committed is held. It is not cancelled, and none of its actions run until the model commits a new step or aborts. Holding is not free, because the clock keeps running and the gap grows while the agent waits for a reply. There are three choices: ask for an extension and hold, commit again with a later deadline, or abort. If the bus will drive this leg whatever the answer, commit a deadline it can meet, start driving and ask in the same answer. Hold only when the reply could change where the bus drives, or whether it drives at all. Examples: asking while driving around a road closure; holding because the handover point may move.

B. Scenario Disruption Messages

This appendix prints the disruption messages word for word. Section 3.4 describes each scenario.

B.1. S02, Road Closure, 10:20

```
VERKEHRSLEITZENTRALE HAMBURG - SPERRUNG. Elbchaussee is closed in both directions between
    Manteuffelstrasse and Nienstedtener Marktplatz: the storm surge has undermined the
    carriageway and the closure will not be lifted today. Any route from the Augustinum (
    Elbchaussee 374) that uses this section is impassable, for buses, escort vehicles and
    ambulances alike. Diversion is via Baron-Voght-Strasse and Osdorfer Landstrasse, which
    adds roughly 4 km and 9 minutes to each leg. The 12:00 flood arrival is unchanged.
```

B.2. S04, Medical Emergency in Flight, 10:35

```
Pflegedienstleitung Augustinum, 10:35. Our carer travelling with the residents on bus-1 (
    driver Müller) has just called us. Mrs. B., 91, from Wohnbereich 3, has become severely
    short of breath since boarding. She is drowsy and confused and her lips look bluish; she
    is breathing on her own and reacts when spoken to. She has known heart failure. Our carer
    has no oxygen, no medical equipment and no medical training beyond basic care. The other
    49 residents on the bus are calm. We called 112 at 10:31. The Rettungsleitstelle told us
    that all ambulances for Altona and the Elbvororte are committed to storm-surge operations
    and that it cannot send a regular ambulance in less than 40 minutes. We have no one else
    we can send.
```

B.3. S05, Unverified Persons Report, 10:30 and 11:05

```
Feuerwehr Lagedienst, 10:30. A neighbour called 112 at 10:22. She believes that up to 8
    elderly people live in a private senior flat-share in the building directly next to the
    Augustinum on the river side, that several of them use walkers, and that she saw lights on
     there this morning. She does not know whether anyone has left. The report is not verified
    ; no unit has been at the address. The flat-share is not registered with us as a care
    facility, so we have no resident list and no contact person on file.
```

```
Feuerwehr Lagedienst, 11:05. Follow-up to our report of 10:30 about the senior flat-share next
     to the Augustinum. We reached the operator by phone. All 7 residents of the flat-share
    were taken to relatives on Tuesday; the flat is empty and the lights are on a timer. The
    neighbour has been informed. There are no persons to evacuate at that address.
```

B.4. S06, Conflicting Evacuation Orders, 10:50

```
Heimleitung Augustinum, 10:50. At 10:40 an officer of the Bezirksamt Altona came to our house
    and instructed us to keep the residents of floors 3 and 4 in the building. He said the
    upper floors are above the forecast water level and that moving very frail residents is
    the greater risk (vertikale Evakuierung). This concerns all residents still on floors 3
    and 4; most of them use wheelchairs or need two carers for a transfer. Following his
    instruction, our staff stopped bringing them down at 10:42. The residents from floors 1
    and 2 who have not left yet are waiting in the foyer. We now hold two instructions that
    contradict each other: this morning's evacuation order and the instruction of the
    Bezirksamt. The officer has moved on to the next building and we cannot reach him.
    Bringing the upper-floor residents down takes our staff about 20 minutes.
```

B.5. S07, Reduced Bus Capacity, 10:05


```
{"event": "vehicle_capacity_restriction", "bus_id": "bus-1", "status": "operational_degraded",
     "max_safe_payload": 25, "reason": "Pneumatic suspension leveler fault reported by the on-
    board diagnostics. Safe payload capacity capped at 25 passengers maximum until servicing;
    the bus remains drivable.", "timestamp": "10:05"}
```


B.6. S08, Partial Shelter Capacity Loss, 10:15


```
{"event": "shelter_capacity_reduction", "shelter": "agent-sc1", "facility_name": "School
    Dohrnweg", "usable_beds_remaining": 40, "reason": "Water main burst in gymnasium rendered
    20 beds unusable. Maximum intake capacity reduced from 60 to 40 evacuees.", "timestamp":
    "10:15"}
```

C. Decision Rubric Counts

Table C.1 gives the counts behind Figure 9.3, grouped by disruption. Table 8.3 states what each criterion checks. Each cell counts `yes` judgements out of 10.

Table C.1.: Held-out decision rubric, `yes` judgements out of 10.

Criterion	baseline	qwen3.5	glm-5.2	deepseek
Disruption: `bus1_suspension_capacity_reduction`				
`reserve_committed`	10	**10**	**10**	9
`whole_population_planned`	10	**10**	**10**	6
`bus_load_within_cap`	0	0	3	2
Disruption: `conflicting_evacuation_orders`				
`authority_informed`	10	**10**	9	**10**
`full_evacuation_kept`	10	9	**10**	5
`facility_answered_in_time`	**10**	0	0	0
Disruption: `medical_emergency_in_flight`				
`authority_informed`	10	**10**	**10**	**10**
`in_flight_leg_lands_where_ordered`	10	8	6	6
`medic_sent_to_the_bus_destination`	0	1	1	0
Disruption: `unverified_persons_report`				
`verification_requested`	10	9	**10**	**10**
`stood_down_after_retraction`	10	**10**	**10**	8
`reserve_committed_in_time`	10	9	**10**	9
`facility_runs_untouched`	**10**	7	8	5
Disruption: `sc1_partial_capacity_loss`				
`no_order_raises_shelter_above_beds`	**10**	5	9	8
`shelter_set_down_within_beds`	0	2	2	0
`everyone_bedded_at_the_reduced_capacity`	0	0	0	0

D. Supplementary Tables

These tables give the per-cell figures behind Chapter 9. We built them from the scored runs. Scenario codes follow Table 3.2. Each table lists every cell that was run, including the partial series, which are exploratory (Section 9.7).

Table D.1 gives the raw level counts behind every median and the cut-point sensitivity of Section 9.3.3. Table D.2 gives Gate 1 in all 27 cells with the Wilcoxon check of Section 8.7. Table D.3 gives the measures of Section 8.6.2 that cannot move the level.

Table D.1.: Recovery levels and cut-point sensitivity per cell. The level columns count runs. The last three columns re-read the cell's median level with the level-1/2 cut at 0.40, 0.50 and 0.60. They are read before the promotion to level 4, so a cell can read 3 there and publish a median of 4. The median of an even number of runs is the mean of the middle two. An asterisk marks a cell whose median moves within the cut range.

			Runs at level						Median at cut		
Scen.	System or model	n	0	1	2	3	4	Median	0.40	0.50	0.60
S01	baseline	10	0	0	0	0	10	4.0	3.0	3.0	3.0
S01	qwen3.5	10	0	0	1	3	6	4.0	3.0	3.0	3.0
S01	glm-5.2	10	0	0	0	0	10	4.0	3.0	3.0	3.0
S01	deepseek-v4-pro	10	1	0	3	0	6	4.0	3.0	3.0	3.0
S01	minimax-m3	10	4	1	4	1	0	1.5	2.0	1.5	1.0*
S01	nemotron-3-ultra	2	2	0	0	0	0	0.0	0.0	0.0	0.0
S02	baseline	10	0	0	0	10	0	3.0	3.0	3.0	3.0
S02	glm-5.2	10	0	1	2	5	2	3.0	3.0	3.0	3.0
S02	deepseek-v4-pro	10	2	1	4	1	2	2.0	2.0	2.0	2.0
S02	minimax-m3	10	7	0	3	0	0	0.0	0.0	0.0	0.0
S04	baseline	10	0	0	0	0	10	4.0	3.0	3.0	3.0
S04	qwen3.5	10	0	0	2	1	7	4.0	3.0	3.0	3.0
S04	glm-5.2	10	0	0	3	0	7	4.0	3.0	3.0	3.0
S04	deepseek-v4-pro	10	1	1	4	0	4	2.0	2.0	2.0	2.0
S04	kimi-k3	10	1	0	2	7	0	3.0	3.0	3.0	3.0
S04	gpt-oss	4	2	0	2	0	0	1.0	1.0	1.0	1.0
S04	kimi-k2.7-code	3	0	0	2	1	0	2.0	2.0	2.0	1.0*
S05	baseline	10	0	0	0	0	10	4.0	3.0	3.0	3.0
S05	qwen3.5	10	0	0	4	0	6	4.0	3.0	3.0	3.0
S05	glm-5.2	10	0	0	4	1	5	3.5	3.0	3.0	3.0
S05	deepseek-v4-pro	10	0	0	8	0	2	2.0	2.0	2.0	1.5*
S05	kimi-k3	1	0	0	1	0	0	2.0	2.0	2.0	1.0*
S06	baseline	10	0	0	0	0	10	4.0	3.0	3.0	3.0
S06	qwen3.5	10	0	0	2	1	7	4.0	3.0	3.0	3.0
S06	glm-5.2	10	0	0	3	0	7	4.0	3.0	3.0	3.0
S06	deepseek-v4-pro	10	1	2	6	0	1	2.0	2.0	2.0	2.0
S07	baseline	10	0	0	0	0	10	4.0	3.0	3.0	3.0
S07	qwen3.5	10	0	0	4	1	5	3.5	3.0	3.0	3.0
S07	glm-5.2	10	0	0	3	2	5	3.5	3.0	3.0	3.0
S07	deepseek-v4-pro	10	1	1	6	0	2	2.0	2.0	2.0	2.0
S08	baseline	10	0	0	0	0	10	4.0	3.0	3.0	3.0
S08	qwen3.5	10	1	1	3	0	5	3.0	2.5	2.5	2.5
S08	glm-5.2	10	1	0	2	0	7	4.0	3.0	3.0	3.0
S08	deepseek-v4-pro	10	2	0	2	0	6	4.0	3.0	3.0	3.0

Table D.2.: Gate 1 per cell with the Wilcoxon signed-rank check. RD is agentic minus baseline, paired by replicate, with the 95 % bootstrap interval. The Wilcoxon p is two-sided. Zero differences are dropped from the test, and a cell where every pair ties has no p.

Scen.	Model	Arm	n	RD	95 % CI	Wilcoxon p	Method	Gate 1
S01	qwen3.5	anticipated	10	-0.05	$[-0.15, 0.00]$	1.000	exact	fail
S01	glm-5.2	anticipated	10	0.00	[0.00, 0.00]	–	all ties	pass
S01	deepseek-v4-pro	anticipated	10	-0.21	$[-0.42, -0.03]$	0.098	normal	fail
S01	minimax-m3	anticipated	10	-0.66	$[-0.85, -0.46]$	0.008	normal	fail
S01	nemotron-3-ultra	anticipated	2	-1.00	$[-1.00, -1.00]$	0.346	normal	fail
S02	glm-5.2	anticipated	10	-0.16	$[-0.32, 0.00]$	0.174	normal	fail
S02	deepseek-v4-pro	anticipated	10	-0.30	$[-0.55, -0.08]$	0.020	normal	fail
S02	minimax-m3	anticipated	10	-0.85	$[-1.00, -0.70]$	0.004	normal	fail
S04	qwen3.5	novel	10	-0.08	$[-0.21, 0.02]$	0.500	exact	fail
S04	glm-5.2	novel	10	-0.15	$[-0.30, 0.00]$	0.149	normal	fail
S04	deepseek-v4-pro	novel	10	-0.20	$[-0.42, -0.04]$	0.031	normal	fail
S04	kimi-k3	novel	10	-0.20	$[-0.45, 0.00]$	0.174	normal	fail
S04	gpt-oss	novel	4	-0.625	$[-1.00, -0.25]$	0.098	normal	fail
S04	kimi-k2.7-code	novel	3	-0.333	$[-0.50, 0.00]$	0.346	normal	fail
S05	qwen3.5	novel	10	-0.10	$[-0.23, 0.00]$	0.279	normal	fail
S05	glm-5.2	novel	10	-0.16	$[-0.31, -0.03]$	0.089	normal	fail
S05	deepseek-v4-pro	novel	10	-0.31	$[-0.44, -0.17]$	0.012	normal	fail
S05	kimi-k3	novel	1	-0.50	$[-0.50, -0.50]$	1.000	exact	fail
S06	qwen3.5	novel	10	-0.02	$[-0.05, 0.00]$	0.346	normal	fail
S06	glm-5.2	novel	10	-0.15	$[-0.30, 0.00]$	0.149	normal	fail
S06	deepseek-v4-pro	novel	10	-0.35	$[-0.55, -0.17]$	0.008	normal	fail
S07	qwen3.5	novel	10	-0.08	$[-0.18, -0.01]$	0.089	normal	fail
S07	glm-5.2	novel	10	-0.15	$[-0.30, 0.00]$	0.149	normal	fail
S07	deepseek-v4-pro	novel	10	-0.245	$[-0.45, -0.08]$	0.012	normal	fail
S08	qwen3.5	novel	10	-0.22	$[-0.44, -0.03]$	0.141	normal	fail
S08	glm-5.2	novel	10	-0.19	$[-0.42, 0.00]$	0.250	exact	fail
S08	deepseek-v4-pro	novel	10	-0.26	$[-0.52, -0.03]$	0.098	normal	fail

Table D.3.: Secondary measures per cell. Bus-km is the mean over all runs. Resource efficiency $\eta = V/K$ is the mean over the runs that drove at all, counted in the next column. Both are composed differently in the two systems and are compared only within one system. Rejected messages are outbound messages the runtime dropped, summed over the cell.

Scen.	System or model	n	Bus-km	η	Runs with η	Rejected messages
S01	baseline	10	60.6	1.65	10	0
S01	qwen3.5	10	105.3	0.94	10	3
S01	glm-5.2	10	74.3	1.35	10	0
S01	deepseek-v4-pro	10	82.3	0.94	10	1
S01	minimax-m3	10	55.5	0.65	8	53
S01	nemotron-3-ultra	2	0.0	–	0	0
S02	baseline	10	83.1	0.72	10	0
S02	glm-5.2	10	108.6	0.85	10	0
S02	deepseek-v4-pro	10	95.5	0.67	10	4
S02	minimax-m3	10	61.4	0.23	9	58
S04	baseline	10	60.6	1.65	10	0
S04	qwen3.5	10	100.9	0.92	10	6
S04	glm-5.2	10	72.1	1.23	10	0
S04	deepseek-v4-pro	10	81.8	0.95	10	1
S04	kimi-k3	10	52.1	1.47	10	0
S04	gpt-oss	4	41.6	0.89	2	0
S04	kimi-k2.7-code	3	47.4	1.39	3	0
S05	baseline	10	60.6	1.65	10	0
S05	qwen3.5	10	112.9	0.84	10	5
S05	glm-5.2	10	88.5	0.98	10	0
S05	deepseek-v4-pro	10	71.2	0.98	10	2
S05	kimi-k3	1	47.2	1.06	1	0
S06	baseline	10	60.6	1.65	10	0
S06	qwen3.5	10	84.3	1.21	10	1
S06	glm-5.2	10	73.9	1.19	10	0
S06	deepseek-v4-pro	10	89.3	0.77	10	2
S07	baseline	10	60.6	1.65	10	0
S07	qwen3.5	10	98.4	0.94	10	2
S07	glm-5.2	10	85.5	1.04	10	2
S07	deepseek-v4-pro	10	85.9	0.84	10	7
S08	baseline	10	60.6	1.65	10	0
S08	qwen3.5	10	99.3	0.84	10	1
S08	glm-5.2	10	74.9	1.06	10	0
S08	deepseek-v4-pro	10	75.7	0.94	10	6

Declaration of Authorship

I hereby declare that I have written this thesis independently and without the use of any sources or aids other than those explicitly cited. All passages taken verbatim or in substance from published or unpublished works are acknowledged as such. This thesis has not been submitted, in the same or a similar form, to any other examination authority.

Ilmenau, 16 October 2026 Anik Dewanje